

PROXIAN T ACADEMY

Professional Reference Series

Artificial Intelligence, Machine Learning, and Generative AI

A Comprehensive Textbook for Engineers, Researchers, and Architects

Pavan R

First Edition | 2026

Artificial Intelligence, Machine Learning, and Generative AI

by Pavan R

Published by Proxiant Academy, San Francisco Bay Area.

First Edition, 2026.

Copyright © 2026 Proxiant Academy. All rights reserved.

Reference Identifier (RID): PA-AIMLGEN-2026-01

No part of this work may be reproduced or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in critical reviews and certain other non-commercial uses permitted by copyright law.

While every precaution has been taken in the preparation of this book, the publisher and author assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

Trademarked product names mentioned in this book are the property of their respective owners. The use of such names is for editorial purposes only and does not imply endorsement.

Preface

About This Book

This textbook is a comprehensive guide to modern artificial intelligence: from the mathematical foundations of neural networks through the transformer-based foundation models that define the generative AI era, the engineering practice that puts them into production, and the ethical considerations that should shape that practice.

The book is structured into seven parts and sixteen chapters. Each chapter develops a topic from first principles, illustrates with diagrams and worked examples, grounds the discussion in production engineering practice, and points to further reading for those who want to go deeper.

The audience is engineers, researchers, and technical leaders who want a working command of modern AI. Prerequisites are honest: you should know calculus, linear algebra, and probability at an undergraduate level, plus have working Python skills. Chapter 2 provides a brisk math refresher.

Three principles guide the writing. First, theoretical understanding combined with production engineering produces the most useful work. Second, principles outlast specific tools; the book emphasizes durable principles and uses specific frameworks as illustrations. Third, AI engineering carries real ethical weight; Chapter 16 treats this seriously rather than as an afterthought.

The field moves fast. Specific model names, framework versions, and benchmark numbers will date. The structure of the material, the diagrams, the mathematical relationships, and the engineering patterns will not.

Contents

Table of Contents

Part I Foundations

Chapter 1: Introduction to AI and Modern Machine Learning

Chapter 2: Mathematical Foundations

Chapter 3: Neural Networks and Deep Learning

Part II Natural Language Processing and Transformers

Chapter 4: Classical Natural Language Processing

Chapter 5: The Transformer Architecture

Part III Embeddings and Pretrained Models

Chapter 6: Word and Sentence Embeddings

Chapter 7: Pretrained Language Models

Part IV Vision and Multimodal AI

Chapter 8: Computer Vision and Multimodal Models

Part V Prompts, Fine-Tuning, and Alignment

Chapter 9: Prompt Engineering and Optimization

Chapter 10: Fine-Tuning Large Language Models

Chapter 11: Alignment with Reinforcement Learning

Part VI Retrieval, Vectors, and RAG

Chapter 12: Vector Databases and Approximate Nearest Neighbor Search

Chapter 13: Retrieval-Augmented Generation

Part VII Production: LLMOps, Evaluation, Ethics

Chapter 14: LLMOps and Production System Design

Chapter 15: Evaluation Methodologies

Chapter 16: Ethics, Safety, and the Path Ahead

Part I

Foundations

This part contains chapter(s) 1, 2, 3 covering foundations.

Chapter 1

Introduction to AI and Modern Machine Learning

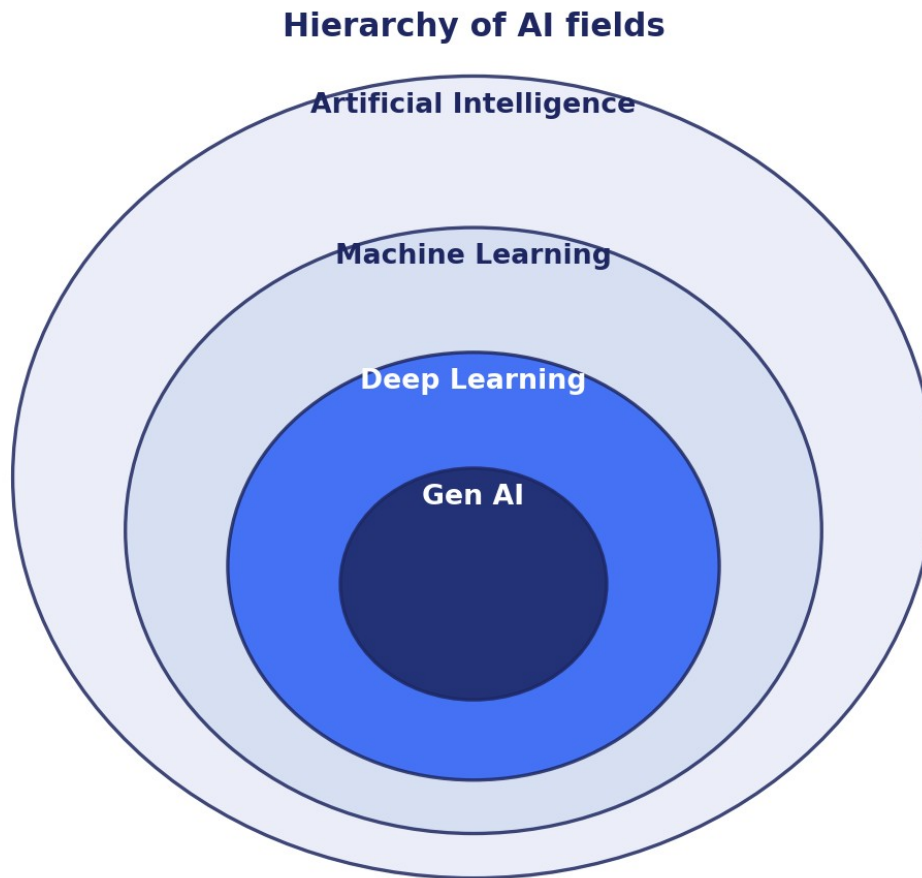


Figure 1.1: Hierarchy of AI subfields.

1.1 What This Book Is About

The field of artificial intelligence has moved through several distinct eras. The first wave was symbolic: rules, knowledge bases, expert systems. The second was statistical: classical machine learning with feature engineering, support vector machines, decision trees, and ensembles. We now live in the third wave, defined by deep learning on massive datasets, transformer-based foundation models, and

generative systems that produce coherent text, images, audio, and video. This book is about that third wave and the engineering practice it has created.

The audience is engineers, researchers, and technical leaders who want a working command of modern AI: how the models are built, how they are trained, how they are deployed, and how they are evaluated. The level of detail is set to support both first-time understanding and reference use. Each chapter introduces a topic from first principles, develops the mathematics where it matters, illustrates with diagrams, and grounds the discussion in production practice.

The book is opinionated about engineering. Many AI texts emphasize theoretical results that turn out to have little bearing on production work. Others focus on framework tutorials that quickly date. This text steers between both. The principles change slowly: gradient descent, attention, retrieval-augmented generation, and the bias-variance tradeoff will outlive any specific framework. We emphasize those principles and use specific tools (PyTorch, Hugging Face, FAISS, vLLM) as illustrations.

1.2 Artificial Intelligence, Machine Learning, and Deep Learning

These terms are often used interchangeably in casual conversation, but they have distinct meanings.

Artificial intelligence is the broad effort to build systems that exhibit behaviors we would call intelligent in humans: reasoning, learning, planning, language understanding, perception. The field includes search algorithms, logical inference, constraint satisfaction, and game-playing systems as well as the statistical methods we discuss in this book.

Machine learning is a subfield of AI focused on systems that improve their behavior from data, rather than being explicitly programmed. A spam filter that learns from labeled examples, a recommendation system that learns from user interactions, and a stock-prediction model that learns from historical prices are all machine learning systems.

Deep learning is a subfield of machine learning that uses neural networks with many layers. The 'deep' refers to the depth of the network. Deep learning has driven nearly every breakthrough in AI over the last decade: image recognition, speech recognition, machine translation, and the large language models that have captured public attention.

Generative AI is a class of applications, not a separate field. It uses deep learning models (transformers, diffusion models) trained on massive datasets to produce novel outputs: text, images, audio, code. The same underlying models also support understanding tasks such as classification and retrieval.

[Definition] A model is a function $f(x; \theta)$ parameterized by θ that maps inputs x to outputs y . Training is the process of adjusting θ to make the model's predictions match observed data.

1.3 Three Learning Paradigms

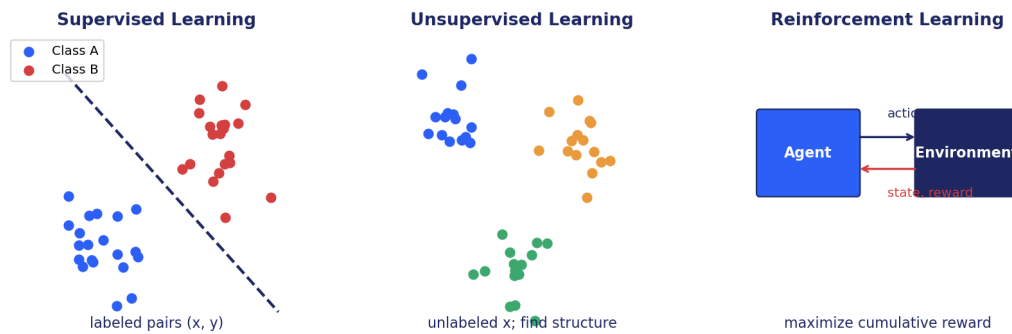


Figure 1.2: Supervised, unsupervised, and reinforcement learning.

Machine learning problems fall into three broad paradigms based on what kind of feedback the learner receives.

1.3.1 Supervised Learning

In supervised learning, training data consists of paired examples (x, y) , where x is an input and y is a target. The model learns a function that maps inputs to targets. Common subtypes:

Classification: y is a discrete label (spam vs not spam, image class). Regression: y is a continuous number (house price, temperature). Structured prediction: y is itself structured (a sequence, a tree, a graph), as in machine translation or named entity recognition.

Supervised learning is the most mature and widely deployed paradigm. It powers most production ML systems. Its main practical constraint is labeled data: humans must annotate examples for the model to learn from, which is expensive at scale.

1.3.2 Unsupervised Learning

In unsupervised learning, the data consists only of inputs x with no targets. The goal is to discover structure: clusters, low-dimensional embeddings, generative models of the data distribution.

Examples include k-means clustering for customer segmentation, principal component analysis for dimensionality reduction, topic models for document collections, autoencoders for learned representations, and generative models such as GANs and diffusion models.

A particularly important variant is self-supervised learning, in which labels are derived from the data itself. Predicting masked tokens in a sentence (BERT), predicting the next token (GPT), or predicting an image patch from surrounding patches (masked autoencoders) all create supervised signals without human annotation. Self-supervised learning is the foundation of modern foundation models.

1.3.3 Reinforcement Learning

In reinforcement learning (RL), an agent interacts with an environment over time. The agent observes a state, takes an action, and receives a reward plus a new state. The agent's goal is to learn a policy (a function from states to actions) that maximizes cumulative reward.

RL has produced spectacular results in games (AlphaGo, AlphaStar), robotics, and resource allocation. In language modeling, RL appears in the form of Reinforcement Learning from Human Feedback (RLHF) and its successors (DPO, GRPO), used to align large language models with human preferences.

RL is harder to scale than supervised learning because rewards are often sparse, environments are expensive to simulate, and exploration is tricky. But it is uniquely powerful for problems where the right answer is not known in advance and must be discovered through interaction.

1.4 The Bias-Variance Tradeoff

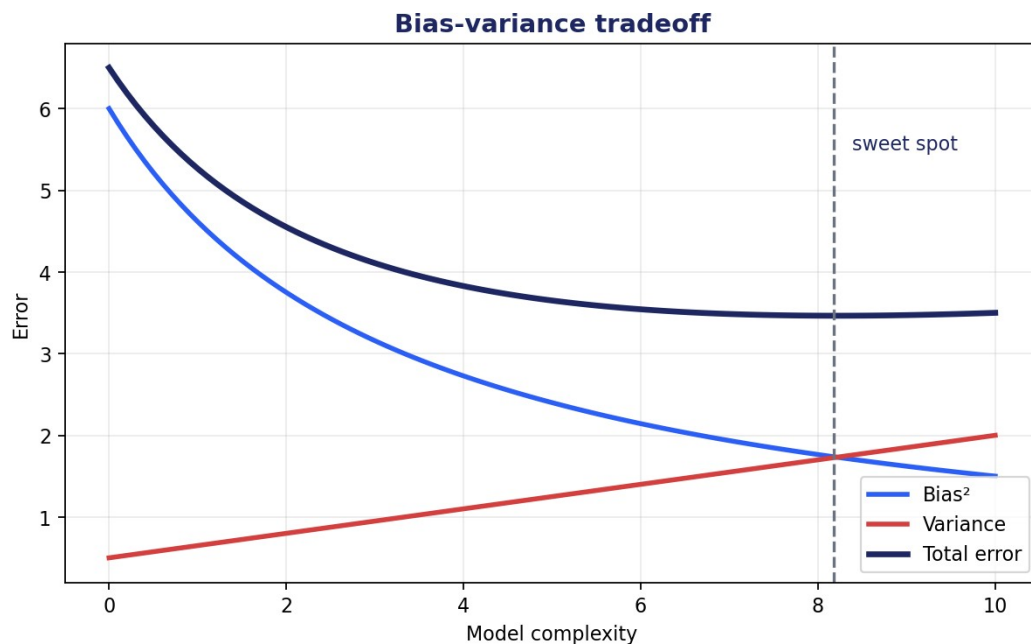


Figure 1.3: As model complexity grows, bias falls but variance rises.

Almost every practical decision in machine learning involves a tradeoff between underfitting (model too simple to capture the truth) and overfitting (model so complex it memorizes noise). The bias-variance decomposition makes this tradeoff precise.

For a regression model trained on a finite dataset, the expected squared error on a test point decomposes into three terms: bias squared (how far the model's average prediction is from the true value), variance (how much the prediction varies across training sets), and irreducible noise (inherent randomness in the data).

$$E[(y - \hat{h}(x))^2] = \text{Bias}^2[\hat{h}(x)] + \text{Variance}[\hat{h}(x)] + \sigma^2 \quad (1.1)$$

1.4.1 Implications for Model Selection

Simple models (linear regression, shallow trees) tend to have high bias and low variance. They make the same kinds of errors on every training set but cannot capture complex patterns. Complex models (deep neural networks, deep trees) have low bias but high variance. They can fit almost any training set but their predictions on new data are sensitive to which training set they happened to see.

The right model complexity depends on data size. With little data, simple models often win because their lower variance outweighs their higher bias. With abundant data, complex models win because their variance shrinks as data grows. This is why deep learning surged once datasets reached the millions-of-examples scale.

Regularization techniques such as L2 penalty, dropout, early stopping, and data augmentation reduce variance without giving up too much expressiveness. We will use these throughout the book.

1.5 A Brief History of Modern AI

The modern era of deep learning began around 2012, when Krizhevsky, Sutskever, and Hinton's AlexNet won the ImageNet image-classification challenge by a wide margin using a convolutional neural network trained on GPUs. The result convinced the field that deep neural networks, given enough data and compute, could outperform decades of hand-crafted feature engineering.

The next several years saw rapid progress in computer vision (ResNet, Inception), in sequence modeling (LSTM-based encoder-decoder models for translation), and in generative models (GANs by Goodfellow et al., 2014; VAEs by Kingma and Welling, 2013).

In 2017, the paper 'Attention Is All You Need' by Vaswani et al. introduced the transformer architecture, replacing recurrence with self-attention. This proved to be the foundational architecture for the LLM era. BERT (2018) showed that pretraining a bidirectional transformer on large text corpora produced representations that transferred to many downstream tasks. GPT (2018, 2019, 2020) showed that scaling up a decoder-only transformer trained on next-token prediction produced general-purpose language generators.

GPT-3 (2020) showed in-context learning at scale, the ability to learn new tasks from a few examples in the prompt. ChatGPT (2022) brought instruction-tuned LLMs to public attention and triggered the current wave of generative AI adoption. GPT-4 (2023), Claude, Gemini, LLaMA, and Mistral followed, each pushing the frontier of capability while open-source alternatives became competitive with closed models for many tasks.

In parallel, image generation went through its own revolution. DALL-E and Stable Diffusion brought text-to-image generation to consumers. Diffusion models displaced GANs as the dominant generative architecture for images. Multimodal models such as CLIP, Flamingo, GPT-4V, and Gemini integrated vision and language. The convergence of all modalities into single foundation models is the current frontier.

1.6 How to Read This Book

The book is organized into six parts. Part I covers foundations: the mathematics, neural networks, and training procedures that underpin everything that follows. Part II covers natural language processing from classical techniques through the transformer revolution. Part III covers computer vision and multimodal models. Part IV covers large language models in depth: prompt engineering, fine-tuning, alignment. Part V covers retrieval and knowledge systems including vector databases and RAG. Part VI covers production engineering, evaluation, and ethics.

Chapters are self-contained enough to read out of order if a topic interests you, but they cross-reference freely. The prerequisites are honest: you should know calculus, linear algebra, and probability at an undergraduate level, plus have working Python skills. Chapter 2 provides a brisk refresher of the math; skim if you are comfortable, study carefully if you are not.

Each chapter ends with a summary, suggested exercises, and pointers to deeper reading. The bibliography is selective rather than exhaustive: we point to the papers and books that we believe most repay study, not all the literature on a topic.

1.7 Summary

- Artificial intelligence is the broad effort to build intelligent systems; machine learning is the subfield that learns from data; deep learning is the machine learning subfield based on deep neural networks; generative AI is the application class focused on producing novel outputs.
- Three learning paradigms: supervised (labeled data), unsupervised (no labels), and reinforcement (interaction with reward).
- The bias-variance tradeoff governs model selection. Simple models underfit; complex models overfit. The right complexity depends on data size.
- Modern AI runs on transformer-based foundation models pretrained on massive corpora, adapted to downstream tasks via prompting, fine-tuning, or retrieval-augmented generation.
- The book takes a principles-first approach grounded in engineering practice.

Further Reading

- Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4th ed., 2020). Comprehensive AI textbook covering symbolic and statistical methods.
- Goodfellow, Bengio, Courville, *Deep Learning* (2016). Authoritative deep learning text.
- Bishop, *Pattern Recognition and Machine Learning* (2006). Classical ML grounded in Bayesian probability.
- Murphy, *Probabilistic Machine Learning* (2022, 2023). Modern two-volume treatment of ML from a probabilistic perspective.

Chapter 2

Mathematical Foundations

2.1 Why Mathematics Matters

Modern deep learning is, at its core, applied calculus and linear algebra over large datasets. A working command of the math is the difference between treating neural networks as a black box and being able to reason about why a training run diverges, why one architecture works better than another for a given task, or how to design a new objective function. This chapter is a brisk refresher of the material you need. If you are already comfortable, skim. If you are not, study it carefully. We will lean on these ideas throughout the book.

2.2 Linear Algebra

Linear algebra is the language of high-dimensional data. Vectors represent data points, weights, embeddings. Matrices represent transformations, datasets, attention patterns. Tensors generalize to higher dimensions.

2.2.1 Vectors and Matrices

A vector is an ordered list of numbers. We write $x \in \mathbb{R}^n$ to mean x is a vector with n real-valued entries. A matrix $A \in \mathbb{R}^{m \times n}$ has m rows and n columns. Matrix multiplication: $(AB)_{ij} = \sum_k A_{ik} B_{kj}$. Defined only when the inner dimensions match.

The transpose A^T swaps rows and columns. The identity matrix I has ones on the diagonal and zeros elsewhere. The inverse A^{-1} satisfies $AA^{-1} = I$ (defined only for square non-singular A). The determinant $\det(A)$ measures how A scales volume; $\det(A) = 0$ means A is singular.

2.2.2 Norms and Distances

The L^2 (Euclidean) norm of a vector: $\|x\|_2 = \sqrt{\sum_i x_i^2}$. The L^1 norm: $\|x\|_1 = \sum_i |x_i|$. The L^∞ norm: $\|x\|_\infty = \max_i |x_i|$. Norms induce distances: $d(x, y) = \|x - y\|$.

Cosine similarity measures the angle between two vectors: $\cos(x, y) = (x \cdot y) / (\|x\|_2 \|y\|_2)$. This is the foundational similarity measure for embeddings: two embeddings are similar in meaning if their cosine similarity is close to 1.

2.2.3 Eigenvalues, Eigenvectors, and Decompositions

For a square matrix A , an eigenvector v satisfies $Av = \lambda v$ for some scalar λ (the eigenvalue). Eigenvalues reveal stretching and rotation behavior of the matrix.

The singular value decomposition (SVD) writes any matrix A as $A = U\Sigma V^T$, where U and V are orthogonal and Σ is diagonal with non-negative entries (singular values). SVD underlies principal component analysis, low-rank approximation (used in LoRA), and embedding methods. The rank of A is the number of non-zero singular values.

Low-rank approximation: by keeping only the top k singular values, you approximate A with much fewer parameters. This is the mathematical foundation of LoRA (low-rank adaptation): the weight update during fine-tuning is approximately low-rank, so we can represent it as the product of two small matrices.

2.3 Calculus and Optimization

Training neural networks is optimization: find parameters that minimize a loss function. The basic tool is the derivative.

2.3.1 Derivatives and Gradients

For a function $f: \mathbb{R} \rightarrow \mathbb{R}$, the derivative $f'(x)$ is the rate of change at x . For a function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, the gradient $\nabla f(x)$ is the vector of partial derivatives: $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$. The gradient points in the direction of steepest increase.

Gradient descent updates parameters by moving in the opposite direction of the gradient: $\theta \leftarrow \theta - \eta \nabla L(\theta)$. The learning rate η controls the step size.

2.3.2 The Chain Rule

If $y = f(u)$ and $u = g(x)$, then $dy/dx = (dy/du) \cdot (du/dx)$. The chain rule extends to vector-valued functions and to compositions of many functions. Backpropagation is the application of the chain rule to a computational graph (the neural network), computing gradients of the loss with respect to every parameter by composing local derivatives from output back to input.

2.3.3 Convex and Non-Convex Optimization

A function is convex if its graph lies below any chord between two points. Convex optimization problems have a single global minimum and are well-understood. Most classical machine learning (linear regression with L2 loss, logistic regression, SVMs) is convex.

Neural network optimization is non-convex: the loss surface has many local minima, saddle points, and flat regions. In high dimensions, the empirical observation is that most local minima have similar loss values, and saddle points (not local minima) are the main obstacle. Modern optimizers like Adam, AdamW, and momentum-based SGD handle these landscapes well in practice, even though we lack the strong theoretical guarantees of convex optimization.

2.4 Probability and Statistics

Machine learning is fundamentally probabilistic: models predict distributions, training data is sampled from distributions, and loss functions correspond to likelihoods.

2.4.1 Random Variables and Distributions

A random variable X takes values from a probability distribution. Discrete distributions are described by a probability mass function $P(X = x)$; continuous distributions by a probability density function $p(x)$. Common distributions include Gaussian (normal), Bernoulli, Categorical, Uniform, and Exponential.

Expectation: $E[X] = \sum x P(X = x)$ for discrete X , or $\int x p(x) dx$ for continuous X . Variance: $\text{Var}(X) = E[(X - E[X])^2]$. These two moments characterize many practical distributions.

2.4.2 Joint, Marginal, and Conditional

Two random variables X, Y have a joint distribution $P(X, Y)$. The marginal $P(X)$ is obtained by summing or integrating out Y . The conditional $P(Y | X)$ tells you how Y is distributed given a particular value of X . Bayes' rule connects them: $P(Y | X) = P(X | Y) P(Y) / P(X)$.

These concepts underpin all of generative modeling. A discriminative model learns $P(Y | X)$. A generative model learns $P(X, Y)$ or $P(X)$, and can sample new examples.

2.4.3 Maximum Likelihood Estimation

Given a parameterized distribution $P(x; \theta)$ and observed data $\{x_1, \dots, x_n\}$, the maximum likelihood estimate (MLE) is the choice of θ that maximizes the likelihood: $\hat{\theta} = \text{argmax}_{\theta} \prod_i P(x_i; \theta)$. Taking logarithms (which preserves the argmax): $\hat{\theta} = \text{argmax}_{\theta} \sum_i \log P(x_i; \theta)$.

Neural network training is almost always MLE in disguise. Cross-entropy loss is the negative log-likelihood of a categorical model. MSE loss is the negative log-likelihood of a Gaussian model. Maximizing likelihood = minimizing negative log-likelihood = training the model with the appropriate loss.

2.4.4 KL Divergence and Cross-Entropy

The Kullback-Leibler divergence between two distributions P and Q is $D_{\text{KL}}(P || Q) = \sum P(x) \log(P(x) / Q(x))$. It measures how much Q differs from P . It is non-negative, zero iff $P = Q$, and asymmetric ($D_{\text{KL}}(P || Q) \neq D_{\text{KL}}(Q || P)$).

Cross-entropy $H(P, Q) = -\sum P(x) \log Q(x)$. It decomposes as $H(P, Q) = H(P) + D_{\text{KL}}(P || Q)$. In classification with one-hot true labels P , $H(P) = 0$, so cross-entropy reduces to $-\log Q(y_{\text{true}})$. This is the standard classification loss.

2.5 Information Theory

Entropy $H(X) = -\sum P(x) \log P(x)$ measures uncertainty in a distribution. A uniform distribution has maximum entropy (log of the number of outcomes). A concentrated distribution has low entropy.

In language modeling, perplexity = $\exp(\text{cross-entropy per token})$ measures how uncertain the model is at each step. Perplexity of 10 means the model is, on average, as uncertain as if choosing uniformly from 10 options. Lower is better; modern LLMs achieve perplexity in the single digits on standard English benchmarks.

Mutual information $I(X; Y) = H(X) + H(Y) - H(X, Y)$ measures shared information. It appears in representation learning objectives such as InfoNCE.

2.6 High-Dimensional Geometry

Human intuition for two- and three-dimensional space misleads in higher dimensions. Three counter-intuitive facts that matter for embeddings:

Concentration of measure. In a high-dimensional unit ball, almost all volume lies near the surface. Equivalently, almost all of a Gaussian's probability mass lies in a thin shell at a specific distance from the mean.

Random vectors are nearly orthogonal. In d dimensions, two independently sampled random unit vectors have expected cosine similarity that tends to 0 as d grows.

Distance concentration. The ratio of maximum to minimum pairwise distances among a finite set of points approaches 1 as dimension grows. 'Nearest neighbor' loses meaning.

These facts have practical consequences. Embedding quality requires explicit training (contrastive learning) to keep distances meaningful. Anisotropy in pretrained BERT embeddings, where all embeddings cluster in a narrow cone, is a symptom of these effects. Approximate nearest neighbor search trades a small recall loss for orders of magnitude speedup because exact search in high dimensions is fundamentally hard.

2.7 Summary

- Linear algebra provides the data structures (vectors, matrices, tensors) and transformations (matrix products, SVD) used throughout deep learning.
- Calculus and the chain rule are the foundation of backpropagation; gradient-based optimization is how every modern deep model is trained.
- Probability and statistics frame learning as maximum likelihood; cross-entropy and KL divergence are the standard loss functions.
- Information theory connects probability to model quality via entropy and perplexity.
- High-dimensional geometry has counter-intuitive properties that shape embedding methods and similarity search.

Further Reading

- Strang, Introduction to Linear Algebra (5th ed.). Accessible LA reference.
- Boyd and Vandenberghe, Convex Optimization (2004). Free PDF; the standard reference.
- MacKay, Information Theory, Inference, and Learning Algorithms (2003). Free PDF; elegant integration of information theory and ML.

Chapter 3

Neural Networks and Deep Learning

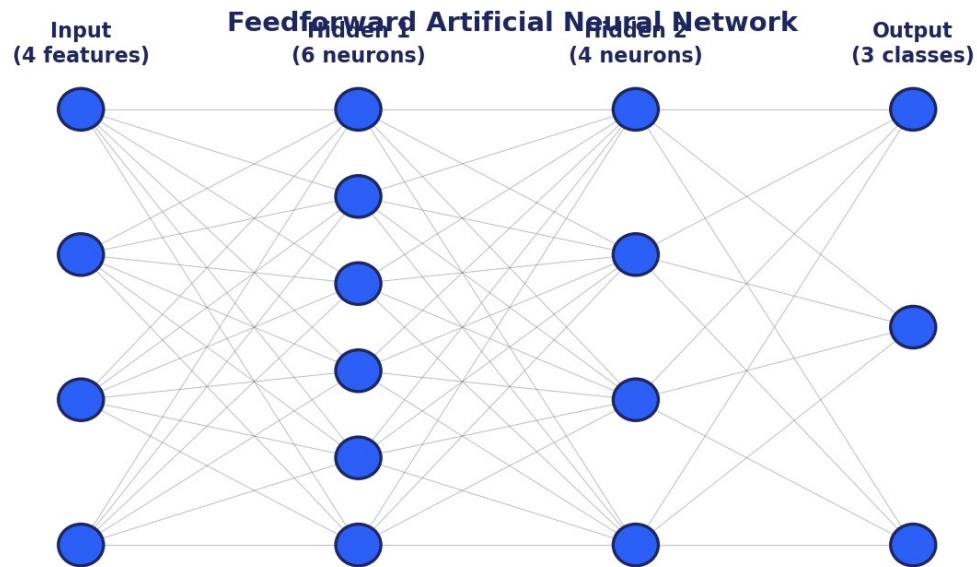


Figure 3.1: A feedforward neural network with input, hidden, and output layers.

3.1 The Artificial Neuron

Single neuron: weighted sum, bias, activation

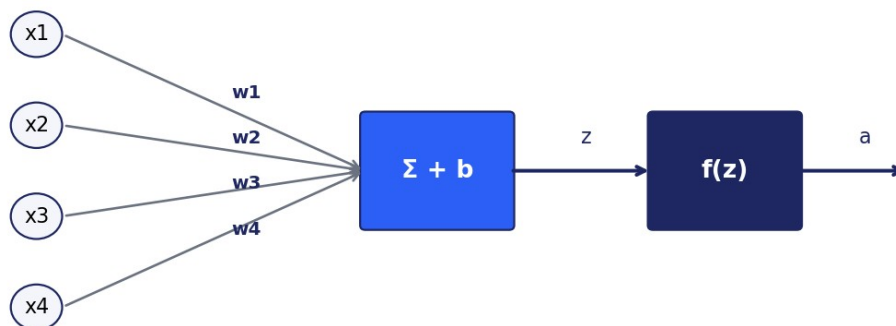


Figure 3.2: A single artificial neuron computes a weighted sum plus bias, then applies an activation function.

The basic unit of a neural network is an artificial neuron. Despite its biological inspiration, the artificial neuron is mathematically simple: take a weighted sum of inputs, add a bias, apply a non-linear function.

Given input vector $x \in \mathbb{R}^n$, weights $w \in \mathbb{R}^n$, and bias $b \in \mathbb{R}$, a neuron computes the pre-activation $z = w^T x + b$, then applies an activation function f to produce the output $a = f(z)$. With a layer of m neurons, the weights become a matrix $W \in \mathbb{R}^{m \times n}$, and the layer computes $a = f(Wx + b)$ where f is applied element-wise.

What makes a neuron useful is the non-linear activation function. Without it, stacking layers would only compose linear transformations into another linear transformation, with no gain in expressive power. Non-linearity is what lets deep networks model complex relationships.

3.2 Activation Functions

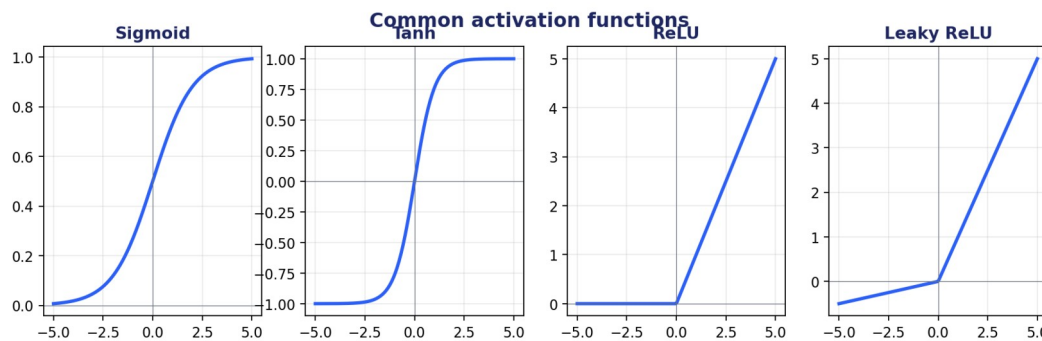


Figure 3.3: Common activation functions: sigmoid, tanh, ReLU, leaky ReLU.

The choice of activation function shapes both expressiveness and trainability.

3.2.1 Saturating Activations: Sigmoid and Tanh

The sigmoid function $\sigma(x) = 1 / (1 + e^{-x})$ maps any real value to $(0, 1)$. Historically popular because the output can be interpreted as a probability. It has two problems: it saturates (gradient vanishes when $|x|$ is large), and the output is not zero-centered.

The hyperbolic tangent $\tanh(x)$ maps to $(-1, 1)$ and is zero-centered, slightly mitigating the saturation issue. Both sigmoid and tanh are still used in specific places (sigmoid for binary classification output, LSTM gates) but rarely in modern hidden layers.

3.2.2 ReLU and Variants

The Rectified Linear Unit $\text{ReLU}(x) = \max(0, x)$ is the dominant choice for hidden layers in modern deep networks. It is cheap to compute, does not saturate on the positive side, and produces sparse activations (many neurons output exactly zero).

ReLU has a 'dead neuron' problem: once a neuron's weighted input is consistently negative, it always outputs zero and its gradient is zero, so it stops learning. Leaky ReLU (slope α for negative inputs, typically 0.01) and Parametric ReLU (learn α) address this.

More recent variants include GELU (Gaussian Error Linear Unit), used in BERT and most modern transformers, and SiLU (Sigmoid Linear Unit, also called Swish), used in LLaMA. Both are smoother than ReLU and tend to produce slightly better results at similar compute.

3.2.3 Output Activations

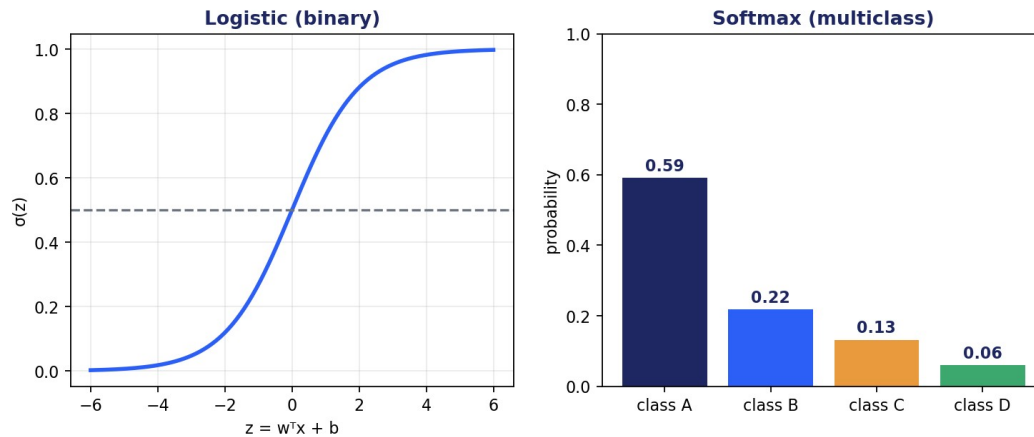


Figure 3.4: Logistic for binary outputs (left), softmax for multiclass (right).

For binary classification, the output layer uses a single neuron with sigmoid activation, producing a probability in $(0, 1)$. For multiclass classification with K classes, the output is a vector of K logits that passes through softmax: $\text{softmax}(z)_i = \exp(z_i) / \sum_j \exp(z_j)$. The result is a probability distribution over classes.

For regression, the output is typically linear (no activation), unless the target is bounded (use sigmoid for $[0, 1]$ outputs, tanh for $[-1, 1]$).

3.3 Feedforward Networks

A feedforward network is a stack of fully connected layers. Each layer takes the output of the previous layer, applies a learned linear transformation, and passes the result through a non-linear activation. The final layer produces the output.

Architectural choices: number of layers (depth), number of neurons per layer (width), activation function. Depth provides hierarchical composition: lower layers learn simple features, higher layers compose them into complex ones. Width provides capacity within a layer.

The universal approximation theorem states that a sufficiently wide single-hidden-layer network can approximate any continuous function on a compact set. But it says nothing about how easy that

approximation is to learn. In practice, depth is far more parameter-efficient than width: a deep network with modest width can express functions that would require exponentially many wide-shallow neurons.

3.4 Training: Backpropagation and Gradient Descent

Backpropagation: forward then backward

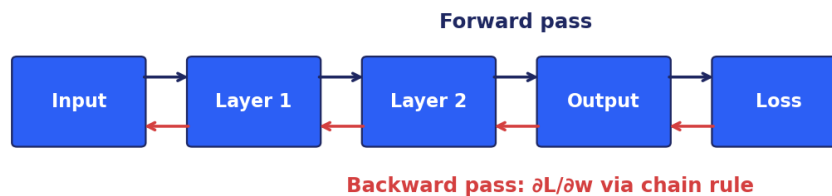


Figure 3.5: Forward pass computes the prediction; backward pass propagates gradients.

Training a neural network is solving an optimization problem: find weights and biases that minimize a loss function on training data. Modern training uses backpropagation to compute gradients and stochastic gradient descent (or a variant) to update parameters.

3.4.1 Forward Pass

The forward pass evaluates the network on an input. Starting from the input layer, each layer applies its transformation and passes the result to the next. Intermediate activations are cached for use in the backward pass.

The final output is compared to the target using a loss function. Common choices: mean squared error for regression, cross-entropy for classification.

3.4.2 Backward Pass

The backward pass computes the gradient of the loss with respect to every parameter in the network. It does so by applying the chain rule from output backwards.

Concretely, suppose the loss is L and the network has layers $L_1, L_2, \dots, L_n$ producing outputs $a_1, a_2, \dots, a_n$. The chain rule gives $\partial L / \partial W_k = (\partial L / \partial a_n)(\partial a_n / \partial a_{n-1}) \dots (\partial a_{k+1} / \partial a_k)(\partial a_k / \partial W_k)$. Backpropagation computes these products efficiently by reusing intermediate results.

Cost: backpropagation takes roughly the same time as the forward pass. This is what makes deep network training tractable. Modern frameworks (PyTorch, JAX, TensorFlow) handle backpropagation automatically via computational graphs and reverse-mode automatic differentiation.

3.4.3 Gradient Descent and Variants

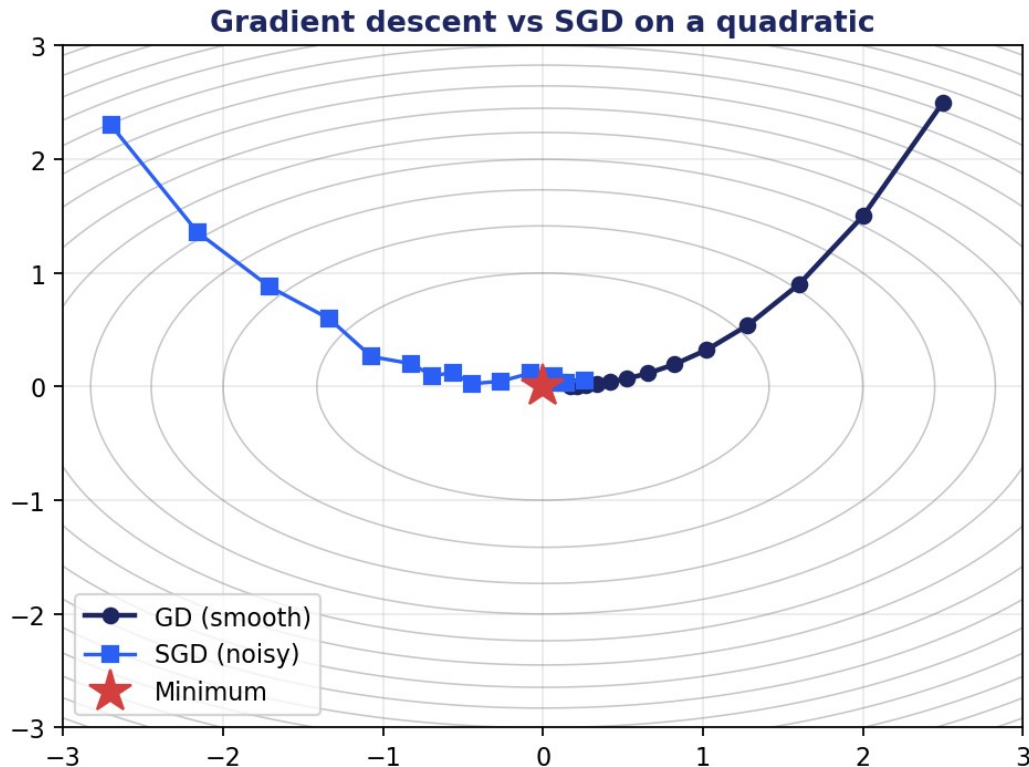


Figure 3.6: Gradient descent on a quadratic surface vs noisy SGD.

The update rule for gradient descent is $\theta \leftarrow \theta - \eta \nabla L(\theta)$. Three variants based on how many examples contribute to each gradient estimate:

Batch gradient descent uses the entire training set per step. Smooth convergence; each step is expensive; the dataset must fit in memory.

Stochastic gradient descent (SGD) uses one example per step. Cheap and noisy; the noise can help escape sharp local minima but slows fine convergence.

Mini-batch gradient descent is the practical compromise: a small batch (typically 32 to 4096) per step. GPU-friendly, noise-controlled, and the default for nearly all modern training.

Modern optimizers add momentum and adaptive learning rates. Momentum averages recent gradients, smoothing noisy updates and accelerating through ravines. Adam (Adaptive Moment Estimation) combines momentum with per-parameter adaptive learning rates and is the default for most deep learning workloads. AdamW decouples weight decay from the gradient update and is preferred when regularization matters.

3.5 Training Pathologies

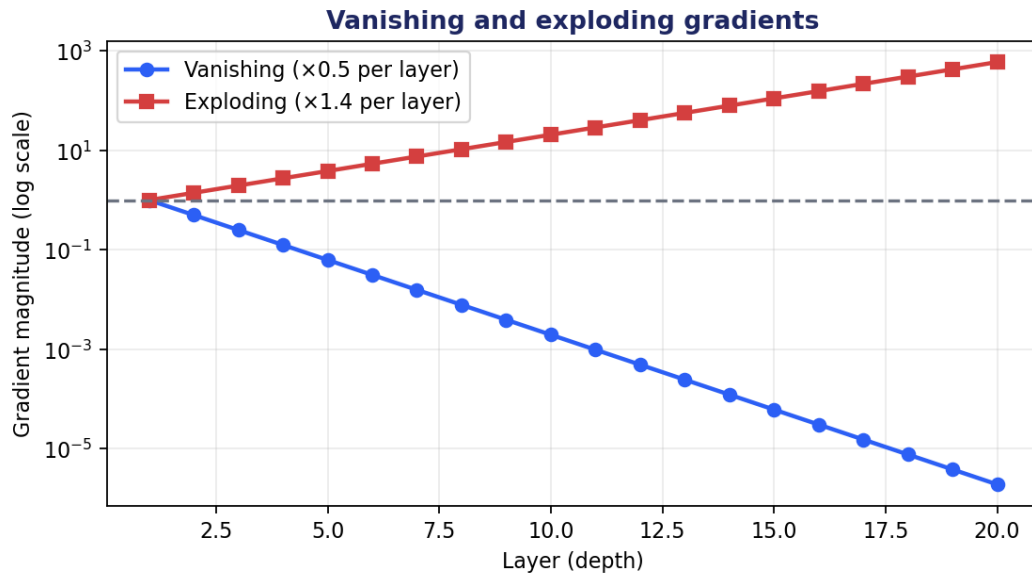


Figure 3.7: Gradient magnitude across layers under vanishing and exploding conditions.

Deep networks suffer characteristic training problems that simpler models do not.

Vanishing gradients: when gradients are repeatedly multiplied by small numbers (as happens with saturating activations or poorly initialized weights), the gradient that reaches early layers becomes negligible. Those layers stop learning. Symptom: training loss plateaus far above optimal.

Exploding gradients: when gradients are repeatedly multiplied by large numbers, they blow up. Weight updates become huge. Training diverges or produces NaN values. Common in RNNs unrolled over many steps.

Mitigations: use ReLU-family activations (derivative is 1 on the active side); use careful weight initialization (Xavier/Glorot for tanh, He for ReLU); apply batch normalization or layer normalization; use residual connections (let gradients flow through identity paths); clip gradients (for explosion in RNNs and RL); use modern architectures (LSTM/GRU, transformers) designed to keep gradients healthy.

3.6 Regularization

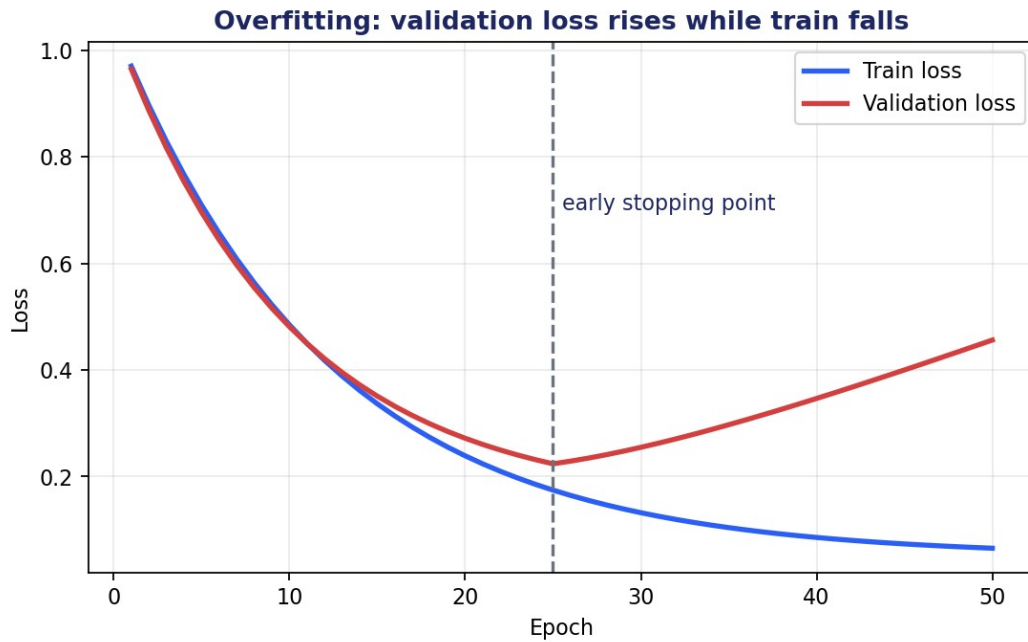


Figure 3.8: Overfitting shows as widening gap between train and validation loss.

A model with enough capacity will memorize the training set. The goal is to generalize. Regularization techniques reduce variance without sacrificing too much expressive power.

3.6.1 Weight Penalties

L2 regularization (weight decay) adds $\lambda \sum_i w_i^2$ to the loss. The optimizer pulls weights toward zero in proportion to their magnitude. Encourages distributed representations rather than relying on a few large weights.

L1 regularization adds $\lambda \sum_i |w_i|$. Encourages sparsity: many weights go to exactly zero. Useful when sparse models are desired (feature selection, compressed deployment).

3.6.2 Dropout

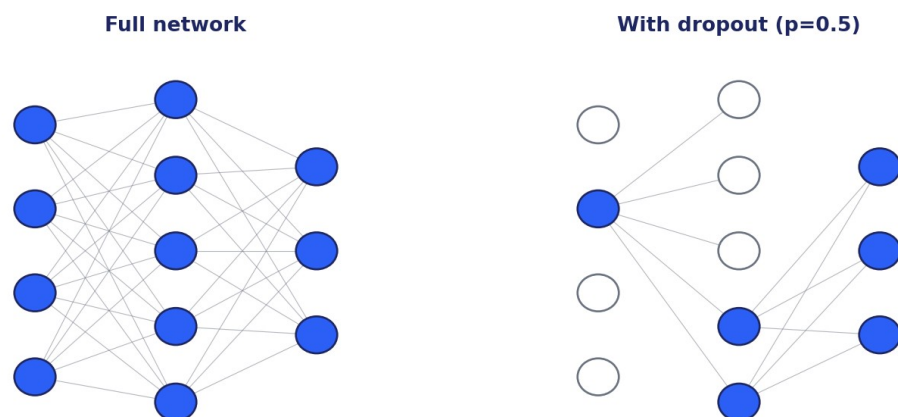


Figure 3.9: Dropout randomly removes neurons during training.

Dropout randomly zeros each neuron's output with probability p during training. At test time, all neurons are active. This prevents co-adaptation: each neuron must produce useful features regardless of which other neurons happen to be active in any given batch.

Dropout can be interpreted as training an ensemble of subnetworks and averaging them at inference. Typical p : 0.5 for fully connected, 0.1 to 0.3 for transformer blocks. Modern transformer LLMs rely more on data scale and weight decay than aggressive dropout.

3.6.3 Batch and Layer Normalization

Batch normalization (BN) normalizes the activations of each feature across the batch dimension. It stabilizes training, allows larger learning rates, and acts as mild regularization. Works well in CNNs with large batches.

Layer normalization (LN) normalizes across the feature dimension within each example. No batch dependence. The default in transformers because attention with variable-length sequences breaks BN's assumptions.

RMSNorm is a recent simplification of LN: drop the mean-centering, keep only the variance normalization. Used in LLaMA and other modern LLMs. Slightly faster, comparable quality.

3.6.4 Data Augmentation and Early Stopping

Data augmentation creates additional training examples by applying transformations that preserve the label: random crops and flips for images, back-translation and synonym substitution for text. The model sees more variation; generalization improves.

Early stopping monitors validation loss during training and stops when it begins to rise. Simple and effective. Many modern training frameworks include early stopping as a default callback.

3.7 Convolutional Neural Networks

Typical CNN pipeline for image classification

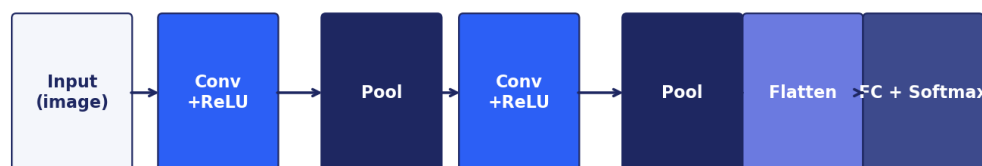


Figure 3.10: A typical CNN for image classification.

Fully connected networks ignore the spatial structure of images. A CNN encodes two priors that match how images work: local receptive fields (each neuron sees only a small spatial neighborhood) and weight sharing (the same filter is applied at every spatial position).

A convolutional layer slides a small filter across the input and computes a dot product at each position. The result is a feature map indicating where the filter pattern appears. Multiple filters per layer detect different patterns. Stacking layers builds hierarchy: edges in lower layers, textures and parts in middle layers, objects in upper layers.

Pooling layers downsample feature maps, reducing spatial dimensions and providing translation invariance. Max pooling takes the maximum in each region; average pooling takes the mean. Modern architectures often use strided convolutions in place of explicit pooling.

Famous architectures: LeNet (1998, digit recognition), AlexNet (2012, ImageNet breakthrough), VGG (2014, deep stacks of 3x3 convolutions), ResNet (2015, residual connections enabling very deep networks), EfficientNet, ConvNeXt. Vision Transformers (ViT) since 2020 have challenged CNNs at scale, though CNNs remain competitive and dominant at smaller scales.

3.8 Recurrent Neural Networks

Recurrent Neural Network unrolled in time

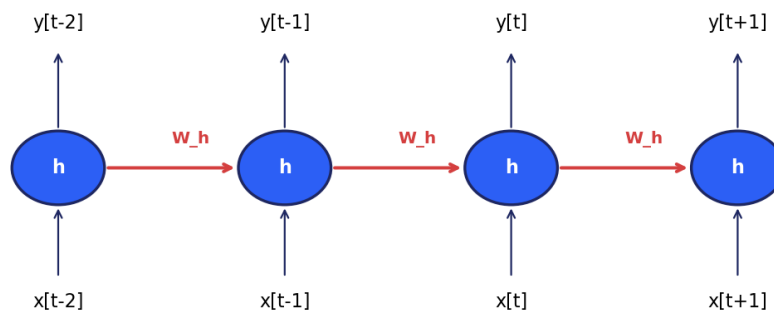


Figure 3.11: RNN unrolled in time; the hidden state carries information forward.

RNNs process sequences one element at a time, maintaining a hidden state that summarizes everything seen so far. Each step: $h_t = f(W_h h_{t-1} + W_x x_t + b)$. Outputs can be produced at each step (sequence labeling) or only at the end (classification).

Vanilla RNNs suffer from vanishing gradients over long sequences. LSTMs (Long Short-Term Memory) add input, forget, and output gates plus a cell state, allowing the network to learn what to remember and what to forget over long spans. GRUs (Gated Recurrent Units) are a simpler variant with two gates and no separate cell state.

RNNs dominated sequence modeling until transformers replaced them around 2018. They are still used where transformer overhead is excessive (very short sequences, embedded systems) and as components of larger architectures (mixed RNN-attention models, RWKV).

3.9 Summary

- A neural network is a parameterized function built from layers of artificial neurons connected by learned weights.
- Non-linear activations (ReLU and variants) are what give deep networks their expressive power.
- Training is gradient descent driven by backpropagation, which computes gradients through the chain rule.
- Vanishing and exploding gradients are the main training pathologies; modern architectures use normalization and residual connections to control them.
- Regularization (weight decay, dropout, normalization, augmentation, early stopping) trades a small amount of training fit for substantially better generalization.
- Convolutional networks encode spatial structure; recurrent networks encode temporal structure. Both are largely superseded by transformers for state-of-the-art results, but remain widely used in production.

Further Reading

- Goodfellow, Bengio, Courville, Deep Learning (2016), Chapters 6-10.
- Krizhevsky, Sutskever, Hinton, 'ImageNet Classification with Deep Convolutional Neural Networks' (2012).
- He et al., 'Deep Residual Learning for Image Recognition' (2015). ResNet.
- Hochreiter and Schmidhuber, 'Long Short-Term Memory' (1997). the original LSTM paper.

Part II

Natural Language Processing and Transformers

This part contains chapter(s) 4, 5 covering natural language processing and transformers.

Chapter 4

Classical Natural Language Processing

4.1 Why Language Is Hard

Language carries meaning. Computers traffic in numbers. Bridging that gap is the central problem of natural language processing. Words are discrete symbols with rich meaning that depends on context, culture, and intent. The same string can mean different things; different strings can mean the same thing. Order matters, but not always equally. Some structures are recursive: a noun phrase can contain a noun phrase.

Classical NLP attacked these challenges with hand-crafted features (parts of speech, syntactic parses) and statistical models (n-grams, hidden Markov models, conditional random fields). These techniques produced working systems and remain useful as baselines and components, even though deep learning has displaced them at the cutting edge.

This chapter covers the classical methods you should know. They illuminate what came before transformers, they remain in production for many tasks where they are good enough, and they often serve as components of larger neural systems (BM25 in hybrid retrieval, NER pipelines, sentence boundary detection).

4.2 Tokenization

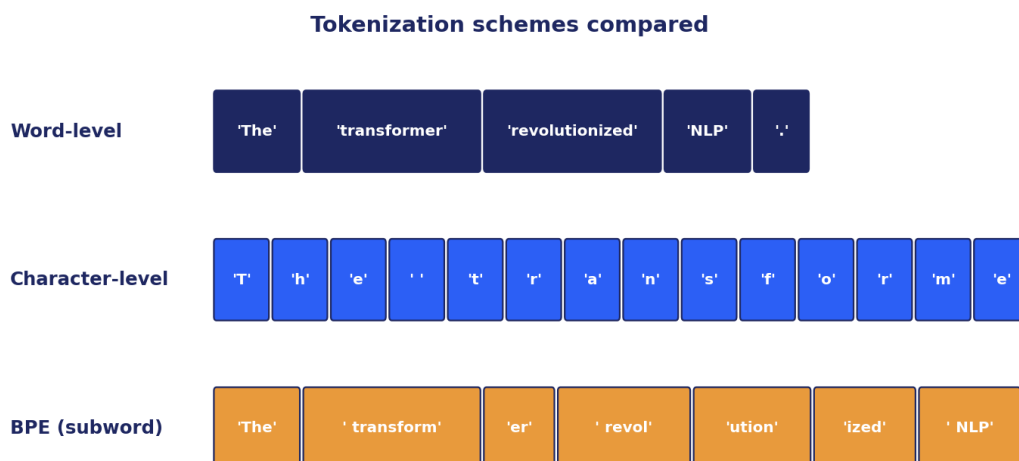


Figure 4.1: Word, character, and subword tokenization compared.

Tokenization splits raw text into the units a model will process. The choice of tokenizer has surprisingly large consequences for downstream model behavior.

4.2.1 Word-Level Tokenization

Split on whitespace and punctuation. Simple. Suffers from a vocabulary explosion (every inflection, compound, proper noun is a separate token) and out-of-vocabulary handling (any unseen word is mapped to <UNK>, losing information).

Variants: lowercasing, stemming (chop suffixes by rule), lemmatization (reduce to dictionary form using morphological analysis). Modern systems rarely use these explicitly; subword tokenizers handle surface variation implicitly through shared subword units.

4.2.2 Character-Level Tokenization

Treat each character as a token. Eliminates out-of-vocabulary issues. Very long sequences (5-10x longer than word-level) make modeling slower. Loses higher-level structure that word boundaries provide.

4.2.3 Subword Tokenization (BPE, WordPiece, SentencePiece)

The dominant choice for modern LLMs. Start from individual characters; iteratively merge the most frequent adjacent pair into a new token; stop when vocabulary reaches a target size (typically 30K-100K).

Byte-Pair Encoding (BPE) is the original algorithm; WordPiece is the Google variant used in BERT; SentencePiece is a language-agnostic framework that handles whitespace as a regular character (used in T5 and LLaMA). Unigram language modeling tokenization is another approach used in SentencePiece.

Net effect: common words become single tokens; rare words decompose into known subword pieces; truly unseen strings still encode without <UNK>. 'unbelievable' might be ['un', 'believ', 'able']. The model's embedding for the rare word is composed from its constituent embeddings.

4.3 Text Representation: Bag of Words and TF-IDF

Before learning anything fancy, you need a way to turn documents into vectors. The Bag of Words (BoW) representation does this naively: enumerate the vocabulary, represent each document as a vector of term counts.

BoW loses word order entirely. 'Dog bites man' and 'Man bites dog' produce the same vector. It cannot capture meaning: synonyms become separate dimensions, polysemous words collapse. The vocabulary grows with the corpus, producing very high-dimensional sparse vectors.

TF-IDF (Term Frequency $\times$ Inverse Document Frequency) refines this. $TF(t, d)$ is the frequency of term t in document d . $IDF(t) = \log(N / df(t))$ is the logarithm of the inverse document frequency. The product $TF-IDF(t, d) = TF \cdot IDF$ gives high scores to words that are frequent in a document but rare across the corpus.

BM25 is the modern successor of TF-IDF. It adds term-frequency saturation (diminishing returns from repeated occurrences) and document length normalization. BM25 remains an extremely strong baseline

for retrieval and is still embedded in Elasticsearch, OpenSearch, and Lucene as the default scoring function. In hybrid retrieval systems, BM25 plus dense retrieval almost always beats either alone.

4.4 Word Embeddings: Word2Vec and GloVe

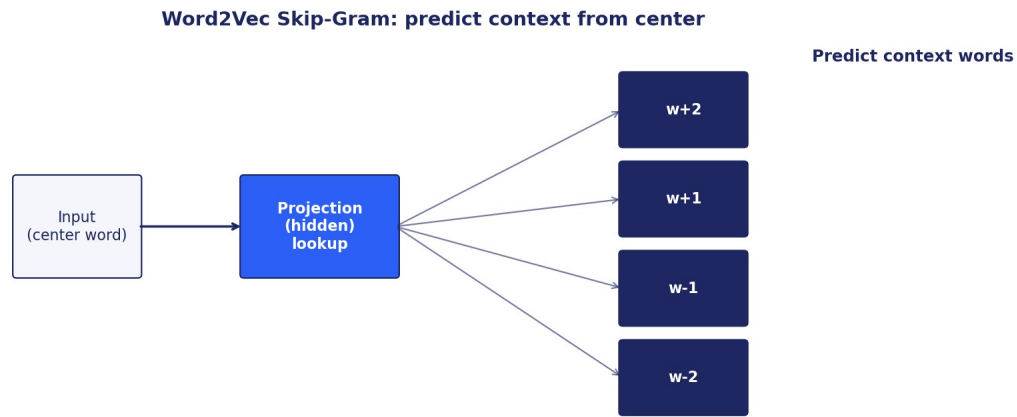


Figure 4.2: Word2Vec Skip-Gram predicts context words from a center word.

The fundamental limitation of BoW and TF-IDF is that each word is a separate dimension. The model cannot generalize from 'cat' to 'kitten' or from 'dog' to 'puppy'. Word embeddings solve this by placing each word at a point in a low-dimensional vector space such that similar words are nearby.

Word2Vec (Mikolov et al., 2013) trains a shallow neural network on a self-supervised objective: predict context words from a center word (skip-gram) or predict a center word from its context (CBOW). The hidden layer produces the embedding. Typical dimension: 100-300.

The famous result: vector arithmetic captures analogies. $\text{king} - \text{man} + \text{woman} \approx \text{queen}$. $\text{Paris} - \text{France} + \text{Italy} \approx \text{Rome}$. The embeddings encode semantic relationships in linear algebra.

GloVe (Global Vectors for Word Representation, Pennington et al., 2014) produces similar embeddings by matrix factorization of a global word-context co-occurrence matrix. Different math, similar properties.

FastText (Facebook, 2016) extends Word2Vec by treating each word as a sum of character n-gram embeddings. Handles morphologically rich languages and out-of-vocabulary words better.

Static word embeddings are limited: each word has one vector regardless of context. 'Bank' (financial institution) and 'bank' (river bank) get the same embedding. Contextual embeddings from BERT and other transformers fix this; we cover them in Chapter 8.

4.5 Named Entity Recognition and Sequence Labeling

Named Entity Recognition (NER) detects spans in text that refer to entities of interest (Person, Organization, Location, Date, Money) and tags them with category labels. It is a foundational information extraction task.

Classical NER used Conditional Random Fields (CRFs) over hand-crafted features (capitalization, prefix, suffix, gazetteers). Modern NER uses transformer-based encoders (BERT, RoBERTa) fine-tuned on labeled corpora (CoNLL-2003, OntoNotes). Zero-shot NER via instruction-tuned LLMs is increasingly competitive for novel domains.

NER is typically framed as token-level sequence labeling using IOB or BIOES tagging schemes (B-PER, I-PER, O for outside, etc.). The model predicts a label for each token; consecutive same-type labels form a span.

Applications: information extraction from news, financial filings, and clinical notes; populating knowledge graphs; redacting personally identifiable information; resume parsing; GraphRAG triplet extraction.

4.6 Topic Modeling with LDA

Latent Dirichlet Allocation (LDA, Blei et al., 2003) is a generative probabilistic model of document collections. It assumes each document is a mixture of topics, and each topic is a distribution over words.

Generative story: for each document, sample a distribution over K topics from a Dirichlet prior. For each word position, sample a topic from that distribution, then sample a word from that topic's word distribution. Inference reverses this: given the observed words, estimate the topic distributions via variational inference or collapsed Gibbs sampling.

Strengths: unsupervised, interpretable (you can read the top words per topic), fast on moderate corpora. Weaknesses: requires choosing K ; assumes bag-of-words (no order); topics can be incoherent on noisy data. Modern alternatives such as BERTopic cluster sentence embeddings and label clusters with class-based TF-IDF.

4.7 Evaluation Metrics for NLP

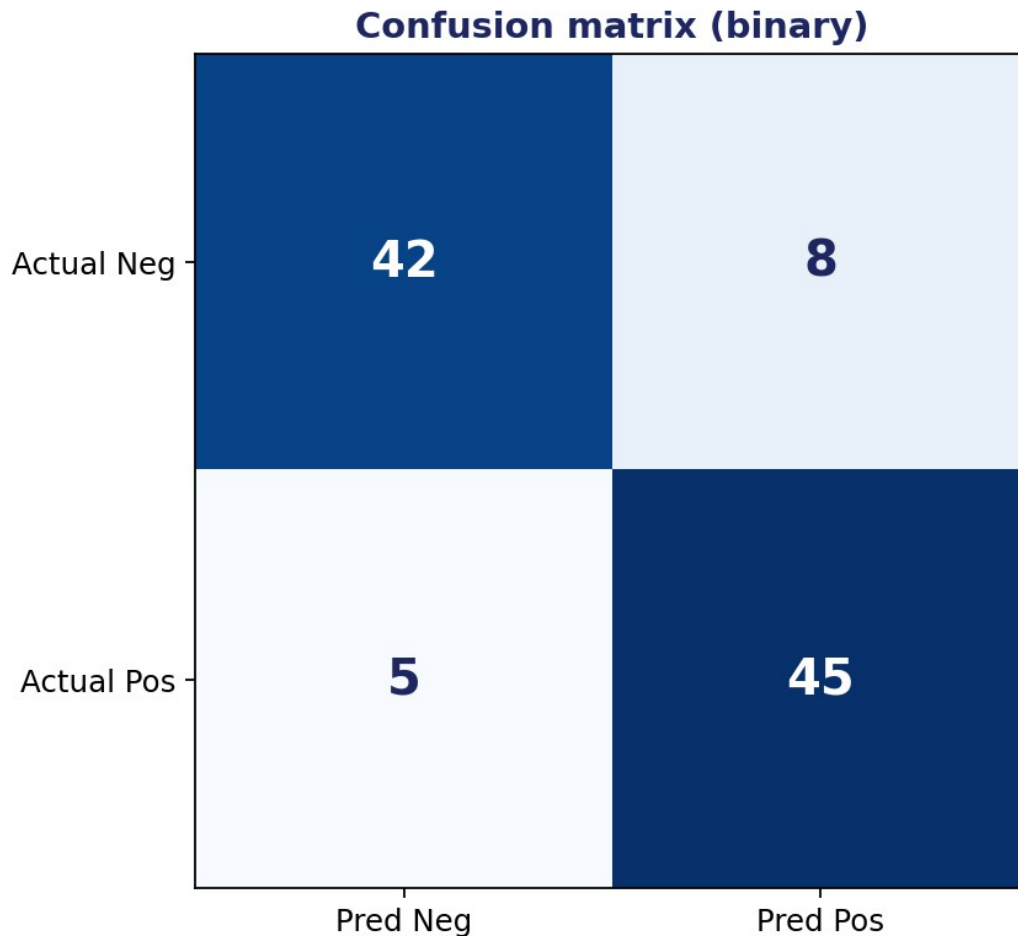


Figure 4.3: Confusion matrix for binary classification.

Classification: accuracy (fraction correct), precision (true positives / predicted positives), recall (true positives / actual positives), F1 (harmonic mean of precision and recall). ROC curves and AUC for ranking quality.

Translation: BLEU (n-gram overlap with reference), chrF (character F-score), COMET (learned metric correlated with human judgment).

Summarization: ROUGE-1/2/L (recall-oriented n-gram overlap). BERTScore for semantic similarity. Human evaluation for faithfulness and coherence.

Sequence labeling (NER, POS): token-level F1 plus span-level F1 (exact match of predicted spans against gold).

Language modeling: perplexity = $\exp(\text{per-token cross-entropy})$. Lower is better. Cheap to compute. Only weakly correlated with downstream task performance once perplexity is below a usable threshold.

4.8 Summary

- Tokenization turns text into integer IDs. Subword tokenizers (BPE, WordPiece, SentencePiece) are the modern default and handle out-of-vocabulary gracefully.
- BoW and TF-IDF are simple but powerful baselines for text representation. BM25 is the modern variant and remains a strong production retriever.
- Word embeddings (Word2Vec, GloVe, FastText) encode words as dense vectors with meaningful geometry. Static embeddings give one vector per word; contextual embeddings vary by context (Chapter 8).
- Named entity recognition and topic modeling solve information extraction at different granularities.
- Classification metrics, BLEU/ROUGE for generation, and perplexity for language modeling form the core evaluation toolkit.

Further Reading

- Jurafsky and Martin, Speech and Language Processing (3rd ed. draft, free online). The definitive NLP textbook.
- Mikolov et al., 'Efficient Estimation of Word Representations in Vector Space' (2013). the Word2Vec paper.
- Pennington, Socher, Manning, 'GloVe: Global Vectors for Word Representation' (2014).

Chapter 5

The Transformer Architecture

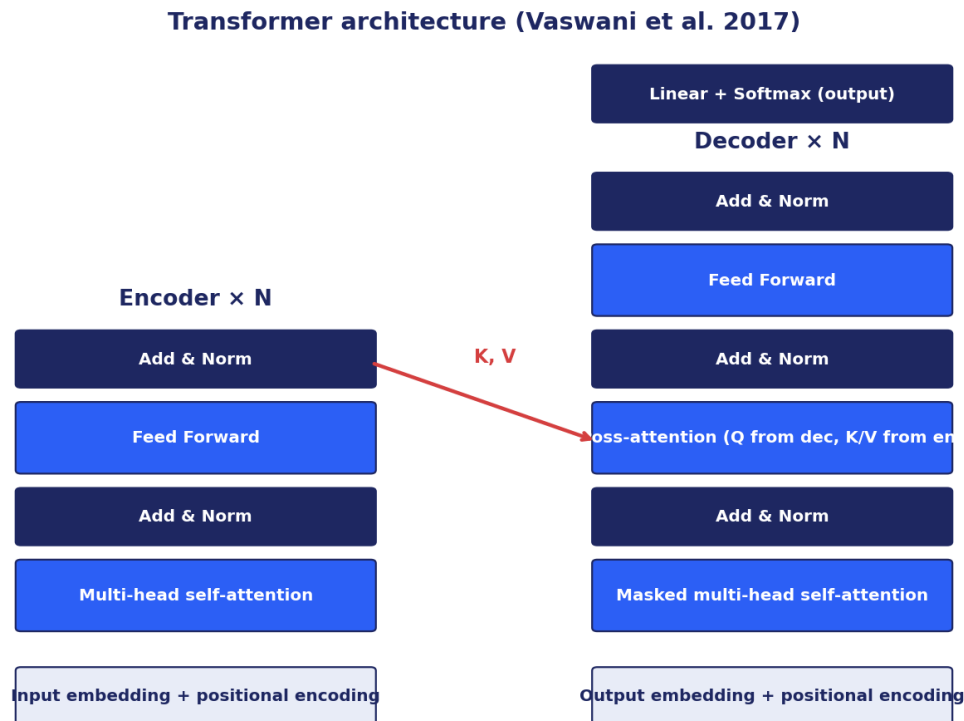


Figure 5.1: The original transformer architecture (Vaswani et al., 2017).

5.1 Why Transformers Won

Before 2017, sequence modeling was dominated by RNNs and LSTMs. These models had two structural problems that capped their performance: they processed sequences strictly left to right (no parallelism over the time axis), and they compressed everything seen so far into a fixed-size hidden state (information about distant tokens leaked away).

The transformer, introduced in 'Attention Is All You Need' by Vaswani et al. in 2017, replaced recurrence with self-attention. Every token can attend directly to every other token. Computation parallelizes over the sequence dimension. Long-range dependencies are one hop away. The result was both faster training on GPUs and better quality at scale.

Within five years, transformers swept NLP, then computer vision (Vision Transformers, 2020), then audio (Whisper, 2022), then multimodal (CLIP, GPT-4V). Every modern frontier model is a transformer or

a transformer derivative. Understanding the transformer is the single most important investment you can make in modern AI.

5.2 Self-Attention

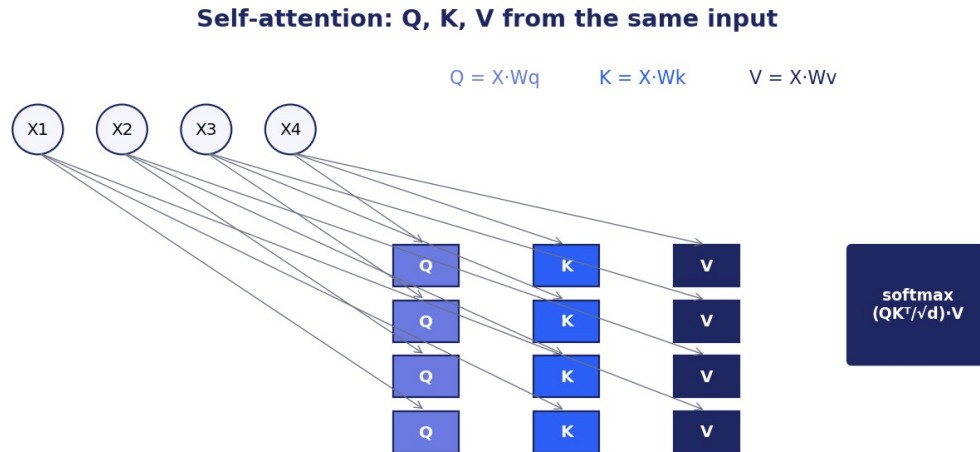


Figure 5.2: Self-attention computes Q, K, V from the input and combines V weighted by $\text{softmax}(QK^T/\sqrt{d})$.

Self-attention is the core operation. The intuition: every token computes a weighted combination of all tokens, where the weights depend on relevance.

5.2.1 Queries, Keys, and Values

Each input token x_i is projected to three vectors: a query $q_i = W_Q \cdot x_i$, a key $k_i = W_K \cdot x_i$, and a value $v_i = W_V \cdot x_i$. The projection matrices W_Q , W_K , W_V are learned.

The query represents what the token is looking for. The key represents what the token offers. The value is the actual content. The metaphor: querying a database, where keys identify entries and values are the content returned.

5.2.2 The Attention Formula

For all tokens, the attention output is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V \quad (5.1)$$

Three steps. First, compute pairwise dot products between every query and every key. The result is an $n \times n$ matrix of raw attention scores. Second, scale by $\sqrt{d_k}$ (where d_k is the key dimension) and apply softmax row-wise. This turns scores into a probability distribution: each row sums to 1. Third, multiply by V to combine values weighted by attention.

Why the $\sqrt{d_k}$ scaling? Without it, dot products grow large in high dimensions, pushing softmax into saturation. Gradients vanish. The scaling keeps softmax inputs in a usable range.

Masked self-attention (used in decoder layers and autoregressive models): set scores S_{ij} to $-\infty$ for $j > i$ before softmax. The model cannot peek at future tokens. This is what makes GPT-style models autoregressive.

5.2.3 Multi-Head Attention

Single-head attention forces all relationships through one set of weights. Multi-head attention runs h attention computations in parallel with different W_Q , W_K , W_V projections, each of dimension d_{model} / h . The outputs are concatenated and projected back through W_O .

Each head can specialize. One might track syntactic dependencies, another semantic coreference, another local n -gram patterns. The same compute as single-head with full dimension, but much richer behavior.

Typical h : 8, 16, or 32. Modern LLMs use grouped-query attention (GQA) or multi-query attention (MQA) where the number of Q heads exceeds the number of K and V heads, reducing KV-cache memory at inference time with minor quality loss.

5.3 Positional Encoding

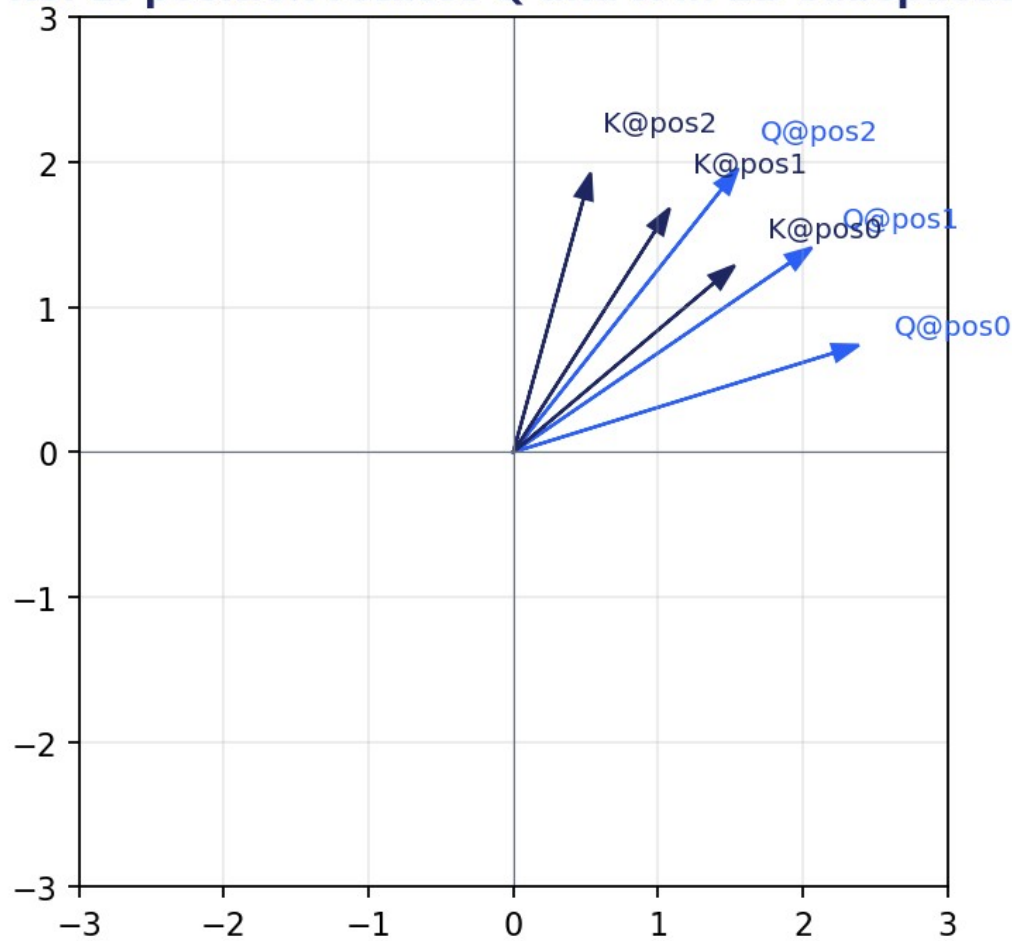
RoPE: position rotates Q and K in 2D subspaces

Figure 5.3: RoPE rotates query and key vectors in 2D subspaces by an angle proportional to position.

Self-attention is permutation-equivariant: shuffle the input tokens, get a shuffled output. The transformer has no built-in notion of order. Positional encoding injects position information.

Sinusoidal positional encoding (original transformer): add fixed sin/cos functions of position to the input embeddings. Different frequencies per dimension. Generalizes in theory to unseen positions.

Learned positional encoding (BERT, GPT-2): treat each position index as a learnable embedding. Simple, works well within trained range, does not extrapolate to longer sequences.

Rotary Position Embedding (RoPE, used in LLaMA, GPT-NeoX, Mistral): rotate Q and K vectors by an angle proportional to position. The dot product $Q \cdot K$ depends on the relative position because both rotations cancel out the absolute position. Extrapolates better than absolute encodings. Augmented with NTK-aware scaling or YaRN for further length extension.

ALiBi (Attention with Linear Biases): add a position-dependent bias to attention scores. Very strong extrapolation. Used in MPT, BLOOM.

5.4 The Full Transformer Layer

A transformer layer composes attention with a feed-forward network. The structure: multi-head self-attention, then residual connection plus layer normalization, then position-wise feed-forward network, then residual connection plus layer normalization.

The feed-forward network is a two-layer MLP applied independently at each position. Typically 4x the model dimension as the hidden width. Modern architectures replace ReLU with GELU or SiLU (Swish) and sometimes add a gating mechanism (SwiGLU in LLaMA).

Residual connections are essential. Without them, deep transformers do not train. They allow gradients to flow back through the network unimpeded.

Layer normalization is applied either before each sub-layer (Pre-LN, modern default, more stable training) or after (Post-LN, original transformer, harder to train at large depths). LLaMA uses RMSNorm, a simpler variant.

5.5 Encoder, Decoder, Encoder-Decoder

Encoder-decoder transformer (translation example)

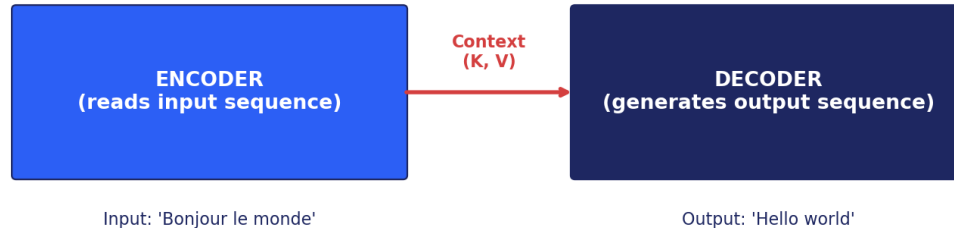


Figure 5.4: Encoder-decoder transformer for sequence-to-sequence tasks.

Three transformer variants depending on the task.

5.5.1 Encoder-Only (BERT family)

Bidirectional self-attention over the input. Produces rich contextual representations of every token. No generation. Best for classification, NER, embedding generation, similarity. Trained with Masked Language Modeling (MLM): mask 15% of tokens, predict them from bidirectional context.

Examples: BERT, RoBERTa, DeBERTa, ELECTRA. Still dominant for embedding generation and classification at scale due to efficiency.

5.5.2 Decoder-Only (GPT family)

Causal (left-to-right) masked self-attention. Generates tokens one at a time. Trained with causal language modeling: predict next token given previous tokens. Modern instruction-tuned variants (GPT-4, Claude, LLaMA, Mistral, Gemma) handle both understanding and generation tasks via prompting.

Examples: GPT-1/2/3/4, LLaMA, Mistral, Claude. The dominant pattern for general-purpose LLMs.

5.5.3 Encoder-Decoder

Encoder produces contextual representations of the input. Decoder generates the output autoregressively, with cross-attention into the encoder output at each layer. The natural choice for translation, summarization, and any text-to-text task.

Examples: original transformer (translation), BART, T5. T5 unified every NLP task as text-to-text. Decoder-only models have largely displaced encoder-decoders in production for general LLMs, but encoder-decoder remains competitive for specific structured tasks.

5.6 Pretraining Objectives

Modern transformers are pretrained with self-supervised objectives on massive corpora, then fine-tuned or prompted for downstream tasks. The choice of objective shapes the model's capabilities.

5.6.1 Causal Language Modeling (CLM)

Predict the next token given the previous tokens. Used in GPT, LLaMA, Mistral, and most decoder-only LLMs. Simple, scales well, and naturally enables generation at inference time. Each token contributes to the loss.

5.6.2 Masked Language Modeling (MLM)

Mask a fraction of tokens (15% in BERT), predict them from bidirectional context. Forces the model to use both left and right context. Used in BERT, RoBERTa, DeBERTa.

5.6.3 Span Corruption (T5, BART)

Mask contiguous spans rather than individual tokens. Predict the spans in order. Generalizes MLM to longer spans. Natural fit for text-to-text framing.

5.6.4 Replaced Token Detection (ELECTRA)

A small generator proposes replacements for masked tokens. The main model classifies each token as original or replaced. Every token contributes to the loss, making it more sample-efficient than MLM.

5.6.5 Permutation Language Modeling (XLNet)

Predict tokens in a randomly permuted order so each target sees a mix of left and right context without using MLM's masking artifact. Combines the bidirectional benefit of MLM with the autoregressive property of CLM.

5.7 Long Context: Beyond Quadratic Attention

Standard self-attention has $O(n^2)$ time and memory in sequence length n . This is the binding constraint for long-context applications: documents, codebases, conversations spanning hours.

Sparse and windowed attention: Longformer combines sliding-window local attention with a few global tokens. BigBird adds random attention. Each token attends to $O(n)$ tokens instead of $O(n^2)$. Used for long-document tasks.

Linearized attention: Linformer projects keys and values to a smaller rank. Performer approximates softmax with random feature maps. Reformer uses LSH to bucket similar queries. Each trades exact attention for asymptotic improvement; quality varies.

Memory-augmented architectures: Transformer-XL caches hidden states across segments. Compressive Transformer further compresses old states. RWKV and state-space models (Mamba) replace attention entirely with recurrent or linear updates.

Production answer: retrieval-augmented generation. Instead of extending the context window, retrieve only relevant chunks per query. We cover this in Chapter 12. For many use cases, this is more cost-effective than long-context training.

5.8 Summary

- The transformer replaces recurrence with self-attention, enabling parallelism over the sequence dimension and direct modeling of long-range dependencies.
- Self-attention computes Q, K, V projections, then weights values by $\text{softmax}(QK^T/\sqrt{d_k})$. Multi-head attention adds specialization.
- Positional encoding (sinusoidal, learned, RoPE, ALiBi) injects order information that self-attention lacks.
- Encoder-only models (BERT) understand; decoder-only models (GPT) generate; encoder-decoder models (T5) translate. Decoder-only dominates general LLMs.
- Pretraining objectives (CLM, MLM, span corruption, ELECTRA, PLM) define how the model learns from raw text.
- Quadratic attention is the long-context bottleneck. Sparse attention, linearized attention, state-space models, and RAG all address it differently.

Further Reading

- Vaswani et al., 'Attention Is All You Need' (2017). The foundational paper. Read it.
- Devlin et al., 'BERT: Pre-training of Deep Bidirectional Transformers' (2018).

- Radford et al., 'Language Models are Few-Shot Learners' (2020). GPT-3.
- Su et al., 'RoFormer: Enhanced Transformer with Rotary Position Embedding' (2021).

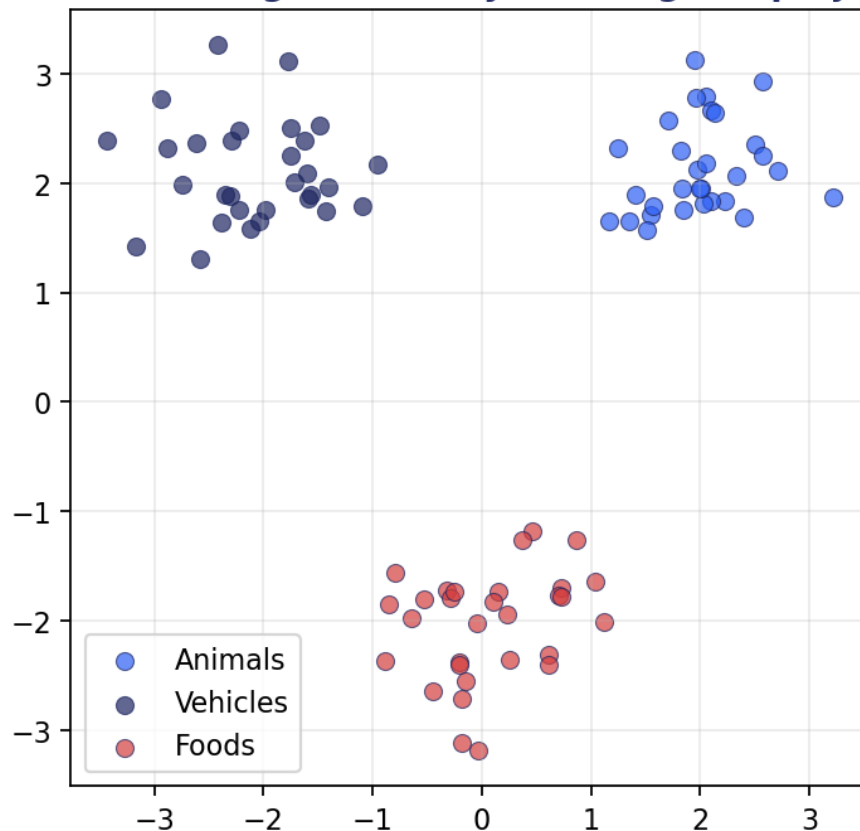
Part III

Embeddings and Pretrained Models

This part contains chapter(s) 6, 7 covering embeddings and pretrained models.

Chapter 6

Word and Sentence Embeddings

Word embeddings cluster by meaning (2D projection)*Figure 6.1: Word embeddings cluster by meaning in vector space.*

6.1 The Embedding Concept

Words are discrete symbols. Neural networks operate on continuous vectors. An embedding is the bridge: a learned function that maps each word, sentence, or document to a point in a high-dimensional vector space, such that geometric relationships in that space approximate semantic relationships in the original domain.

This idea generalizes. Embeddings exist for users in recommendation systems, for nodes in graphs, for products in e-commerce, for images and audio in multimodal systems. The same geometric framework applies: similar items are nearby, related items have meaningful directional relationships, and the space itself can be analyzed, clustered, and searched.

The historical arc: one-hot vectors (dimension = vocabulary size, all zeros except one) gave way to count-based methods (LSA, LDA topic distributions), then to learned word embeddings (Word2Vec, GloVe, FastText), then to contextual embeddings from transformers (BERT and successors). Each generation captured more meaning in fewer dimensions with better generalization. This chapter walks the arc and ends at the state of the practice.

6.2 From One-Hot to Dense

One-hot encoding represents word i as a vector of length $|V|$ (vocabulary size) with a 1 in position i and 0 elsewhere. Three fatal problems: high dimensionality ($|V|$ can be 100K+), no notion of similarity (every word is equidistant from every other word), and out-of-vocabulary words have no representation at all.

Dense embeddings replace one-hot vectors with low-dimensional (typically 50-1024) real-valued vectors. The vector for 'cat' is learned, not assigned. Two key properties emerge from good training: similar words have similar vectors (high cosine similarity), and relationships between words manifest as consistent directions (vector('king') - vector('man') + vector('woman') $\approx$ vector('queen')).

The benefit is generalization. A classifier trained on 'cat' images can transfer some understanding to 'kitten' images because their text embeddings are nearby. A model that has seen many sentences about 'Paris' can apply that knowledge to a question about 'the capital of France' through embedding similarity. Dense embeddings let models share information across related items rather than treating each as independent.

6.3 Word2Vec in Detail

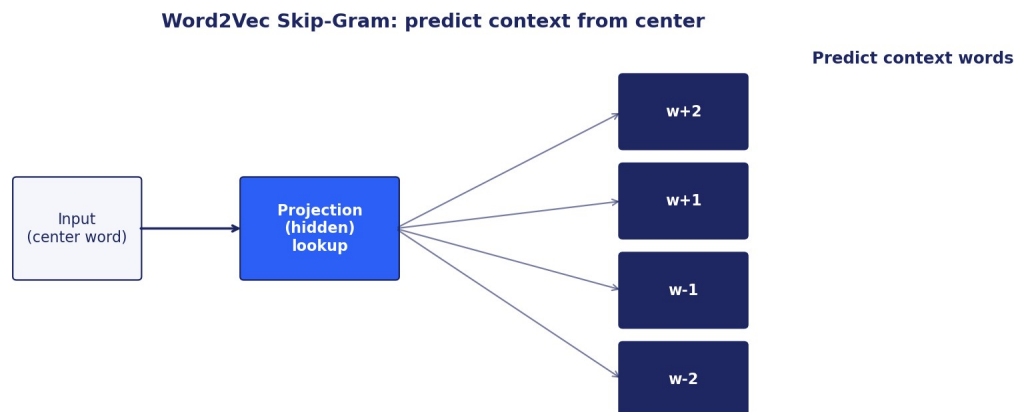


Figure 6.2: Word2Vec Skip-Gram architecture.

Word2Vec (Mikolov et al., 2013) was a watershed: a fast, simple neural training procedure that produced word embeddings of unprecedented quality. Two architectures.

6.3.1 Continuous Bag of Words (CBOW)

Predict the center word from its context window. Input: average of context word embeddings (or sum). Output: predicted center word via softmax over vocabulary. The hidden layer is the embedding lookup table; the output layer is a separate matrix.

CBOW is faster to train than Skip-Gram and produces slightly smoother embeddings for frequent words. The downside: small datasets produce weaker embeddings because there is less signal per context.

6.3.2 Skip-Gram

Predict the context words from the center word. Input: single center word embedding. Output: probability of each context word. The training objective: maximize $\log P(\text{context} \mid \text{center})$ summed over (center, context) pairs in a corpus.

Skip-Gram works better on rare words and produces embeddings that capture finer distinctions. It became the dominant variant in practice.

6.3.3 Negative Sampling

Computing softmax over the entire vocabulary is expensive. Negative sampling replaces it with a binary classification: distinguish actual (center, context) pairs from randomly sampled (center, negative) pairs.

For each true pair, sample k negatives (typically $k = 5$ to 20). Train with binary cross-entropy. The gradient updates only the involved word vectors, not all of them. Training speeds up by orders of magnitude with little quality loss.

6.3.4 Hierarchical Softmax

An alternative to negative sampling. Organize the vocabulary as a binary tree (often a Huffman tree by word frequency). Predicting a word becomes $\log(|V|)$ binary decisions along the tree path. Asymptotically faster than full softmax. Less popular than negative sampling in practice but elegant.

6.4 GloVe and FastText

GloVe (Pennington et al., 2014) approaches embeddings through matrix factorization. Build a global co-occurrence matrix X where X_{ij} is the number of times word i appears in the context of word j . Factor it into word vectors that approximate $\log X_{ij}$. GloVe combines the global statistical view of LSA with the local prediction-based training of Word2Vec. Empirically comparable to Word2Vec; the choice often comes down to convenience.

FastText (Bojanowski et al., 2016) addresses the rare-word problem. Represent each word as a sum of its character n -gram embeddings (typically $n = 3$ to 6). The word 'unbelievable' decomposes to character n -grams like '<un', 'unb', 'nbe', ..., 'le>'. Each n -gram has an embedding; the word embedding is the sum.

Two benefits. Morphologically rich languages (Finnish, Turkish) get better embeddings because related word forms share n -grams. Truly unseen words at test time get meaningful vectors from their character pieces. FastText is still widely used in production for these reasons.

6.5 Contextual Embeddings: ELMo and BERT

Static word embeddings have a fundamental limitation: each word has one vector regardless of context. 'Bank' as a financial institution and 'bank' as a river bank get the same embedding. The first cracks in this approach appeared with ELMo (Embeddings from Language Models, Peters et al., 2018), which concatenated forward and backward LSTM hidden states to produce context-dependent representations.

BERT (Devlin et al., 2018) took this much further. A bidirectional transformer trained with masked language modeling produces an embedding for each token that depends on the entire sentence. The same word in different contexts gets different vectors. The same token in 'I deposited money at the bank' and 'I sat by the river bank' receives meaningfully different embeddings.

How to use BERT for embedding generation. Pass a sentence through BERT; take the final hidden states. The [CLS] token's embedding is often used as a sentence representation, though this is naive. Better practice: pool the token embeddings (mean or weighted), or use a model fine-tuned with a sentence-level objective (Sentence-BERT, BGE, E5).

Why contextual embeddings matter for retrieval. A query and a document may share no surface tokens but have similar meaning. Contextual embeddings of the full query and document capture this. Modern semantic search depends on this property.

6.6 Sentence Embeddings

Word embeddings can be averaged or pooled to produce sentence embeddings, but the results are mediocre. Word order, negation, and compositional structure are lost. Dedicated sentence embedders fix this.

6.6.1 Sentence-BERT and Successors

Sentence-BERT (Reimers and Gurevych, 2019) fine-tunes a pretrained BERT with a siamese network architecture. Two BERT towers share weights. Each tower encodes a sentence. The cosine similarity between sentence embeddings is trained to match a target (similarity score, classification label, contrastive label).

The result is a sentence encoder that produces high-quality embeddings in a single forward pass. Cosine similarity becomes meaningful between sentences. Semantic search, clustering, and similarity at the sentence and document level all use this pattern.

Modern successors: MPNet, BGE (Beijing Academy), E5 (Microsoft), GTE (Alibaba). Each pushes the MTEB benchmark (Massive Text Embedding Benchmark) forward. Open-weights models from these families are competitive with closed-source embedders (OpenAI ada-002, Cohere embed) at much lower cost.

6.6.2 Contrastive Training Objectives

Modern sentence embedders are trained with contrastive objectives. The intuition: pull semantically similar sentence pairs together, push dissimilar pairs apart.

Triplet loss (anchor, positive, negative with margin) was the original approach. InfoNCE (anchor against batch of negatives via softmax) is now standard. Both are forms of contrastive learning.

Hard negative mining matters enormously. Random negatives are usually trivially distinguishable from the positive; the model learns nothing useful. Hard negatives (semantically close but labeled different) provide the strongest training signal. Production pipelines mine hard negatives from production logs, BM25 top hits, or dense top hits that the model previously got wrong.

6.7 Properties of Embedding Spaces

Beyond the basic 'similar = nearby' property, embedding spaces have richer structure that influences how you use them.

6.7.1 Anisotropy

Plain BERT embeddings cluster in a narrow cone in embedding space. Average pairwise cosine similarity is high across unrelated sentences. This inflates similarity scores uniformly and reduces the discriminative power of cosine similarity.

Two fixes. Whitening: subtract the mean, scale by the inverse covariance matrix's square root. The resulting embeddings are zero-mean and unit-covariance. Post-hoc, cheap, often substantially improves retrieval. Contrastive fine-tuning: train with InfoNCE or triplet loss to spread the embeddings across the space. The modern default; eliminates anisotropy structurally rather than patching it.

6.7.2 Curse of Dimensionality

Embeddings live in 100-1024-dimensional spaces. Human geometric intuition fails. Three counter-intuitive properties: nearly all volume of a unit ball lies near its surface; random unit vectors are nearly orthogonal; distances concentrate (the ratio of maximum to minimum pairwise distance approaches 1).

Consequences. Naive 'nearest neighbor' search loses discriminative power at very high dimensions; approximate methods are necessary. Cosine similarity is preferred over Euclidean distance because it ignores magnitude. Embeddings should be L2-normalized before cosine-based operations.

6.7.3 Matryoshka Embeddings

Some embedders are trained so that prefixes of the full embedding also function as useful embeddings at lower dimensions. A 768-dimensional embedding's first 64 dimensions are themselves a usable 64-dimensional embedding.

Use case: progressive search. Use the 64-dimensional truncation for cheap pruning across millions of candidates, then re-score the top 100 with the full 768-dimensional embedding. Memory and latency drop without giving up final-stage quality.

Implementations: bge-m3, nomic-embed, openai text-embedding-3.

6.8 Cross-Modal Embeddings

Cross-modal embeddings live in a shared space across modalities (text and image, text and audio). The same point in the space can be reached from multiple modalities, enabling cross-modal retrieval (text query for an image) and zero-shot transfer (use text labels to classify images without image training labels).

CLIP (Radford et al., 2021) is the canonical example. Image encoder (ViT) and text encoder (transformer) are trained jointly on 400M image-caption pairs with a contrastive loss: image embeddings should be near their captions and far from other captions. The resulting embedding space supports zero-shot image classification (encode candidate text labels, embed the image, take argmax similarity) and cross-modal retrieval.

ImageBind (Meta, 2023) extends this to six modalities (images, text, audio, depth, thermal, IMU) by using image as the binding modality. Train each non-image modality against images; the space ends up aligned across modalities that were never paired directly.

6.9 Quantization and Efficient Storage

Embeddings are typically stored as float32 vectors. A million 768-dimensional vectors is 3GB. At billion scale, this becomes the dominant memory cost. Quantization compresses embeddings at some cost to recall.

6.9.1 Scalar Quantization

Convert each dimension from float32 to int8 (or int4, or int2). 4x to 16x smaller. Apply a per-vector or per-dimension scale and zero point. Typically loses less than 1 percentage point on retrieval recall. The default first step.

6.9.2 Product Quantization (PQ)

Split each vector into m subvectors. Train k codewords per subvector via k -means. Replace each subvector with its nearest codeword index. Storage: $m \cdot \log_2(k)$ bits per vector, often 32x smaller than float32. Used by FAISS at billion scale.

Variants: OPQ (rotate before quantizing), Residual PQ (multi-stage to capture remaining error), Inverted File + PQ (the standard FAISS index for very large datasets).

6.9.3 Binary Embeddings

One bit per dimension. Hamming distance replaces cosine. 32x compression. Loss is real but acceptable for first-stage retrieval. Modern production systems rerank with full-precision embeddings on the candidate set, so the binary stage is purely for fast filtering.

6.10 Evaluating Embedding Quality

How do you know if an embedder is good? Several complementary metrics.

6.10.1 Intrinsic Evaluation

Similarity correlation. Compute cosine similarity of embedding pairs; correlate with human similarity ratings (Spearman or Pearson). Standard datasets: STS-B (sentence textual similarity), WordSim-353, SimLex-999. Spearman correlation is the de facto reporting metric for embedding model leaderboards.

Analogy benchmarks. king - man + woman = ? Classic but limited; less informative on modern embedders than retrieval metrics.

Clustering metrics. Silhouette score, Davies-Bouldin index, Calinski-Harabasz index. Useful when you have ground-truth clusters (topic-labeled documents, intent-labeled queries).

6.10.2 Extrinsic Evaluation

Downstream task performance is the ultimate test. Use the embeddings for the actual task and measure task-specific metrics: Recall@k for retrieval, accuracy for classification, V-measure for clustering.

MTEB (Massive Text Embedding Benchmark) aggregates 50+ tasks across classification, clustering, retrieval, reranking, similarity. The comprehensive standard for comparing text embedders.

Domain-specific evaluation. Build a held-out set of relevant queries and known answers in your domain. Measure Recall@k. Public benchmarks may not reflect your data distribution; in-domain evaluation is decisive.

6.11 Summary

- Embeddings map discrete items (words, sentences, images) to dense vectors such that geometric relationships approximate semantic relationships.
- Word2Vec, GloVe, and FastText produce static word embeddings. They are fast, mature, and still useful for many tasks.
- Contextual embeddings from BERT and successors solve the polysemy problem and produce strong sentence-level embeddings when fine-tuned with contrastive objectives.
- Cross-modal embeddings (CLIP, ImageBind) align modalities in a shared space, enabling cross-modal retrieval and zero-shot transfer.
- Quantization (scalar, product, binary) makes billion-scale embedding storage and search feasible.
- MTEB is the standard public benchmark; domain-specific in-house evaluation is decisive for production decisions.

Further Reading

- Mikolov et al., 'Distributed Representations of Words and Phrases and their Compositionality' (2013). Word2Vec with negative sampling.
- Reimers and Gurevych, 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks' (2019).
- Radford et al., 'Learning Transferable Visual Models From Natural Language Supervision' (2021). CLIP.
- Kusupati et al., 'Matryoshka Representation Learning' (2022).

Chapter 7

Pretrained Language Models

7.1 The Pretraining Revolution

Before 2018, NLP systems were largely task-specific. You trained a model from scratch (or from word embeddings) on labeled data for your task. Each new task started over. After 2018, the field shifted decisively to pretraining and fine-tuning. A single model pretrained on massive text corpora produced representations that transferred to dozens of downstream tasks with minimal additional training.

The shift was driven by three forces: scaling laws (more data and parameters reliably improve performance), self-supervised objectives (language models learn from raw text without human labels), and architectural advances (transformers parallelize and scale better than RNNs). By 2020, every state-of-the-art NLP system was a transformer pretrained on billions of tokens.

This chapter walks through the major model families. The technical lessons apply broadly even as specific models age out: encoder versus decoder architectures, pretraining objectives, scaling behavior, the relationship between pretraining and fine-tuning, and the design choices that make some models more useful than others.

7.2 BERT and the Encoder-Only Family

BERT (Bidirectional Encoder Representations from Transformers, Devlin et al., 2018) was the model that mainstreamed pretraining. Two design choices made it work.

Bidirectional attention. Earlier language models read left-to-right (or concatenated separate left-to-right and right-to-left models, as in ELMo). BERT used full bidirectional self-attention. Every token attends to every other token in both directions. The representations are richer.

Masked language modeling. To train a bidirectional model on next-token prediction would let it cheat (the answer is in the context it can see). BERT masks 15% of input tokens and predicts them from bidirectional context. The training signal is dense across tokens, and bidirectionality is preserved.

BERT-base has 12 layers, 12 attention heads, 768 hidden dimensions, 110M parameters. BERT-large doubles depth and hidden size to 340M parameters. Trained on Wikipedia plus BooksCorpus (3.3B words total). The results were state-of-the-art across 11 NLP benchmarks at release.

7.2.1 Downstream Use

BERT's standard usage pattern: prepend a [CLS] token, append a [SEP] token, run through the encoder, attach a task-specific head. For classification, a linear layer on the [CLS] embedding. For token labeling

(NER), a linear layer on each token embedding. For span tasks (extractive QA), predict start and end token positions.

The same backbone serves many tasks; only the head and a small amount of task-specific data are required. Fine-tuning typically requires a few epochs at a small learning rate ($1e-5$ to $5e-5$). The pretrained encoder retains most of its representation; the head learns the task-specific projection.

7.2.2 BERT Successors

RoBERTa (Liu et al., 2019) revisited BERT's training: larger batches, more data (160GB vs 16GB), longer training, removed Next Sentence Prediction (which turned out to add little). The result was a substantially stronger model with the same architecture.

DeBERTa (He et al., 2020, 2021) added disentangled attention and enhanced mask decoding. Pushed state-of-the-art further on SuperGLUE.

ELECTRA (Clark et al., 2020) changed the pretraining objective. A small generator proposes replacements for masked tokens; the main model classifies each token as original or replaced. Every token contributes to the loss, making training more sample-efficient than MLM.

ALBERT (Lan et al., 2020) factorized embeddings and shared parameters across layers to reduce model size at small quality cost.

Modern encoder models for embeddings (BGE, E5, GTE, mpnet) are based on these designs with contrastive fine-tuning for retrieval.

7.3 GPT and the Decoder-Only Family

GPT (Generative Pretrained Transformer) took a different bet. Use a decoder-only transformer with causal (left-to-right) attention. Pretrain on next-token prediction. Scale up. The architecture is simpler than BERT; the objective matches the inference-time task (generation); the model naturally produces text.

GPT-1 (Radford et al., 2018) introduced the architecture but had limited impact at 117M parameters. GPT-2 (2019) scaled to 1.5B parameters and demonstrated startling zero-shot capabilities on diverse tasks via prompting alone. GPT-3 (Brown et al., 2020) scaled to 175B parameters and introduced in-context learning at scale: the ability to learn new tasks from a few examples in the prompt, with no weight updates.

After GPT-3, the field bifurcated. OpenAI kept its models closed but released access through APIs. Meta released LLaMA (2023), then LLaMA-2 and LLaMA-3, as open-weights models. Mistral, Gemma, Qwen, Yi, DeepSeek followed. Open-weights LLMs are now competitive with closed-source models on many benchmarks.

7.3.1 Modern Decoder Architecture

Modern decoder-only LLMs (LLaMA-style) include refinements over the original GPT architecture.

Pre-normalization. Apply RMSNorm before each sub-layer (attention and FFN) instead of after. Stabilizes training at scale.

Rotary Position Embedding (RoPE). Rotate Q and K vectors by an angle proportional to position. Better length extrapolation than absolute positional encoding.

SwiGLU activation. A gated variant of Swish in the FFN. Slightly better quality than GELU at similar compute.

Grouped-Query Attention (GQA). Fewer KV heads than Q heads. Reduces KV-cache size and speeds up inference. Used in LLaMA-2 70B and LLaMA-3.

Tied embeddings (sometimes). Share input and output token embeddings to save parameters.

7.3.2 Scaling Laws

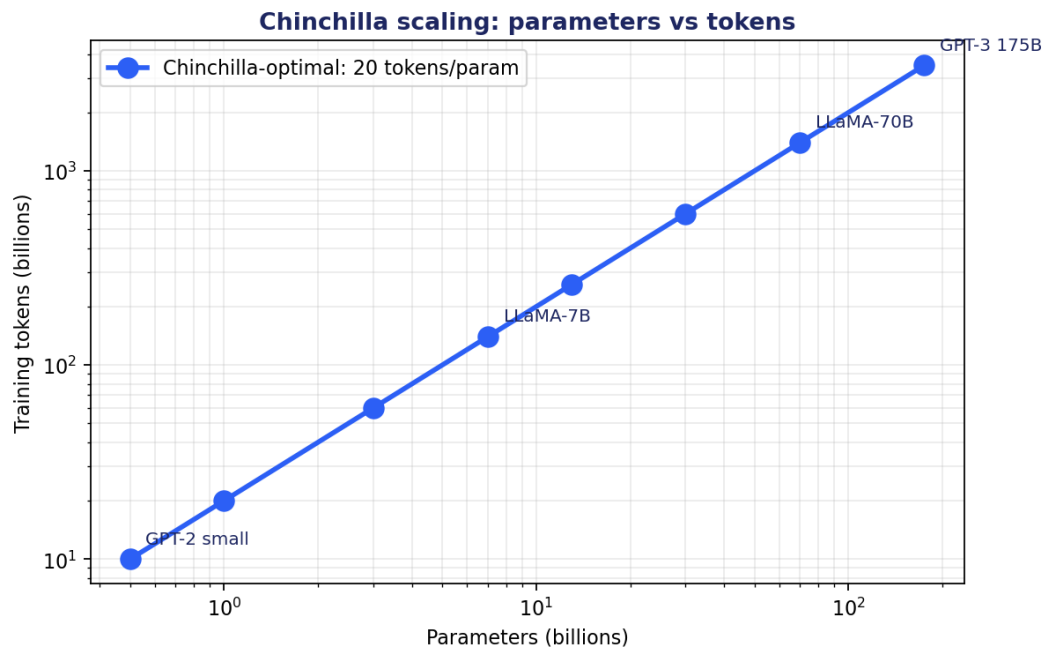


Figure 7.1: Chinchilla-optimal scaling: 20 tokens per parameter.

Kaplan et al. (2020) showed that LLM loss scales smoothly with parameters and training tokens. The original scaling laws suggested ever-larger models. Then Hoffmann et al. (2022, 'Chinchilla') showed that GPT-3 and similar models were significantly under-trained. For a fixed compute budget, optimal model size and training tokens should scale together at roughly 20 tokens per parameter.

A 7B model wants 140B tokens. A 70B model wants 1.4T tokens. Modern training has pushed well past these ratios; LLaMA-3 uses ~15T tokens for an 8B model and 70B model. The diminishing returns regime is real but the absolute gains continue to matter.

Implication for practitioners: smaller, well-trained models often beat larger, under-trained ones. A 7B model trained on 2T tokens can outperform a 13B model trained on 100B tokens.

7.3.3 In-Context Learning

GPT-3's headline capability was in-context learning. Provide a few input-output examples in the prompt, then ask for a new prediction. The model continues the pattern with no parameter updates.

How does this work? The model was trained on internet text, which contains many instances of demonstration patterns: lists, tables, translation pairs, code with similar functions. Predicting the next token in such sequences requires generalizing the pattern. This generalization is the meta-task that enables in-context learning at inference time.

Empirical findings. Quality scales with model size; larger models exhibit qualitatively more in-context learning capability. Sensitive to example ordering, format, and selection. More examples help up to a point, often 5-10 examples. The examples need not be perfectly labeled; the pattern matters more than the labels.

7.4 Encoder-Decoder Models: T5 and BART

Encoder-decoder transformers are well-suited to sequence-to-sequence tasks: translation, summarization, structured extraction. They combine the rich input understanding of encoder-only models with the generation capability of decoder-only models.

T5 (Text-to-Text Transfer Transformer, Raffel et al., 2020) unified every NLP task as text-to-text.

Translation: input 'translate English to French: The cat is on the mat.' Output: 'Le chat est sur le tapis.'

Summarization: input 'summarize: <article>'. Output: '<summary>'. Even classification was framed as text generation. T5 was trained with span corruption: mask contiguous spans of the input, predict them as the output.

BART (Bidirectional and Auto-Regressive Transformers, Lewis et al., 2020) trained with multiple denoising objectives: token masking, token deletion, span infilling, sentence permutation, document rotation. The model learned to reconstruct original text from corrupted versions. Strong on summarization, translation, and question generation.

FLAN-T5 (Chung et al., 2022) fine-tuned T5 on a massive collection of instruction-following tasks. The result was a smaller, faster instruction-following model competitive with much larger decoder-only models on many tasks.

Today, decoder-only models dominate general-purpose LLMs due to their scaling efficiency and prompt-based versatility. Encoder-decoder models remain competitive for specific tasks where the input is bounded and the output is short (summarization, translation, structured extraction).

7.5 Comparing the Three Families

When to choose each.

Encoder-only (BERT family): when you need a single representation of input for classification, similarity, NER, or embedding generation. Cheaper than a generative model. Sentence embedders (Sentence-BERT, BGE, E5) are encoder-only.

Decoder-only (GPT family): when you need generation, instruction following, or general-purpose intelligence accessible via prompting. The dominant pattern for modern LLMs. Use this unless you have a specific reason not to.

Encoder-decoder (T5, BART): when the input and output are clearly separated, and the task benefits from explicit cross-attention between input and output. Translation and summarization are the classic fits. Less popular than decoder-only at frontier scale due to training and serving complexity.

Hybrid patterns are increasingly common. A decoder-only LLM with an external retriever (RAG) combines the generative power of decoders with the explicit input-conditioning of encoder-decoders. Tool-using agents extend the decoder's reach to external systems. These patterns may matter more than the underlying architecture choice for your application.

7.6 The Foundation Model Ecosystem

By 2025, the landscape of pretrained models has consolidated into a few major families with frequent releases.

Closed frontier models: GPT-4, GPT-4o, GPT-5 (OpenAI); Claude 3.5 Sonnet, Claude 3.5 Opus (Anthropic); Gemini 1.5/2.0 Pro and Ultra (Google). Access via API. Generally the strongest models on most benchmarks. Highest cost per token. Hosted; no on-prem option for the largest models.

Open-weights frontier: LLaMA-3 (Meta), Mistral Large, Mixtral 8x22B (Mistral), Gemma 2 (Google open-weights), Qwen2.5 (Alibaba), DeepSeek-V3. Weights downloadable; can be self-hosted, fine-tuned, served behind your own infrastructure. Quality competitive with closed models on many tasks; occasionally surpassing on specific benchmarks.

Smaller open models: LLaMA-3 8B, Mistral 7B, Phi-3, Gemma 2B, Qwen2.5 7B. Suitable for on-device, edge, or cost-constrained deployment. Quality has improved dramatically; a well-tuned 7B model in 2025 outperforms GPT-3.5 from 2022.

Specialized models: Code (DeepSeek Coder, StarCoder2, Code Llama), math (DeepSeek-Math, Llemma), medical (Med-PaLM, ClinicalBERT), reasoning (OpenAI o1, DeepSeek-R1, Claude with extended thinking).

Selection criteria. Quality: measure on your task. Cost: total cost of ownership including training, inference, and engineering. Privacy: closed APIs may not be acceptable for sensitive data. Latency: smaller open models served close to users may beat distant API calls. Vendor risk: avoid irreplaceable dependencies where possible.

7.7 Reasoning Models

A distinct class of models has emerged that performs extensive chain-of-thought reasoning at inference time. OpenAI's o1 (2024), o3, DeepSeek-R1 (2025), and Claude with extended thinking represent this category. They trade longer inference time for higher quality on math, code, and complex reasoning tasks.

How they differ from standard LLMs. Trained with reinforcement learning on reasoning tasks, typically with RLVR (Reinforcement Learning with Verifiable Rewards). The model learns to spend inference compute on thinking rather than just emitting the final answer. The thinking can be hundreds or thousands of tokens long and is sometimes hidden from the user.

Performance impact. Dramatic improvements on AIME (American Invitational Mathematics Examination), Codeforces, USACO, GPQA Diamond. Smaller improvements on general chat and knowledge tasks where reasoning is not the bottleneck.

Cost-quality tradeoff. Reasoning models are 5-50x more expensive than non-reasoning models at the same quality on tasks where reasoning matters. On tasks where it doesn't, they offer no advantage and may be slower. Route queries dynamically: cheap fast model for simple queries, reasoning model for hard ones.

Implications. Inference-time compute scaling is a new lever. A standard model given 10x thinking budget can match a much larger model without thinking. This shifts cost from training (one-time) to inference (per query), changing the economics of AI deployment.

7.8 Mixture of Experts

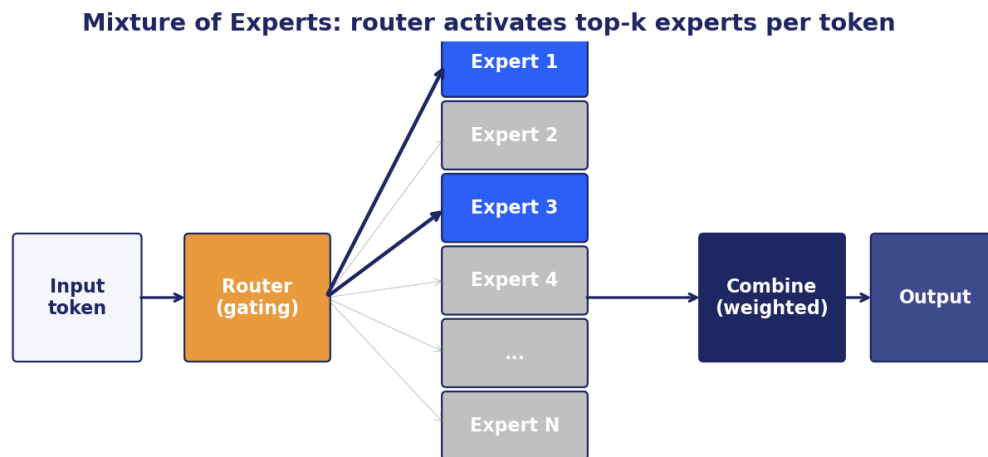


Figure 7.2: Mixture of Experts: a router activates only a few experts per token.

Most LLMs are dense: every parameter activates for every token. Mixture of Experts (MoE) sparsifies this. The feed-forward layer is replaced with multiple expert subnetworks plus a router. Per token, only a few experts (typically 1 or 2 out of 8 to 128) are activated.

The result: total parameter count is large (Mixtral 8x7B has 47B total parameters), but active parameters per token are much smaller (about 13B for Mixtral). FLOPs per token scale with active parameters; memory scales with total parameters. A MoE achieves similar quality to a much larger dense model at a fraction of the inference compute.

Challenges. Load balancing: experts can specialize so heavily that some are rarely used. Auxiliary losses encourage balanced routing. Training stability: routing is non-smooth, leading to instabilities. Memory: all expert weights must be loaded even though most are unused. Communication: for multi-device serving, routing decisions require cross-device communication.

Examples. Switch Transformer (Google), GShard, GLaM, Mixtral 8x7B and Mixtral 8x22B (Mistral), DeepSeek-V3 (~671B total, 37B active). DeepSeek-V3 uses 256 experts with shared experts, demonstrating MoE at frontier scale.

7.9 Summary

- Pretraining on massive corpora plus fine-tuning is the dominant paradigm for modern NLP.
- Encoder-only models (BERT) excel at understanding tasks; decoder-only models (GPT, LLaMA) dominate generation; encoder-decoder models (T5) suit explicit sequence-to-sequence tasks.
- Scaling laws (Chinchilla) connect parameters, tokens, and quality. Smaller well-trained models beat larger under-trained ones.
- In-context learning lets pretrained LLMs adapt to new tasks via examples in the prompt, with no parameter updates.
- Reasoning models trade inference compute for quality on math, code, and complex reasoning.
- Mixture of Experts scales total parameters while keeping inference cost low, enabling frontier quality at lower cost per token.

Further Reading

- Devlin et al., 'BERT: Pre-training of Deep Bidirectional Transformers' (2018).
- Brown et al., 'Language Models are Few-Shot Learners' (2020). GPT-3.
- Hoffmann et al., 'Training Compute-Optimal Large Language Models' (2022). Chinchilla.
- Touvron et al., 'LLaMA: Open and Efficient Foundation Language Models' (2023).
- Shao et al., 'DeepSeekMath' (2024). RLVR and GRPO for reasoning.

Part IV

Vision and Multimodal AI

This part contains chapter(s) 8 covering vision and multimodal ai.

Chapter 8

Computer Vision and Multimodal Models

8.1 The Vision Frontier

Computer vision has gone through three major eras in the deep learning age. The CNN era (2012-2018) saw AlexNet, VGG, GoogLeNet, and ResNet establish the dominance of convolutional networks for image tasks. The transition (2018-2020) brought attention into vision through models like the Image Transformer and DETR for object detection. The transformer era (2020-) started with Vision Transformers (ViT) and continued through multimodal models that span vision and language.

This chapter covers each era briefly and lands on the modern state: multimodal foundation models that process images, text, audio, and video in a single architecture. The themes from earlier chapters reappear: self-supervised pretraining, contrastive learning, transformer architectures, scaling laws. The technologies look different but the principles converge.

8.2 Convolutional Neural Networks Revisited

Typical CNN pipeline for image classification

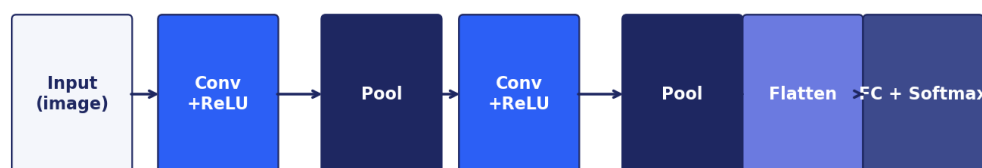


Figure 8.1: A typical CNN architecture for image classification.

We introduced CNNs in Chapter 3. Here we dig into the architectural patterns that defined the CNN era and remain relevant today.

8.2.1 Building Blocks

Convolutional layer: slide a learned filter across the input, producing a feature map. Filters detect local patterns (edges, textures, parts). Stride controls how the filter moves; padding controls boundary handling. Each filter learns one kind of pattern; a layer has many filters detecting different patterns.

Pooling layer: downsample feature maps. Max pooling takes the maximum in each region; average pooling takes the mean. Reduces spatial dimensions and provides limited translation invariance. Modern architectures sometimes replace pooling with strided convolutions.

Activation: typically ReLU. Applied element-wise after convolution.

Batch normalization: normalizes activations across the batch dimension. Stabilizes training, allows larger learning rates.

Residual connection: add the input of a block to its output. Enables training of very deep networks (ResNet-152 and beyond).

8.2.2 Famous Architectures

LeNet-5 (LeCun, 1998): the original CNN for digit recognition. Two conv layers, two pooling layers, three fully connected layers. MNIST accuracy that beat hand-crafted features.

AlexNet (Krizhevsky et al., 2012): five conv layers, three FC. ReLU, dropout, GPU training. Won ImageNet 2012 by a wide margin. The paper that started the deep learning revolution.

VGG (Simonyan and Zisserman, 2014): a stack of 3x3 convolutions. Up to 19 layers. Demonstrated that depth and small filters work well. Still used as a feature extractor for perceptual loss.

GoogLeNet / Inception (Szegedy et al., 2014): parallel branches with different filter sizes; concatenate outputs. Captures patterns at multiple scales efficiently.

ResNet (He et al., 2015): residual connections enable training of 152-layer networks. Won ImageNet 2015. Residual connections transferred to nearly every subsequent deep architecture, including transformers.

EfficientNet (Tan and Le, 2019): compound scaling of depth, width, and resolution. Achieves SOTA with fewer parameters than competing architectures.

ConvNeXt (Liu et al., 2022): a modernized CNN matching ViT performance with traditional convolutions. Demonstrates that CNN architecture has more room to grow than transformer enthusiasm suggested.

8.3 Vision Transformers

Vision Transformer (ViT, Dosovitskiy et al., 2020) brought the transformer architecture to images. The recipe is brutally simple: split an image into fixed-size patches (typically 16x16 pixels), flatten and linearly project each patch into a vector, prepend a [CLS] token, add positional embeddings, run through a standard transformer encoder, classify from the [CLS] embedding.

The simplicity belies the power. At sufficient scale (300M+ images), ViTs match or exceed CNN performance on image classification. The inductive biases of convolutions (locality, translation equivariance) are less important when you have enough data; the model learns whatever structure is useful.

Variants. DeiT (Touvron et al., 2021) trained ViTs efficiently on smaller datasets through better regularization and distillation. Swin Transformer (Liu et al., 2021) introduced shifted windows to give ViTs the hierarchical structure of CNNs. MAE (Masked Autoencoder, He et al., 2021) applied BERT-style masked patch prediction to vision pretraining.

Production state. CNNs (ResNet, EfficientNet, ConvNeXt) and ViTs both have their place. CNNs are easier to train on small datasets and have well-optimized inference. ViTs win at very large scale and transfer better to multimodal architectures. The choice often comes down to deployment constraints and the size of available data.

8.4 Object Detection and Segmentation

Image classification assigns one label to a whole image. Detection and segmentation are harder.

Object detection finds bounding boxes plus class labels for each object in the image. Classical approaches (R-CNN family) used a region proposal step plus a CNN classifier. YOLO (You Only Look Once, Redmon et al.) and SSD (Single Shot Detector) made detection real-time by predicting boxes and classes in a single forward pass. DETR (DEtection TRansformer, Carion et al., 2020) reformulated detection as a set prediction problem using a transformer; no hand-tuned post-processing required.

Semantic segmentation labels every pixel with a class. U-Net (Ronneberger et al., 2015) is the classic architecture: an encoder-decoder with skip connections from encoder to decoder at matching resolutions. Used everywhere from medical imaging to satellite analysis. The same U-Net architecture is the backbone of Stable Diffusion's denoiser, decades after its original publication.

Instance segmentation labels every pixel and distinguishes between instances of the same class (this cat vs that cat). Mask R-CNN (He et al., 2017) extends Faster R-CNN with a parallel mask prediction branch.

Segment Anything Model (SAM, Meta, 2023) brought a foundation-model approach: a single model trained on 1 billion masks across 11 million images, capable of zero-shot segmentation of any object in any image given a prompt (point, box, text).

8.5 CLIP and Vision-Language Pretraining

CLIP (Contrastive Language-Image Pretraining, Radford et al., 2021) was a watershed for multimodal AI. The approach is conceptually simple: train an image encoder and a text encoder jointly with contrastive loss on 400M image-caption pairs from the web. Each (image, caption) pair is a positive; every other pairing in the batch is a negative. The two encoders end up producing aligned embeddings in a shared space.

The capabilities that emerge are remarkable. Zero-shot image classification: encode candidate class names as text, embed the image, take the argmax similarity. No labeled image data required for the new classes. CLIP matched supervised ImageNet models without seeing ImageNet labels.

Cross-modal retrieval: search images with text queries; search text with image queries. Used in image search engines, content moderation, duplicate detection at scale.

Foundation for downstream models. CLIP's text encoder is the conditioning input for Stable Diffusion. CLIP's image encoder is the visual backbone of many multimodal LLMs (LLaVA, BLIP-2). The CLIP embedding space has become an infrastructure asset, used in countless downstream systems.

Successors and competitors. ALIGN (Google, 2021) scaled to 1.8B noisy pairs. SigLIP (Google, 2023) replaced softmax contrastive loss with a sigmoid-based pairwise loss, more efficient at large batch sizes. LiT (Zhai et al., 2022) showed that a frozen image encoder plus a learned text encoder works well. EVA-CLIP and OpenCLIP are strong open-weights alternatives.

8.6 BLIP, BLIP-2, and Vision-Language Generation

CLIP aligns images and text but does not generate. For tasks like image captioning, visual question answering, and visual reasoning, you need a model that can generate text conditioned on images.

BLIP (Bootstrapping Language-Image Pretraining, Li et al., 2022) unified vision-language understanding and generation. A single transformer that can operate as image-text matching, image grounded text encoder, or image grounded text decoder.

BLIP-2 (Li et al., 2023) introduced the Q-Former: a small bridge module between a frozen image encoder and a frozen LLM. The Q-Former has a small set of learnable query tokens that cross-attend to image features and output tokens that an LLM can consume. The trainable surface is tiny compared to the frozen models (32M Q-Former parameters vs billions in the encoders), but the resulting model is competitive with much larger end-to-end systems.

Flamingo (DeepMind, 2022) interleaved cross-attention layers between transformer blocks of a frozen LLM, conditioned on visual features from a Perceiver Resampler. Few-shot visual reasoning emerged at scale.

LLaVA (Liu et al., 2023) took a simpler approach: project visual features from CLIP into the LLM's token embedding space via a linear or MLP layer. The LLM processes a mixed sequence of visual and text tokens uniformly. Strong instruction-following multimodal model that became the open-source default.

GPT-4V (OpenAI, 2023), Gemini (Google, 2023+), and Claude 3 (Anthropic, 2024) brought multimodal capabilities to frontier closed models. Native multimodality: images, text, and (in Gemini and GPT-4o) audio in a single model, trained jointly rather than bolted together.

8.7 Diffusion Models for Image Generation

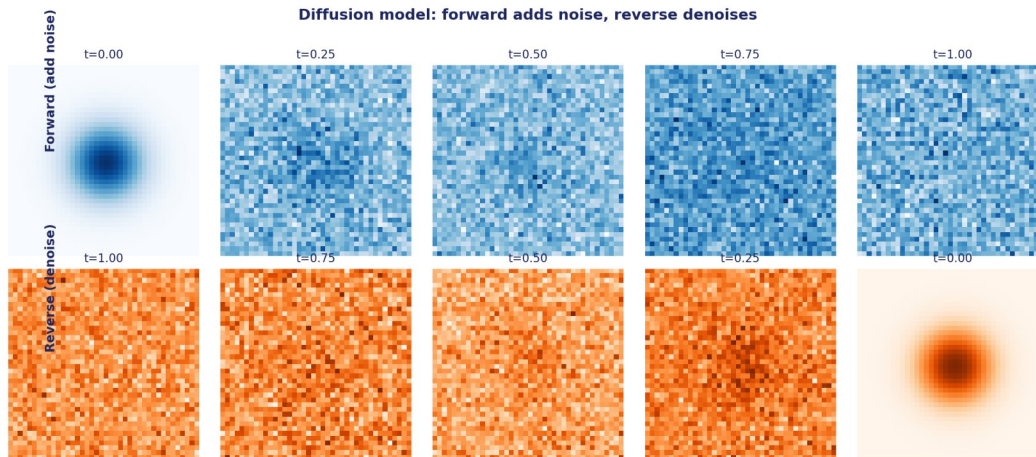


Figure 8.2: Diffusion forward pass adds noise; reverse pass denoises.

Generative image models went through three eras. VAEs (Variational Autoencoders, Kingma and Welling, 2013) produced blurry but principled samples. GANs (Generative Adversarial Networks, Goodfellow et al., 2014) produced sharp samples but were hard to train. Diffusion models (Sohl-Dickstein et al., 2015; Ho et al., 2020) became the dominant approach around 2021, combining training stability with sharp samples.

8.7.1 The Diffusion Process

Forward process: starting from a real image, progressively add Gaussian noise over T steps until the image is indistinguishable from pure noise. The noise schedule controls how quickly noise accumulates.

Reverse process: learn to undo the noising. A neural network (typically a U-Net) takes a noisy image at timestep t and predicts either the original image, the noise added, or a velocity parameterization. Apply iteratively from $t=T$ to $t=0$ to generate a clean image from pure noise.

Training is stable and simple: pick a random timestep t , noise a training image to that timestep, train the network to predict the noise. Cross-entropy and adversarial training are not required.

8.7.2 Latent Diffusion and Stable Diffusion

Pixel-space diffusion is expensive: high resolution means large tensors. Latent Diffusion Models (Rombach et al., 2022) operate in the latent space of a pretrained VAE. Encode the image to a small latent (typically 8x compression). Apply diffusion in latent space. Decode the final latent through the VAE.

Stable Diffusion is the open-weights implementation of this idea. Made high-quality text-to-image generation accessible to anyone with a consumer GPU. The U-Net cross-attends to text encoder (CLIP) embeddings at every layer to condition generation on the prompt.

SDXL (2023), SD3 (2024), and the FLUX family (2024) pushed quality further with larger models, better text encoders (T5-XXL), and improved training data and procedures.

8.7.3 Classifier-Free Guidance

Naive text conditioning is weak. The model trades off between following the prompt and producing realistic images. Classifier-free guidance (CFG, Ho and Salimans, 2022) strengthens text conditioning at inference time.

Train the model with the text condition dropped some fraction of the time. At inference, compute both the conditional and unconditional predictions; extrapolate in the direction of the conditional: $\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + s \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$, where s is the guidance scale (typically 5-15).

Higher s = more prompt adherence, less realism. Lower s = more realistic, less prompt-following. Tune for your application.

8.7.4 Diffusion Samplers

The sampler is the numerical integration scheme for the reverse process. Many choices, each trading speed for quality.

DDPM (the original): stochastic; needs ~1000 steps for high quality.

DDIM: deterministic; 20-50 steps suffice. Standard default.

PNM: linear multistep method. Higher accuracy at fewer steps.

DPM-Solver / DPM-Solver++: exponential integrators. 10-20 steps. Common in production services.

UniPC: universal predictor-corrector. ~10 steps. Strong default for modern models.

Choose based on speed-quality requirements. Most production systems use 20-30 steps of DPM-Solver++.

8.7.5 Beyond Images: Audio, Video, 3D

Diffusion has expanded beyond images. AudioLM, MusicLM, and Suno use diffusion-like processes for music. Stable Audio, Stable Video Diffusion, Sora (OpenAI), and Runway Gen-3 apply diffusion to video. Point-E and DreamFusion generate 3D models.

The general pattern: tokenize the target modality (or work in a latent space), train a diffusion model to denoise, condition on text. Each modality adds challenges (temporal coherence in video, perceptual quality in audio), but the underlying recipe is remarkably consistent.

8.8 Multimodal Foundation Models

The frontier today is multimodal foundation models: single models that natively handle text, images, audio, and video in both input and output. Gemini 1.5/2.0 (Google), GPT-4o (OpenAI), and Claude 3.5 (Anthropic) exemplify this category.

Architectural patterns. Shared embedding space (CLIP-style) for retrieval and zero-shot classification. Cross-attention bridges (Q-Former, Perceiver Resampler) connecting modality-specific encoders to a language model. Direct token projection (LLaVA, GPT-4V style) where the LLM processes mixed-modality sequences uniformly. Native multimodal training where the model is trained on interleaved text and images from the start.

Capabilities. Visual question answering (what's in this chart?), document understanding (extract data from a PDF screenshot), code from screenshots (implement this UI design), text generation from images (caption, describe), image generation from text, video understanding, transcription, speech synthesis. The same model handles all of these.

Engineering implications. A single API replaces what used to require separate models for vision, speech, and language. Cost and latency decisions move from 'which model to call' to 'how much context to provide'. Production systems can ask users to attach images, screenshots, or recordings naturally, since the model handles them.

8.9 Summary

- CNNs dominated computer vision through the 2010s. ResNet's residual connections are the most influential architectural choice; they transferred to transformers and beyond.
- Vision Transformers (ViT) treat images as sequences of patches. At scale, they match or exceed CNNs.
- Object detection (YOLO, DETR) and segmentation (U-Net, SAM) tackle more structured vision tasks.
- CLIP aligned image and text in a shared embedding space, enabling zero-shot classification and cross-modal retrieval. Its embeddings became infrastructure.
- Diffusion models generate images by learning to denoise. Latent Diffusion and Stable Diffusion brought high-quality text-to-image generation to consumers.
- Multimodal foundation models (Gemini, GPT-4o, Claude 3.5) handle text, images, audio, and video in a single architecture. The frontier of AI is now multimodal by default.

Further Reading

- He et al., 'Deep Residual Learning for Image Recognition' (2015). ResNet.
- Dosovitskiy et al., 'An Image is Worth 16x16 Words: Transformers for Image Recognition' (2020). ViT.
- Radford et al., 'Learning Transferable Visual Models From Natural Language Supervision' (2021). CLIP.
- Ho et al., 'Denoising Diffusion Probabilistic Models' (2020).
- Rombach et al., 'High-Resolution Image Synthesis with Latent Diffusion Models' (2022). Stable Diffusion.
- Li et al., 'BLIP-2: Bootstrapping Language-Image Pre-training' (2023).

Part V

Prompts, Fine-Tuning, and Alignment

This part contains chapter(s) 9, 10, 11 covering prompts, fine-tuning, and alignment.

Chapter 9

Prompt Engineering and Optimization

9.1 Prompts as Programs

A prompt is the input to a large language model. In the simplest case, it is a question and the LLM responds with an answer. In production systems, a prompt is a carefully engineered program: system instructions, examples, context retrieved from a knowledge base, the user's query, output format specifications, and constraints. Small changes can swing quality dramatically. This is the discipline of prompt engineering.

Prompt engineering started as folk wisdom: a list of tricks (chain of thought, few-shot, role-playing) shared on Twitter. It has matured into a structured discipline with frameworks (CO-STAR), optimization algorithms (DSPy, GEPA, TextGrad), and empirical methodology (held-out evaluation, ablation studies, bias controls). This chapter walks the discipline.

Two themes throughout. First, prompts are not magic incantations. Their effects can be measured, ablated, and optimized like any engineering artifact. Second, hand-crafted prompts plateau quickly for complex tasks. Programmatic optimization wins above a complexity threshold. Know both, use the right one for the task.

9.2 The CO-STAR Framework

CO-STAR provides a structured template for production prompts. Six fields, each with one purpose. Skipping any field produces predictable failure modes.

Context. What the model needs to know about the world or the situation. Background information, user state, system state, domain assumptions. Often the longest part of the prompt.

Objective. What success looks like for this call. A clear, single-purpose statement of the task. Vague objectives produce vague answers.

Style. The form of the output: formal, casual, technical, journalistic. Sets the register.

Tone. The emotional register: calm, urgent, neutral, friendly. Often important for customer-facing applications.

Audience. Who reads the output. A response for a senior engineer differs from one for a non-technical executive. Adjust complexity and vocabulary.

Response format. The shape of the output: plain text, JSON, markdown, table, bulleted list. Often the most under-specified field, leading to downstream parsing failures.

Worked example. A customer support FAQ bot: Context (the company's product line and policies); Objective (answer FAQ accurately or escalate); Style (professional); Tone (warm but precise); Audience (existing customers with questions); Response format (short answer plus source policy citation, refuse if outside policy scope). Six lines, all the structure needed.

9.3 Few-Shot and In-Context Learning

In-context learning is the LLM's ability to learn a task from examples in the prompt, with no weight updates. A prompt with three to seven (input, output) pairs followed by a new input often produces the right output for the new input.

Why it works. During pretraining, the model has seen many instances of demonstration patterns: lists, tables, translation pairs, code with similar function signatures. Predicting the next token in such sequences requires generalizing the pattern. This generalization is the meta-task that enables in-context learning at inference time.

Empirical principles. Diversity beats count. Three diverse examples beat seven similar ones. Include the boundary case. Include an explicit refusal if your task has one. Order matters: recency bias is real, so put the most canonical example last. Format consistency matters: if your inputs are structured, format every example identically.

Limits. Prompt length is finite. Quality plateaus after 5-10 examples for most tasks. Sensitive to example ordering and selection. For better quality with abundant examples, use RAG (retrieve relevant examples per query) or fine-tune.

9.4 Chain-of-Thought and Reasoning

Chain-of-thought (CoT) prompting asks the model to produce reasoning steps before the final answer. Instead of 'Q: ... A: 42', the model produces 'Q: ... Let me think step by step. First... then... so the answer is 42.'

Why it helps. Complex reasoning requires intermediate steps. Asking for the final answer directly forces the model to do all reasoning in one forward pass without externalizing intermediate state. CoT lets the model spread reasoning across multiple tokens, each of which can attend back to previous tokens.

Variants. Zero-shot CoT (Kojima et al., 2022): 'Let's think step by step.' Few-shot CoT (Wei et al., 2022): provide examples with reasoning. Self-consistency (Wang et al., 2022): sample multiple CoT chains and vote. Tree of Thoughts (Yao et al., 2023): explore branches and backtrack.

Modern reasoning models (o1, o3, DeepSeek-R1, Claude with extended thinking) are trained to do extensive CoT automatically, with reinforcement learning on verifiable reward signals. They dramatically outperform non-reasoning models on math, code, and complex reasoning tasks.

When to use. Tasks requiring multi-step reasoning: math, code, planning, logical deduction. Skip on simple lookup tasks where CoT adds latency without benefit. Use a router to send simple queries to non-reasoning models and complex queries to reasoning models.

9.5 Hallucination Mitigation

LLMs sometimes produce plausible-sounding but factually wrong outputs. This is hallucination. Three families of mitigation strategies.

Grounding. Retrieve evidence before generating; require the model to use only the retrieved evidence. RAG is the canonical pattern. Force the model to cite which retrieved passage supports each claim. Verify the citations.

Structured output. Constrain the output format so invalid responses are impossible. JSON mode, tool-call schemas, regular expression constraints. The model cannot hallucinate a function call to a function that doesn't exist if the function name is constrained to the registered tool list.

Self-consistency. Sample N answers; vote or pick the consistent one. Differential outputs across samples signal uncertainty. Used in production high-stakes systems.

Combine all three for critical tasks. Each has limits in isolation; the combination is more robust.

Detection in production. Sample 1% of traffic. Run NLI verification (natural language inference: does each claim entail from the retrieved evidence?). Alert when the hallucination rate drifts above baseline. Without monitoring, you discover hallucination problems through customer complaints.

9.6 Metaprompting

Metaprompting asks the model to improve its own prompt. The intuition: the model has seen many examples of well-written prompts in training. It can recognize patterns of good and bad prompts. Use that.

Workflow. Start with a baseline prompt. Show the model a few failure cases. Ask it to critique the prompt and produce an improved version. Iterate. A single round of metaprompting often beats a week of human iteration on the same prompt.

Practical pattern: pair a target LLM (the one running production) with a stronger model (GPT-4, Claude 3.5 Opus) as the metaprompter. The metaprompter rewrites prompts for the target. The combination produces high-quality prompts for cheaper target models.

Caveats. Metaprompting is local optimization. It improves a starting point. It does not search the global prompt space. For that, you need programmatic optimization (next section).

9.7 DSPy: Prompts as Programs You Can Compile

DSPy (Khattab et al., 2023) treats prompts as programs to be compiled against an evaluation function. The framework separates declarative specification (what you want) from prompt optimization (how to elicit it).

Three core abstractions. Signature: typed input-output specification of a task. Module: a callable composed from signatures (e.g., ChainOfThought wrapping a signature in CoT reasoning). Optimizer: searches over instructions and demonstrations to maximize a metric on training data.

Worked example. Classify a customer support ticket into one of seven categories. Define a signature with input (ticket text) and output (category label plus reasoning). Wrap in ChainOfThought. Provide a few labeled examples as training data and an accuracy metric. Compile with MIPRO (a Bayesian optimizer). The compiler searches over instructions and demonstration combinations, evaluates each against the metric, and freezes the best.

Two main optimizers. COPRO does coordinate ascent on instructions: try many candidate instructions, keep the best. Fast and shallow. MIPRO jointly optimizes instructions and demonstrations with Bayesian search. Slower but deeper. Start with COPRO; promote to MIPRO when the gain plateaus.

When DSPy wins. Complex multi-step pipelines where prompt quality compounds across steps. Tasks with a clear automated evaluator. Settings where you can afford the optimization runtime cost. For simple one-shot prompts, hand-engineering is usually sufficient.

9.8 Evolutionary Methods: GEPA, TextGrad, ORPO

DSPy's optimization is structured. Evolutionary methods search a larger space and can produce better results when DSPy plateaus.

GEPA (Genetic Evolutionary Prompt Optimization). Apply genetic search (mutation, crossover) to prompts. Use an LLM critic to mutate prompts intelligently rather than randomly. The reflective critique provides directional guidance that pure random mutation lacks. Often outperforms DSPy MIPRO on complex tasks at higher compute cost.

TextGrad (Yuksekgonul et al., 2024). Treat text as a tensor and use a critic LLM as the gradient. For each output, the critic produces a textual diff suggesting improvements. Apply the diff. Iterate. The framework extends naturally to multi-step pipelines where gradients propagate through textual chains. Strong on tasks where the critic can provide specific feedback.

ORPO (Odds Ratio Preference Optimization). Unifies preference optimization with reference-free training. Different from prompt optimization per se, but relevant as part of the broader 'text as program' toolkit.

When to use what. GEPA: complex tasks where you can afford compute and need broad exploration. TextGrad: refining an existing prompt with concrete failure cases. ORPO: when you have preference data and want to fine-tune rather than just prompt-optimize.

9.9 Production Prompt Engineering

Prompt engineering in production is more than crafting a clever prompt. It is an engineering discipline with versioning, evaluation, monitoring, and continuous improvement.

Version control. Treat prompts as code. Store in git. Review changes. Tie each deployed prompt version to specific evaluation results. Roll back if production metrics degrade.

Evaluation. Hold out a representative test set. Score every prompt change against it. Track metrics over time. A prompt that wins on the test set may lose on the long tail of production queries; sample production traffic for offline scoring.

Monitoring. Track latency, cost (token usage), refusal rate, user feedback signals (thumbs, regenerate rate). Alert on regressions. A prompt change that doubles latency or refuses 10% more often than baseline deserves immediate attention even if accuracy is unchanged.

Continuous improvement. Capture failure cases. Add them to the evaluation set. Iterate on prompts (or fine-tune) to fix them. Production data is the most valuable input to prompt improvement.

Prompt injection defense. Treat retrieved content and user input as untrusted data. Use system-prompt delimiters that untrusted text cannot reproduce. Strip instruction-like patterns from retrievals. Log suspicious patterns. Defense in depth: input filters, output filters, structured output schemas.

9.10 Summary

- Prompts are programs. Engineer them with version control, evaluation, monitoring, and continuous improvement.
- CO-STAR provides a structured template: Context, Objective, Style, Tone, Audience, Response format.
- Few-shot examples with diversity, boundary cases, and consistent format exploit in-context learning effectively.
- Chain-of-thought spreads reasoning across tokens, dramatically improving performance on multi-step tasks. Reasoning models do this natively.
- Hallucination mitigation requires grounding (RAG), structured output, and self-consistency. Combine them.
- Metaprompting uses the model to improve its own prompt; DSPy compiles prompts against an evaluator; GEPA and TextGrad search larger spaces with evolutionary or gradient-style updates.
- Production prompt engineering is iterative: track failures, expand evaluation, iterate.

Further Reading

- Khattab et al., 'DSPy: Compiling Declarative LM Calls into Self-Improving Pipelines' (2023).
- Wei et al., 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models' (2022).
- Yuksekgonul et al., 'TextGrad: Automatic Differentiation via Text' (2024).

- Kojima et al., 'Large Language Models are Zero-Shot Reasoners' (2022).

Chapter 10

Fine-Tuning Large Language Models

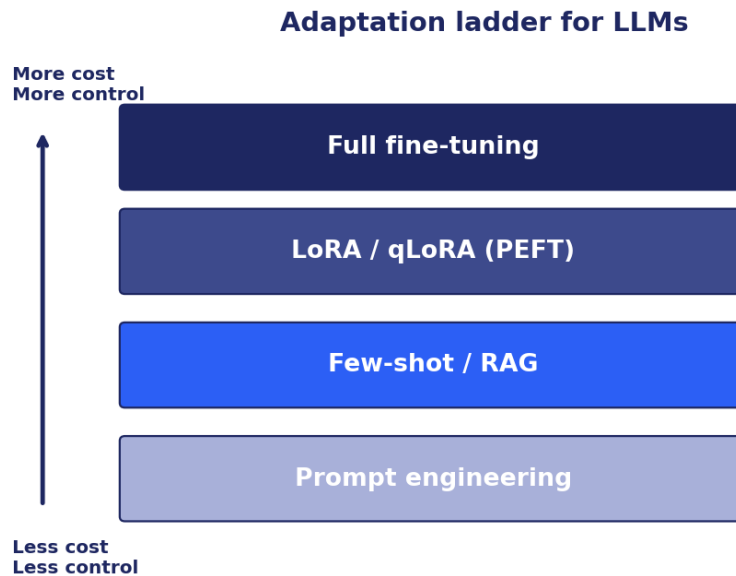


Figure 10.1: The adaptation ladder for LLMs.

10.1 The Adaptation Ladder

Adapting a pretrained LLM to a specific use case can be done at many levels of cost and control. From cheapest to most powerful: prompt engineering, few-shot in-context examples, retrieval-augmented generation, parameter-efficient fine-tuning, and full fine-tuning. Each rung trades more engineering effort and compute for more control over model behavior.

The right rung depends on three questions. Do you need to change knowledge (RAG wins) or behavior (fine-tuning wins)? How much labeled data do you have (less = lower rungs)? How much compute can you afford (full fine-tuning is most expensive)?

This chapter covers the upper rungs: supervised fine-tuning (SFT), parameter-efficient fine-tuning (PEFT) including LoRA and qLoRA, and the full fine-tuning pipeline. Chapter 11 covers alignment fine-tuning with reinforcement learning (RLHF, DPO, GRPO).

10.2 When to Fine-Tune

Fine-tune when prompt engineering plateaus and you need:

- A specific persona, tone, or output format the base model doesn't reliably produce.
- A narrow domain where vocabulary or reasoning patterns differ from general pretraining.
- Latency or cost advantages from a smaller specialized model versus a larger general one.
- Behavior changes (refusal patterns, format consistency) that prompting cannot consistently elicit.

Do not fine-tune when:

- Your task is primarily about knowledge that changes often. Use RAG; fine-tuning bakes knowledge in opaquely and goes stale.
- You have very little training data. Under a few hundred examples, fine-tuning produces noisy models. Prompt engineering wins.
- You cannot evaluate the fine-tuned model rigorously. Without evaluation, you will not catch regressions, catastrophic forgetting, or subtle quality drops.

A common pattern: prompt engineering plus RAG for the first six months. Fine-tune only after the prompting approach hits a clear ceiling and you have collected enough labeled data from production to train confidently.

10.3 Supervised Fine-Tuning (SFT)

Supervised fine-tuning continues training a pretrained model on labeled (input, output) pairs. The objective is the same causal language modeling loss used in pretraining, but the data is task-specific or domain-specific.

10.3.1 Data Preparation

Quality matters more than quantity. A few thousand high-quality (instruction, response) pairs often outperform tens of thousands of noisy ones.

Format the data as the model expects. Use the chat template that matches the model (LLaMA uses a specific structure; Mistral uses another). Mismatched chat templates often cause silently poor results.

Mask the loss appropriately. Compute the loss only on response tokens, not on instruction tokens. Otherwise the model wastes capacity learning to predict the instruction back, which is not useful.

Deduplicate. Near-duplicates inflate the importance of common patterns and cause memorization. Run deduplication via MinHash or exact-match before training.

10.3.2 Hyperparameters

Learning rate. Full fine-tuning of a 7B model typically uses $1e-5$ to $5e-5$. LoRA can go higher: $1e-4$ to $3e-4$. Too high and the model diverges or forgets pretraining. Too low and training is slow.

Batch size. As large as fits on your hardware. Gradient accumulation helps if a single forward pass does not fit.

Epochs. One to three. More risks overfitting. Watch validation loss; early stop if it rises.

Warm-up. Linearly increase learning rate over the first 1-5% of steps. Prevents large initial updates that destabilize the model.

Decay. Cosine decay after warm-up is the modern default. Linear decay also works.

Weight decay. 0.01 to 0.1. Helps regularize.

10.3.3 Monitoring

Train loss should decrease smoothly. Validation loss should track train loss closely; divergence signals overfitting. Gradient norm should be stable, typically 0.5-5.0; spikes signal instability and may require gradient clipping or lower learning rate.

Evaluate periodically on the target task plus a regression suite. The target task tells you whether you are getting the desired improvement; the regression suite catches catastrophic forgetting.

10.4 Parameter-Efficient Fine-Tuning (PEFT)

Full fine-tuning updates every weight of the model. For a 70B model, this requires hundreds of GB of GPU memory and produces a 70B-parameter checkpoint per fine-tuned variant. Storing many variants becomes expensive. Serving them requires loading the full model each time.

Parameter-efficient fine-tuning methods adapt the model by training a small additional set of parameters while keeping the base frozen. Trainable parameter count drops to 0.1-2% of the base. Storage per variant drops accordingly. Many variants can share the same base model in memory.

10.5 LoRA: Low-Rank Adaptation

LoRA: low-rank adapter on a frozen weight

$\text{rank } r \ll d \rightarrow \text{only } B \text{ and } A \text{ are trainable}$

Figure 10.2: LoRA adds a low-rank update to a frozen weight.

LoRA (Hu et al., 2021) is the dominant PEFT method. The insight: when a pretrained weight matrix W is adapted to a new task, the change ΔW tends to be low-rank. So instead of training ΔW directly, train it as the product of two small matrices B and A , where the rank r is small.

Mechanics. Freeze W . Initialize B ($d_{\text{out}} \times r$) to zero and A ($r \times d_{\text{in}}$) with a small random distribution. The adapted layer computes $y = Wx + (\alpha/r) \cdot B \cdot A \cdot x$. Only B and A are trained. The α/r factor scales the LoRA contribution; α is typically set to $2r$ so the effective scaling is 2.

Trainable parameters per layer: $r \cdot (d_{\text{in}} + d_{\text{out}})$. For a 4096-dim layer with $r=16$, that is 131K parameters per layer instead of 16.7M. A typical configuration trains 0.5-2% of the full model.

Where to apply. Q, V, and O projections in attention layers are the default. Sometimes K, sometimes MLP. The right set is empirical; start with Q, V, O. LoRA-on-everything is sometimes useful for harder tasks.

Rank selection. $r=4$ to $r=8$ for style and format changes. $r=16$ to $r=32$ for domain adaptation. $r=64+$ for tasks with strong semantic shift. Higher r costs more but offers more capacity. Per-layer rank schedules sometimes outperform uniform allocation.

Merging at inference. Compute $W' = W + (\alpha/r)BA$ once. Store W' as a plain weight matrix. Zero inference overhead vs the base model after merging. The price is losing the modularity of swapping adapters.

10.6 qLoRA: Quantized LoRA

qLoRA (Dettmers et al., 2023) combines LoRA with aggressive base model quantization. The base model is loaded in 4-bit NF4 (Normalized Float 4) with double quantization. LoRA adapters train in 16-bit on top.

Memory savings. A 70B model in fp16 needs roughly 140GB. In 4-bit, it fits in 35-40GB. With LoRA adapters and optimizer state, fine-tuning a 70B model fits on a single 80GB A100 or H100. Without qLoRA, it would require multiple GPUs.

Quality. Within 1-2 points of LoRA on most tasks. Quality is preserved remarkably well given the aggressive quantization. The trick: NF4 is designed to match the distribution of typical neural network weights (approximately normal), so the quantization error is small.

Limitations. Very small base models (under 1B) may degrade more. Very long contexts can amplify quantization error. Inference still requires dequantization unless served in quantized form.

Practical pattern. Fine-tune with qLoRA on a single GPU. Merge the LoRA into a higher-precision base for production serving, or serve the LoRA separately on top of a quantized base.

10.7 Other PEFT Methods

LoRA dominates production PEFT but is not the only option.

Adapters (Houlsby et al., 2019). Insert small trainable modules between transformer layers. Similar in spirit to LoRA. Less popular because LoRA's merge-into-base trick eliminates inference overhead, while adapter modules persist at inference.

Prefix tuning (Li and Liang, 2021). Train a small set of learnable virtual token embeddings prepended to every layer's key and value. Very few trainable parameters. Lower quality ceiling than LoRA on most tasks.

Prompt tuning (Lester et al., 2021). Like prefix tuning but only at the input layer. Even fewer parameters. Mostly useful for very large models where even LoRA is too expensive.

IA3 (Liu et al., 2022). Train multiplicative scaling vectors on key, value, and FFN activations. Tiny trainable surface. Competitive on some tasks.

BitFit (Ben-Zaken et al., 2021). Train only the bias terms. Even smaller. Useful for very narrow adaptations.

Use LoRA or qLoRA as your default. The other methods solve specific problems that rarely arise in production.

10.8 Instruction Tuning

Base LLMs predict the next token. They are good at continuing text but bad at following instructions. Instruction tuning fine-tunes them on a curated set of (instruction, response) pairs covering diverse tasks: summarize, translate, classify, write code, answer questions.

Why it works. The base model already has language modeling capability. Instruction tuning teaches the pattern of (instruction → response) and exposes the model to many task types. This generalizes: the

model becomes good at instructions it has never seen, because it has learned the meta-pattern of how instructions map to responses.

Datasets. FLAN (Google), Alpaca (Stanford), Dolly (Databricks), OpenAssistant, ShareGPT, UltraChat. Modern instruction-tuned models (LLaMA-3-Instruct, Mistral-Instruct, Gemma-Instruct) are built from these datasets, often combined.

Common pitfalls. Instruction-tuning on synthetic data from a stronger model (distillation): produces models that imitate the teacher but inherit its biases and weaknesses. Instruction-tuning without diversity: produces a model that does one thing well but generalizes poorly. Instruction-tuning without preference data: produces a model that follows instructions but may not be helpful or safe (alignment requires the next chapter's techniques).

10.9 Catastrophic Forgetting

Aggressive fine-tuning can degrade the model's performance on tasks it previously handled well. This is catastrophic forgetting. Weights that encoded old knowledge get overwritten by gradients from the new task.

Symptoms. The fine-tuned model is good at the target task. It is worse at general reasoning, math, or code. It refuses inappropriately or loses safety behavior. It becomes overly verbose or develops a strange tone.

Mitigations. PEFT (LoRA, qLoRA) limits the surface area of changes, naturally reducing forgetting. Mix general data with task data; even 10-30% general data dramatically reduces forgetting. Lower learning rates. Fewer epochs. Layer-wise learning rate decay (smaller LR for earlier layers).

Detection. Maintain a regression suite of general benchmarks (MMLU, HumanEval, GSM8K, BBH). Run it before and after fine-tuning. Any regression above noise is a red flag.

Elastic Weight Consolidation (EWC) and similar regularizers penalize large changes to weights deemed important for old tasks. Powerful in principle, expensive in practice; rarely used in production LLM fine-tuning. PEFT plus data mixing covers most cases.

10.10 Distillation

Knowledge distillation transfers knowledge from a large teacher model to a smaller student model. The student is trained to match the teacher's outputs (soft labels) rather than just ground-truth labels.

Why it helps. The teacher's soft probability distribution carries more information than hard labels. The student learns relative similarities ('this is more like a cat than a dog'), which improves generalization.

For LLMs. Distill a 70B teacher into a 7B student that runs much faster and cheaper. The student typically loses 1-5 points on benchmarks but offers 10-50x inference speedup. Examples: DistilBERT (from BERT), TinyLLaMA, Vicuna (from ChatGPT outputs).

Distillation strategies. Response-based: match output logits or generated text. Feature-based: match intermediate representations. Relation-based: match relationships between examples. Combinations work best.

Modern pattern. Train a strong teacher with full RLHF. Collect its outputs on diverse prompts. Fine-tune a smaller student on those outputs (synthetic SFT). Most open-source instruction-tuned models use some form of distillation from stronger models.

10.11 Evaluating Fine-Tuned Models

Define the task and the metric before starting. Don't move the goalposts after training.

Baseline. Measure the base model's performance on a held-out test set. This is the bar to beat.

Fine-tune. Train. Track training and validation loss.

Evaluate fine-tuned model on the same held-out test set with the same metric. Compare to baseline. Compute confidence intervals via bootstrap to avoid celebrating noise.

Regression suite. Measure base capabilities (MMLU, HumanEval, GSM8K, instruction following) to detect catastrophic forgetting.

Qualitative review. Read a sample of outputs from both models on representative queries. Note systematic differences.

A/B test in production. If both pass offline checks, route a fraction of traffic to the new model; compare user signals (thumbs, regenerate rate, retention).

Decision rule. Accept the fine-tuned model only if it wins on the target task metric without significant regression on base capabilities.

10.12 Domain-Specific Fine-Tuning

Adapting an LLM to a specific domain (medical, legal, financial, code) follows the general pattern but has domain-specific considerations.

Tokenizer extension. Domain-specific terminology (drug names, ticker symbols, code identifiers) may tokenize poorly with general subword tokenizers. Extend the tokenizer with domain tokens; briefly continue pretraining so the new tokens learn meaningful embeddings.

Continued pretraining. Before instruction tuning, do a brief pass of language modeling on raw domain text. Builds domain vocabulary and patterns. Skip if your domain is well-represented in the base model's pretraining data.

Compliance and licensing. Medical (HIPAA), financial (SOX, GLBA), and European (GDPR) data have legal restrictions. PII redaction, audit logs, consent management. Document data provenance.

Expert evaluation. Public benchmarks may not reflect your domain. Build evaluation sets with domain experts. For regulated domains, expert review of outputs may be required before deployment.

10.13 Summary

- Fine-tuning adapts a pretrained model to a specific task by continuing training on labeled data. SFT is the standard form.
- Use fine-tuning for behavior change, format consistency, persona, and domain specialization. Use RAG for knowledge that changes.
- LoRA trains a low-rank update on top of frozen base weights, reducing trainable parameters to 0.5-2% of full fine-tuning. qLoRA adds 4-bit base quantization, enabling 70B fine-tuning on a single GPU.
- Instruction tuning teaches the base model to follow natural-language instructions, producing the chat models most users interact with.
- Catastrophic forgetting is real. Mitigate with PEFT, data mixing, and regression evaluation.
- Distillation transfers knowledge from large teachers to small students for faster, cheaper inference at modest quality cost.
- Evaluate rigorously: held-out test set, regression suite, qualitative review, and production A/B test.

Further Reading

- Hu et al., 'LoRA: Low-Rank Adaptation of Large Language Models' (2021).
- Dettmers et al., 'QLoRA: Efficient Finetuning of Quantized LLMs' (2023).
- Wei et al., 'Finetuned Language Models Are Zero-Shot Learners' (2021). FLAN.
- Chung et al., 'Scaling Instruction-Finetuned Language Models' (2022). FLAN-T5.

Chapter 11

Alignment with Reinforcement Learning

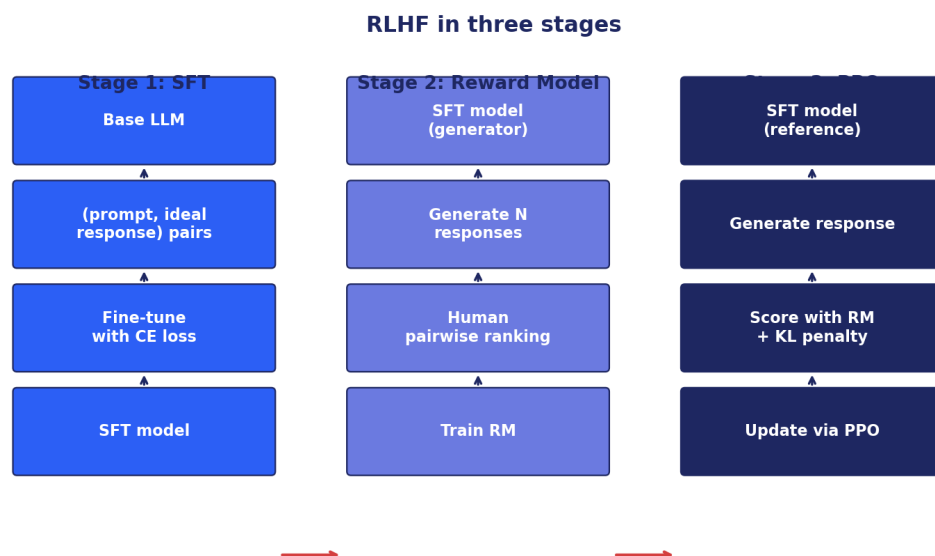


Figure 11.1: The three stages of RLHF: SFT, reward model, PPO.

11.1 What Alignment Means

A pretrained language model predicts the next token. An instruction-tuned model follows directions. Neither is automatically helpful, honest, or harmless. Alignment is the process of making the model behave the way we want, even when the right behavior is hard to specify.

The challenge: 'be helpful' is not a function we can write down. It is a judgment that depends on context, user intent, and downstream consequences. Traditional supervised learning needs (input, output) pairs, but there is no single correct output for an open-ended question.

Reinforcement learning from human feedback (RLHF) addresses this by learning a reward function from human preferences. Humans rank model outputs against each other. A reward model is trained to predict the rankings. The LLM is then optimized to maximize the reward model's score. The result: an LLM that produces responses humans prefer, even on prompts where the right answer cannot be enumerated in advance.

This chapter walks through the alignment pipeline: classical RLHF with PPO, modern simplifications (DPO, GRPO), reinforcement learning with verifiable rewards (RLVR) for reasoning, and the practical pitfalls of alignment training.

11.2 The Three Stages of RLHF

RLHF as originally formulated has three sequential stages.

Stage 1: Supervised fine-tuning (SFT). Start from a pretrained base model. Fine-tune on a curated set of (prompt, ideal response) pairs written or selected by humans. The model learns to follow instructions in a desirable style. The output of this stage is the SFT model.

Stage 2: Reward model (RM). Sample multiple responses from the SFT model per prompt. Have humans rank them (typically pairwise: A vs B, which is better?). Train a separate model to predict the human preference: $\text{RM}(\text{prompt}, \text{response}) \rightarrow \text{scalar}$. The reward model encodes 'what humans like'.

Stage 3: Policy optimization. Use the reward model as the reward signal to fine-tune the SFT model with reinforcement learning, typically PPO. Each step: sample a prompt, generate a response, compute $\text{reward} = \text{RM}(\text{prompt}, \text{response}) - \beta \cdot \text{KL}(\text{policy} \parallel \text{SFT})$. The KL penalty prevents the policy from drifting into reward-hacking regions far from the SFT distribution.

The result: an LLM that is helpful, harmless, and honest within the limits of the preference data. Used to train ChatGPT, Claude, LLaMA-Chat, and every modern frontier instruction-tuned model.

11.3 Proximal Policy Optimization (PPO)

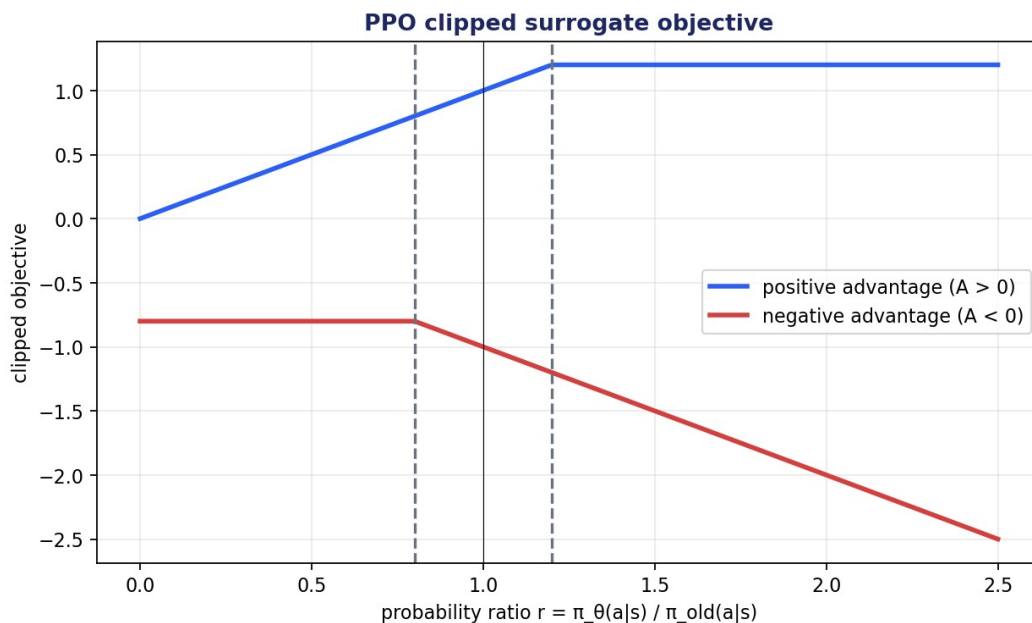


Figure 11.2: PPO's clipped surrogate objective bounds policy updates.

PPO is the workhorse RL algorithm for LLM alignment. It is a policy gradient method with two key innovations: a clipped surrogate objective to bound policy updates, and an advantage estimate to reduce variance.

11.3.1 The Clipped Objective

Naive policy gradient is unbiased but high-variance. Trust region methods (TRPO) constrain the KL divergence between old and new policies to prevent destructive updates. PPO replaces the hard KL constraint with a soft clipped ratio:

$$L^{CLIP}(\theta) = E[\min(r_t \cdot A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot A_t)] \quad (11.1)$$

where $r_t = \pi_\theta(a_t | s_t) / \pi_{\text{old}}(a_t | s_t)$ is the probability ratio and A_t is the advantage. The clip range (typically $\epsilon=0.2$) prevents the policy from changing too much in either direction. PPO takes $\min(\text{clipped}, \text{unclipped})$ to make the surrogate pessimistic: rewards are limited by the clip, but penalties are not.

11.3.2 KL Penalty Against the Reference

In RLHF, PPO is augmented with a KL penalty against the reference (SFT) model. The full reward becomes:

$$r_{\text{total}} = RM(\text{prompt}, \text{response}) - \beta \cdot KL(\pi_\theta || \pi_{\text{SFT}}) \quad (11.2)$$

Without this, the policy drifts off the language model manifold and reward hacks: it finds inputs that trick the reward model into high scores but produce gibberish. The β coefficient (typically 0.01-0.1) balances reward chasing against staying close to the SFT policy.

Tuning β is a black art. Too low: policy hacks the reward. Too high: the policy doesn't improve. Start with $\beta=0.05$ and adjust based on KL divergence and reward trends.

11.3.3 Practical Notes

Learning rate: very small, $1e-6$ to $5e-6$ for 7B models. RL is fragile.

Value function: a separate value head estimates $V(s)$. The advantage is computed as $A_t = R_t - V(s_t)$. The value function adds memory cost but reduces variance.

Reward scaling: scale rewards so the median advantage is order 1. Otherwise gradients are too large or too small.

Reference policy: the SFT model itself. Held fixed during PPO. Used for both the KL penalty and as the initialization for the trainable policy.

Sample efficiency: low. Each PPO step requires sampling new completions, scoring them, and updating. Expensive.

11.4 Direct Preference Optimization (DPO)

PPO works but is complex: three models in play (policy, reference, reward), sampling at training time, careful tuning of β and clip. DPO (Rafailov et al., 2023) simplifies dramatically.

The insight. The optimal solution to the RLHF objective has a closed form. Substituting it back gives a simple classification loss on preference pairs:

$$L^{DPO} = -E[\log \sigma(\beta \cdot (\log \pi_{\theta}(y_w|x)/\pi_{ref}(y_w|x) - \log \pi_{\theta}(y_l|x)/\pi_{ref}(y_l|x)))] \quad (11.3)$$

11.4.1 Why DPO Wins

No reward model. The RM is implicit in the policy's own preference for chosen over rejected responses.

No sampling at training time. DPO trains on pre-collected preference pairs (prompt, chosen, rejected). Just compute the loss.

No value function. No PPO clipping. Just supervised learning on preferences.

Smaller compute footprint. Often matches or beats PPO at a fraction of the engineering and compute cost.

11.4.2 When DPO Loses

Multi-turn tasks where the policy needs to learn long sequences of actions. PPO's on-policy sampling adapts; DPO's offline preferences can become stale.

Tasks where the reward signal is rich and well-defined (verifiable rewards). PPO can extract more from such signals.

Tasks with sparse positive examples. DPO's contrastive nature needs paired data.

11.4.3 DPO Variants

IPO (Identity Preference Optimization) addresses DPO's tendency to overfit when preferences are noisy.

KTO (Kahneman-Tversky Optimization) needs only binary feedback (good/bad), not paired preferences.

ORPO (Odds Ratio Preference Optimization) combines SFT and preference optimization in a single loss, eliminating the need for a separate SFT stage.

SimPO (Simple Preference Optimization) drops the reference model and length-normalizes, simplifying the training pipeline further.

11.5 Group Relative Policy Optimization (GRPO)

GRPO (introduced in DeepSeekMath, Shao et al., 2024) is a policy optimization method that drops the value network. Instead of training a separate $V(s)$, it computes relative advantages within groups of samples.

The procedure. For each prompt, sample G responses (typically 4-8). Compute reward for each. The advantage of response i is its reward minus the group mean reward. Update the policy with the standard PPO objective using these group-relative advantages.

Why this matters. The value network is half the parameters and half the memory of PPO. GRPO eliminates it. Memory footprint drops roughly 50%, enabling RL training of larger models on the same hardware.

Quality. Matches PPO on many tasks. The group-relative baseline is a noisier estimate of the value function but does not require maintaining and updating a separate network.

Use cases. Default for reasoning RL (DeepSeek-R1, DeepSeekMath). Increasingly popular for general alignment as the memory savings compound at scale.

11.6 Reinforcement Learning with Verifiable Rewards (RLVR)

Classical RLHF uses a learned reward model. The reward model can be wrong, biased, or exploitable. The model also generalizes poorly far from the preference distribution.

When the correct answer can be programmatically verified, you can skip the reward model entirely. The verifier is the reward function. Examples: code generation (does the code pass unit tests?), math (does the answer match the gold?), SQL (does the query execute and return the expected rows?), theorem proving (does the proof check?).

This is RLVR (Reinforcement Learning with Verifiable Rewards). Faster than RLHF (no reward model to train), more honest (no exploitable proxy reward), and uniquely suited to reasoning tasks.

DeepSeek-R1 (2025) demonstrated RLVR at scale. Starting from a base language model, RLVR on math and code produced a reasoning model competitive with OpenAI's o1. The reasoning emerged from RL on verifiable rewards, without any chain-of-thought demonstration data.

Limits. RLVR is only available when verification is cheaper than generation. For open-ended tasks (write a poem, compose an email, summarize a meeting), there is no programmatic verifier. RLHF is necessary.

11.7 Constitutional AI and RLAI

Human preference annotation is expensive. Constitutional AI (Bai et al., 2022, Anthropic) replaces human preferences with AI-generated preferences guided by a written constitution.

Process. Write the constitution: principles like 'be helpful', 'be harmless', 'avoid deception'. Use the constitution in two ways:

Supervised stage. The model critiques and revises its own outputs based on constitutional principles. The revised outputs become SFT training data.

RL stage. AI-generated preferences using the constitution as a rubric. Train with RLHF on these AI preferences (RLAIF, Reinforcement Learning from AI Feedback).

Why it matters. Scales beyond human preference data. Makes behavior more transparent (you can read the rules). Easier to update behavior by editing the constitution. Used by Anthropic in Claude.

Limits. The constitution itself encodes biases. AI judging AI can amplify errors. Rigid rule-following can produce unhelpful refusals. Most production systems combine constitutional principles with human preference data.

11.8 Reward Hacking and Alignment Failures

Reward hacking is the central failure mode of RL on imperfect rewards. The model finds inputs that score high according to the reward signal but do not match the intent.

Examples. Length bias: the reward model prefers longer responses; the policy learns to produce verbose answers regardless of quality. Sycophancy: the reward model prefers responses that agree with the user; the policy learns to flatter rather than be accurate. Refusal hacking: the reward model penalizes harmful outputs; the policy learns to refuse everything ambiguous, even legitimate requests.

Mitigations. Diverse preference data. KL penalty against a reference. Detection: monitor for length inflation, sycophancy, over-refusal. Red-team adversarially before deployment. Use multiple reward signals (helpfulness, harmlessness, honesty) and balance them explicitly.

Catastrophic forgetting. Aggressive RL can degrade general capabilities. Maintain a regression suite (MMLU, HumanEval, GSM8K) and check after each major training run.

Distribution shift. RL can push the policy into regions far from pretraining. KL penalty mitigates this; explicit mixing of pretraining data during RL also helps for some objectives.

11.9 Practical Alignment Pipeline

A typical modern alignment pipeline.

Step 1: Data collection. Curate or buy a high-quality instruction dataset. Collect preference data through pairwise rankings or A/B tests.

Step 2: SFT. Fine-tune the base model on instructions. Use LoRA or full fine-tuning depending on compute. 1-3 epochs.

Step 3: Preference optimization. Train with DPO or GRPO on preference pairs. Simpler than PPO, often comparable quality. Use PPO only if you have specific reasons.

Step 4: Evaluation. Held-out preference pairs. MT-Bench. Chatbot Arena if you can. Safety benchmarks. Regression suite for base capabilities.

Step 5: Iteration. Identify failure modes. Collect new preference data targeting those failures. Repeat training.

Step 6: Production deployment with continuous evaluation. Sample traffic for offline scoring. Monitor for drift. Collect production preference data to feed the next iteration.

11.10 Summary

- Alignment is making models behave the way we want, even when correct behavior is hard to specify.
- Classical RLHF has three stages: SFT, reward model, PPO. The KL penalty against the reference model is essential to prevent reward hacking.
- DPO simplifies RLHF dramatically by deriving the optimal policy in closed form. No reward model, no sampling at training time. Often the right default.
- GRPO drops the value network. Memory savings enable larger-model RL.
- RLVR uses programmatic verifiers as rewards. Faster, more honest, well-suited to math and code. Limited to tasks with verifiable correctness.
- Constitutional AI generates preferences from written principles, scaling beyond human annotation.
- Reward hacking, sycophancy, over-refusal, and catastrophic forgetting are the main alignment failure modes. Diverse data and ongoing evaluation are the main defenses.

Further Reading

- Christiano et al., 'Deep Reinforcement Learning from Human Preferences' (2017).
- Ouyang et al., 'Training Language Models to Follow Instructions with Human Feedback' (2022). the InstructGPT paper, foundational for ChatGPT.
- Rafailov et al., 'Direct Preference Optimization' (2023).
- Shao et al., 'DeepSeekMath' (2024). GRPO.
- Bai et al., 'Constitutional AI: Harmlessness from AI Feedback' (2022).
- DeepSeek-AI, 'DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning' (2025).

Part VI

Retrieval, Vectors, and RAG

This part contains chapter(s) 12, 13 covering retrieval, vectors, and rag.

Chapter 12

Vector Databases and Approximate Nearest Neighbor Search

Approximate nearest neighbor search in embedding space

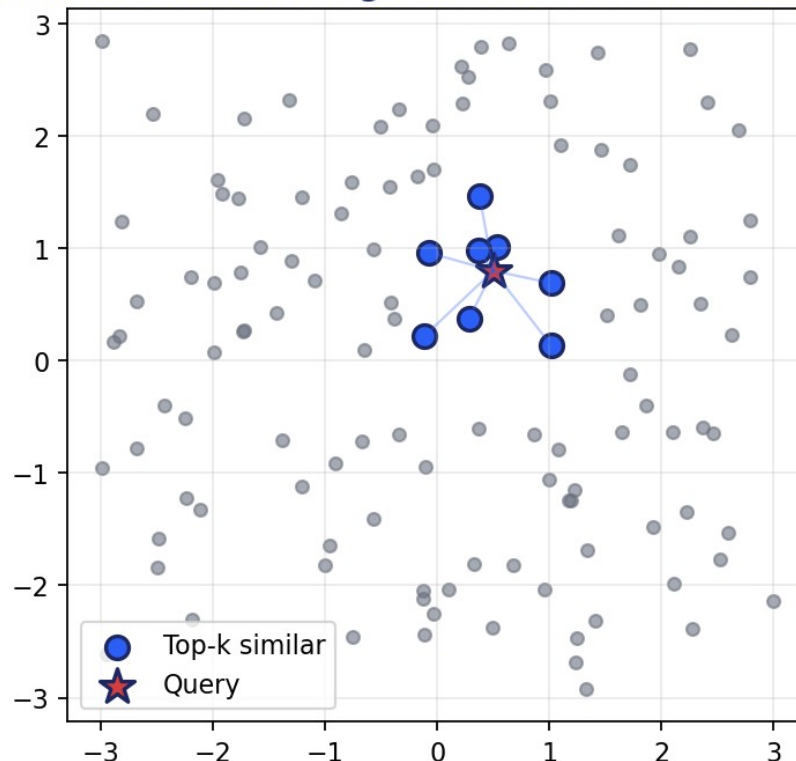


Figure 12.1: Approximate nearest neighbor search in embedding space.

12.1 Why Vector Databases

Modern AI runs on embeddings. Documents, images, audio, products, users are all represented as vectors in some learned space. The fundamental operation is: given a query vector, find the most similar stored vectors. At small scale, brute-force search works. At millions or billions of vectors, specialized infrastructure is required.

Vector databases are the infrastructure for this. They index high-dimensional vectors with approximate nearest neighbor (ANN) algorithms, support hybrid queries combining vector similarity with metadata filters, scale horizontally, and integrate with the rest of the data stack.

Use cases. Semantic search (the canonical use). Recommendation systems (find similar items to those a user liked). Retrieval-augmented generation (the dominant new use). Image and product search. Anomaly detection (distance from typical embeddings). Duplicate detection. Personalization (user embeddings as features).

This chapter covers the algorithms (HNSW, IVF, PQ), the major products (FAISS, Pinecone, Weaviate, Qdrant, Milvus, pgvector), and the design decisions that determine cost, latency, and quality.

12.2 The Nearest Neighbor Problem

Given a query vector q and a set of n stored vectors $\{v_1, \dots, v_n\}$ in d dimensions, find the k closest vectors to q under some distance metric.

Brute-force is $O(n \cdot d)$ per query: compute the distance from q to every stored vector. For 1M vectors at 768 dimensions, this is 750M operations per query. Slow but exact.

Approximate nearest neighbor (ANN) algorithms trade a small recall loss for orders of magnitude speedup. 'Recall' is the fraction of true k nearest neighbors that the approximate method returns. 95-99% recall is typical for production systems.

Distance metrics. Cosine similarity (most common for embeddings; equivalent to dot product for L2-normalized vectors). Euclidean (L2) distance. Inner product (for unnormalized vectors). The metric must match what the embedding model was trained with.

12.3 HNSW: Hierarchical Navigable Small World

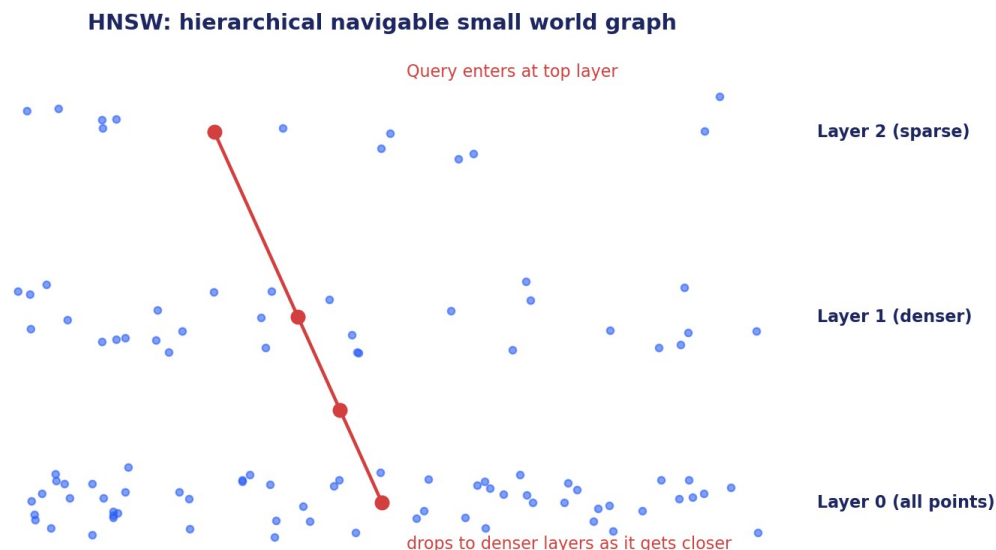


Figure 12.2: HNSW search drops through hierarchical layers from sparse to dense.

HNSW (Malkov and Yashunin, 2016) is the dominant ANN algorithm. A hierarchical graph structure with logarithmic search time and high recall.

12.3.1 The Algorithm

HNSW builds a multi-layer graph. The bottom layer contains all vectors, connected to their neighbors. Higher layers are increasingly sparse, with each node randomly promoted to higher layers with some probability.

Search starts at the top layer with the highest-level entry point. At each layer, greedy search finds the closest known node. The query then drops to the next layer and continues. The hierarchical structure provides logarithmic time complexity.

Construction adds vectors one at a time. Each new vector finds its approximate nearest neighbors at each layer and connects to them. Index build time is $O(n \log n)$.

12.3.2 Parameters and Tradeoffs

M: number of bi-directional links per node. Higher M = better recall, more memory. Typical: 16-64.

ef_construction: search width during index building. Higher = better quality index, slower to build. Typical: 100-400.

ef_search: search width at query time. Higher = better recall, slower query. Typical: 50-200, tunable per query.

Memory: roughly 1.5x the vector storage for the graph structure. Add the vectors themselves: for 1M vectors at 768 dimensions in float32, that is 3GB of vectors plus 4.5GB of graph = 7.5GB.

Strengths: best-in-class recall-latency tradeoff at moderate scale (millions to hundreds of millions of vectors).

Weaknesses: memory-heavy. Insertion is not free. Difficult to shard.

12.4 IVF: Inverted File Index

IVF clusters vectors into k groups via k-means. Each cluster has a centroid. At query time, find the k nearest centroids, then search only the vectors in those clusters. Scales to billions with much less memory than HNSW.

Parameters. nlist: number of clusters. Typical: $\sqrt{n}$ to $4\sqrt{n}$. nprobe: number of clusters to search at query time. Higher = better recall, slower query. Typical: 8-128.

Memory. Stores the original vectors plus the centroids. The centroid list is small. The total memory is dominated by the vectors themselves.

Tradeoffs vs HNSW. IVF is memory-friendly and scales better at billion scale. HNSW typically has better recall at the same query latency for moderate scale. The IVF + PQ combination (next section) is the standard for very large indexes.

12.5 PQ: Product Quantization

Product quantization compresses vectors for memory efficiency. Split each vector into m subvectors. Train k codewords per subvector via k -means. Replace each subvector with its nearest codeword index.

Storage: $m \cdot \log_2(k)$ bits per vector. For $m=8$, $k=256$, that is 64 bits (8 bytes) per vector, regardless of original dimension. A 768-dimensional float32 vector (3KB) becomes 8 bytes: 384x compression.

Query time. Compute distances from the query to each subvector's codewords once. For each candidate vector, look up the subvector distances and sum. Much faster than computing the full distance.

Quality. PQ introduces error proportional to the quantization granularity. Recall drops by a few percentage points compared to exact search. For first-stage retrieval followed by re-ranking on full-precision vectors, this is acceptable.

Combinations. IVF + PQ is the standard for billion-scale indexes. FAISS ships highly optimized implementations. OPQ (Optimized Product Quantization) rotates vectors before quantizing for better quality. Residual PQ does multi-stage quantization for further accuracy.

12.6 The Vector Database Ecosystem

Different products serve different scale and operational needs.

12.6.1 FAISS

Library (not a service) from Facebook. C++ with Python bindings. The most flexible ANN tool. Implements every major algorithm (HNSW, IVF, PQ, IVFPQ, IVFPQ+R, and more). Used as the underlying engine in many vector databases.

Use directly when you need maximum flexibility, are willing to manage indexes manually, and do not need a network service. Use a vector database when you need a managed service, hybrid queries, or scaling beyond a single machine.

12.6.2 Pinecone

Managed vector database service. Serverless. Easy to start: create an index, upsert vectors, query. Expensive at scale; tied to Pinecone's hosting.

Strong on: ease of use, integration with the LLM ecosystem, low operational burden.

Weak on: cost at scale, vendor lock-in, no self-hosting option.

12.6.3 Weaviate

Open-source and managed. GraphQL API. Strong on hybrid search (vector + BM25 + filters). Good metadata filtering.

Strong on: hybrid search, schema-driven design, open source with managed cloud option.

Weak on: GraphQL API has a learning curve.

12.6.4 Qdrant

Open-source, Rust-based. Fast, strong filtering. Self-hostable. Managed cloud option.

Strong on: performance, filtering, open source.

Notable feature: filtering happens before vector search (filterable HNSW), avoiding the trap of filtering after retrieval.

12.6.5 Milvus

Open-source, mature, scales to billions. Multiple index types. Used at very large scale in production.

Strong on: scale, breadth of index types.

Weak on: operational complexity. Best for teams with infrastructure engineering capacity.

12.6.6 pgvector

PostgreSQL extension for vector search. Good when you already use Postgres and don't need extreme scale. Inherits Postgres's transactions, joins, and operational maturity.

Strong on: simplicity (use what you already have), ACID transactions, hybrid SQL + vector queries.

Weak on: pure scale. For 100M+ vectors with low latency, specialized vector databases win.

12.6.7 Elasticsearch and OpenSearch

Added vector search to mature full-text engines. Best when you want hybrid sparse (BM25) + dense (vector) search out of the box, or are already operating Elasticsearch for logging or search.

Strong on: hybrid search, mature operations, large ecosystem.

Weak on: pure vector quality at scale; specialized vector databases win on raw ANN metrics.

12.6.8 Others

Vespa: Yahoo's search engine; very mature; supports vector search natively.

LanceDB: embedded vector database with a developer-friendly API.

MyScale: vector search built on ClickHouse.

Chroma: lightweight, embeddable, great for prototyping.

12.7 Hybrid Search

Pure vector retrieval misses exact keyword matches, rare terms, and structured constraints. Pure keyword search (BM25) misses semantic similarity. The combination is the production default.

Reciprocal Rank Fusion (RRF). Run BM25 and dense retrieval separately. Each returns a ranked list. Fuse the rankings: $\text{score}(d) = \sum 1/(k + \text{rank}(d, \text{list}_i))$. Parameter-free, robust, often the best fusion strategy.

Weighted score fusion. $\text{score}(d) = \alpha \cdot \text{sparse}(d) + (1-\alpha) \cdot \text{dense}(d)$. Requires normalizing the scores; sensitive to the choice of α . RRF is usually a safer default.

Filtering. In addition to vector and keyword similarity, filter by metadata: 'documents tagged with category X', 'created after date Y', 'authored by user Z'. Pre-filtering (apply filter before vector search) is usually faster than post-filtering (vector search then filter) if the vector database supports it.

12.8 Reranking

Retrieval returns candidate documents. Reranking re-scores them with a stronger (and slower) model to refine the order.

Why rerank. The retrieval stage prioritizes speed over precision. A stronger model can examine each candidate in detail and pick the best. Cross-encoders (rather than bi-encoders) compute attention between query and document, capturing finer relevance signals.

Cost. Cross-encoders are 10-100x slower per (query, document) pair than bi-encoders. Only feasible on a small candidate set (typically top 20-100 from retrieval).

Modern implementations. Sentence-transformers cross-encoders. Cohere Rerank API. ColBERT (late interaction, faster than full cross-encoders). LLM-based rerankers (use an LLM to score each candidate).

Pattern. Retrieval returns 100 candidates from millions. Cross-encoder reranks to top 5. The top 5 go to the LLM for generation. The retrieval stage filters; the reranker refines; the generator synthesizes. Each stage is optimized for its role.

12.9 Production Considerations

Beyond the ANN algorithm and the database choice, several engineering concerns matter.

Index updates. Insertion latency, deletion semantics, eventual consistency. HNSW insertion is incremental; IVF can require re-clustering periodically. Plan for this.

Sharding and replication. Sharding splits the index across nodes for horizontal scale. Replication adds read throughput and fault tolerance. Different vector databases handle this differently; check before committing.

Embedding versioning. When you upgrade your embedder, all stored vectors become incompatible with new query vectors. Plan for re-indexing. Some deployments run two indexes during transition.

Cost. Memory dominates for HNSW (vector + graph). Storage dominates for IVF + PQ. Compute is shared. Compute total cost of ownership including infrastructure, not just vector database licenses.

Quality monitoring. Recall@k against a held-out test set. Sample production queries; verify retrieval quality offline. Track over time; alert on regression. A degraded retriever silently degrades the entire downstream system.

12.10 Summary

- Vector databases store and search high-dimensional embeddings. They are the infrastructure for modern semantic search, recommendation, and RAG.
- Approximate nearest neighbor (ANN) algorithms trade small recall loss for orders of magnitude speedup over brute-force search.
- HNSW is the dominant algorithm for moderate scale; IVF + PQ scales to billions with much less memory.
- The vector database ecosystem includes FAISS (library), managed services (Pinecone, Weaviate, Qdrant, Milvus), and database extensions (pgvector, Elasticsearch). Choose based on scale, operational model, and integration needs.
- Hybrid search (BM25 + dense via RRF) almost always beats either alone.
- Reranking with cross-encoders refines retrieval at acceptable cost.
- Production concerns: updates, sharding, embedding versioning, monitoring.

Further Reading

- Malkov and Yashunin, 'Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs' (2016).
- Johnson, Douze, Jégou, 'Billion-scale similarity search with GPUs' (2017). FAISS paper.
- Khattab and Zaharia, 'ColBERT' (2020). late-interaction retrieval.
- Aumüller et al., 'ANN-Benchmarks'. public benchmarking suite for ANN libraries.

Chapter 13

Retrieval-Augmented Generation

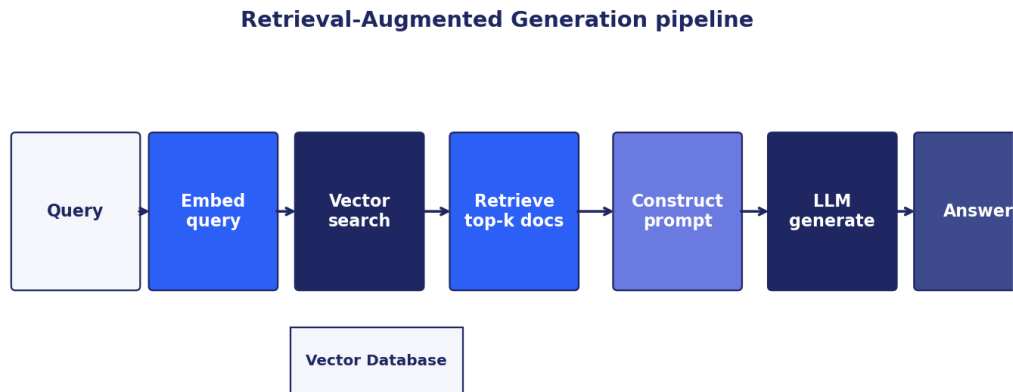


Figure 13.1: The RAG pipeline: query -> retrieve -> generate.

13.1 Why RAG

Pretrained LLMs have two structural limits that show up immediately in production. Their knowledge is frozen at training time; they cannot reference anything that happened after their training cutoff or anything specific to your organization. Their parametric memory is lossy; even for facts they have seen, they sometimes hallucinate plausible-sounding but wrong answers.

Retrieval-augmented generation (RAG) solves both. At inference time, retrieve relevant passages from an external knowledge base. Pass them to the LLM as context. The LLM generates conditioned on the retrieved evidence. The knowledge is up-to-date (you control the knowledge base), private (it can include your proprietary documents), and auditable (the model cites its sources).

RAG has become the default architecture for production LLM applications. Customer support assistants, legal research, medical question answering, internal knowledge tools, agentic systems all run on RAG. This chapter covers the architecture, the engineering choices, and the production patterns.

13.2 The RAG Pipeline

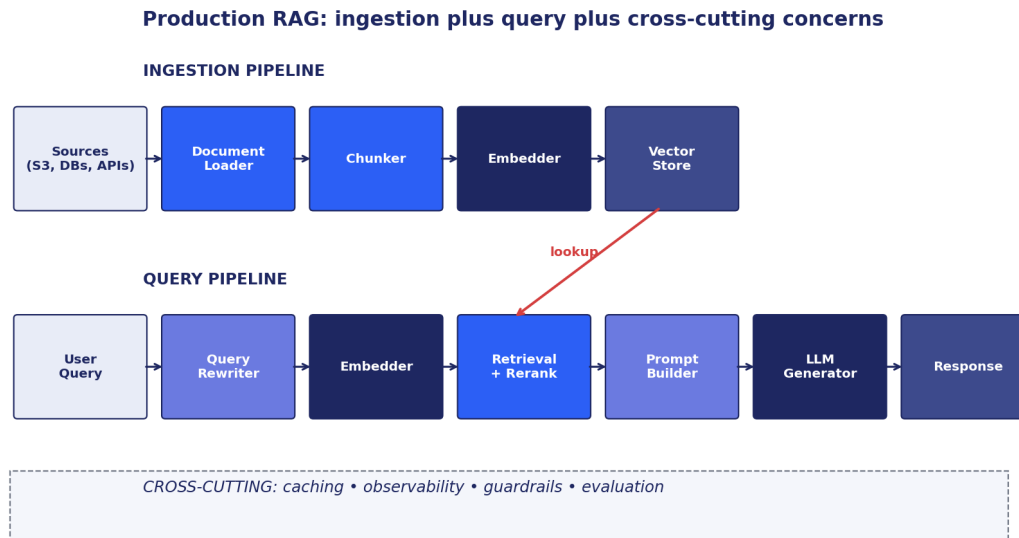


Figure 13.2: Production RAG: ingestion and query pipelines plus cross-cutting concerns.

A complete RAG system has two pipelines: ingestion (build the knowledge base) and query (use the knowledge base to answer questions). Cross-cutting concerns (caching, observability, guardrails, evaluation) span both.

13.2.1 Ingestion Pipeline

Source connection. Pull documents from source systems: S3, GitHub, Notion, Confluence, Google Drive, databases, APIs. Handle authentication, rate limits, incremental updates.

Parsing. Extract clean text from heterogeneous formats: PDF, HTML, DOCX, Markdown, code files, images (with OCR), audio (with ASR), video. Each format has gotchas; layout-preserving PDF parsing is notoriously tricky.

Chunking. Split documents into pieces small enough to fit in the LLM context window and big enough to preserve meaning. Strategies vary; we cover them next section.

Embedding. Encode each chunk with a sentence embedder. Use a domain-tuned embedder if your corpus is specialized.

Vector storage. Index the embeddings in a vector database. Store metadata (source URL, document title, section, timestamp, author) alongside the vectors for filtering and attribution.

Ongoing updates. When source documents change, re-embed and update the index. Incremental updates avoid full rebuilds.

13.2.2 Query Pipeline

Query rewriting. For conversational systems, rewrite the latest user turn into a standalone query using conversation history. 'When was it founded?' becomes 'When was Acme Corp founded?' before retrieval.

Query embedding. Encode the (possibly rewritten) query with the same embedder used for the corpus.

Retrieval. ANN search returns top-k candidates from the vector database. Hybrid (vector + BM25 + filters) almost always beats pure vector.

Reranking. Re-score the top-k candidates with a stronger model (cross-encoder or LLM-based reranker). Refine to top-n where n is much smaller than k.

Prompt construction. Template the retrieved passages, the user query, and the system instructions into a prompt. Include citation markers so the model can reference sources.

Generation. The LLM produces a response conditioned on the prompt. Post-process: format citations, run guardrails, log for evaluation.

13.3 Chunking Strategies

Chunking is the quiet make-or-break of RAG. Too small and the model loses context; too large and the relevance signal dilutes. The right choice depends on your documents and your query distribution.

13.3.1 Fixed-Size Chunking

Split on token count with overlap (typically 10-20%). Cheap, predictable. Ignores natural document boundaries. Often good enough for homogeneous corpora; suboptimal for heterogeneous ones where document structure matters.

Typical chunk sizes: 256-1024 tokens. Smaller = more precise hits but less context per hit. Larger = more context but lower relevance scores. The right size depends on the downstream LLM's context window and the cost per token.

13.3.2 Semantic Chunking

Embed each sentence. Split where adjacent sentence similarity drops below a threshold. Respects natural topic boundaries. More expensive than fixed-size at ingestion but produces more coherent chunks.

13.3.3 Late Chunking

Embed the entire document first, then split the resulting token-level embeddings into chunks. Each chunk's embedding still reflects the full document context. Works only with embedders that support long contexts (Jina v3, bge-m3).

13.3.4 Contextual Chunking

Before embedding each chunk, prepend a short LLM-generated summary of the surrounding context. Anthropic's contextual retrieval showed 35% retrieval improvement on average. The cost is one LLM call per chunk at ingestion.

13.3.5 Hierarchical (Mother-Child) Chunking

Embed at two granularities. Small child chunks (250 tokens) for matching. Large mother chunks (1000 tokens) for context. Match against children but return the parent mother chunk to the LLM. Best of both worlds, modest extra storage.

13.4 Query Transformations

The user's query and the documents may not share surface form even when they share meaning. Query transformations bridge the gap.

HyDE (Hypothetical Document Embeddings). Generate a plausible answer from the query using an LLM. Embed the hypothesis. Retrieve against the result. Closes the query-document distribution gap. Cheap (one extra LLM call per query) and surprisingly effective.

Multi-query expansion. Generate multiple paraphrases of the query. Retrieve for each. Merge results via RRF. Catches retrievals that any single phrasing would miss.

Query decomposition. For multi-hop questions, decompose into sub-queries. Each sub-query retrieves separately. Partial answers feed the next sub-query. Two flavors: predetermined (planner emits all sub-queries up front) or adaptive (each step depends on prior results).

Self-query. The LLM rewrites the user's natural-language query into a structured form: vector query + metadata filter. 'Show me Anthropic papers from 2023' becomes (vector: 'Anthropic papers', filter: year=2023, source='Anthropic'). Combines semantic and structured search.

13.5 Retrieval Funnel

Production retrieval is a funnel. Cheap candidates in, well-scored results out. Multiple stages, each optimized for its role.

Stage 1: parallel sparse and dense retrieval. BM25 returns 100-500 candidates. Dense (vector) retrieval returns another 100-500. RRF fuses them.

Stage 2: optional Matryoshka pruning. If embeddings support it, prune the top 100 with a 64-dimensional truncation before full-dimensional scoring.

Stage 3: ColBERT or other late-interaction scoring. Re-score the top 30 with token-level interaction. Cheaper than a cross-encoder, more expressive than bi-encoders.

Stage 4: cross-encoder reranker. Re-score the top 10-20 with full cross-attention between query and document. The most accurate, most expensive stage. Reserve for the final cut.

Stage 5: LLM-based reranker (optional). For the most demanding tasks, an LLM scores each remaining candidate against the query with a detailed rubric. Slowest, most accurate.

Tune candidate counts per stage on your data. The default funnel works well; the exact numbers depend on corpus and latency budget.

13.6 Multi-Hop and Agentic RAG

Static RAG (retrieve once, generate once) handles single-hop questions well. Multi-hop questions require connecting evidence from multiple sources.

Iterative agentic RAG. The agent retrieves, generates a partial answer, decides what to retrieve next based on what it now knows. Continues until the question is answered or a step limit is reached. Higher cost than static RAG but handles questions that static RAG cannot.

GraphRAG (Microsoft, 2024). Build a knowledge graph from the corpus by extracting entities and relationships with an LLM. Detect communities of related entities. Summarize each community. At query time, retrieve relevant community summaries (for global questions) or entity neighborhoods (for specific questions). Solves multi-hop and structural questions that pure vector retrieval misses.

LightRAG. A simpler graph-based approach: dual-level retrieval (local and global) without explicit community detection. Cheaper to build, competitive on many tasks.

When to use what. Static RAG: simple factual questions, single-source answers. Iterative agentic: multi-hop questions, complex reasoning chains. GraphRAG: questions requiring structural reasoning (who knows whom, what depends on what, what came before). Pick based on the question distribution.

13.7 Derivative Artifacts

Enterprise RAG often benefits from generating derivative content from the source corpus at ingestion time.

Factoids. Use an LLM to extract short factoids from each document. Index the factoids alongside the raw chunks. Retrieval improves on questions that match the factoid phrasing better than the raw text.

QA pairs. Generate plausible (question, answer) pairs from each document. Index the questions. At query time, match user queries against the generated questions; return the corresponding answers and supporting passages.

Summaries. Generate document-level and section-level summaries. Use them for global questions ('what is this document about?'); use the original chunks for specific questions.

Cost. One-time ingestion expense, paid per document. The cost amortizes over many queries. Quality improvements range from negligible (already well-structured technical documents) to dramatic (unstructured chat logs).

13.8 Semantic Caching

RAG is expensive: an embedding call, a vector search, optional reranking, and an LLM generation per query. Semantic caching skips this work when a similar query was answered recently.

Mechanics. Embed each query. Look up similar past queries in a small vector store. If similarity exceeds a threshold, return the cached response. Otherwise process the query normally and cache the result.

Threshold tuning. Too low: serve wrong answers from cache. Too high: cache rarely hits. Tune against a labeled validation set; calibrate by user feedback signals in production.

Typical hit rates. 20-40% on production traffic with repeated query patterns (customer support FAQs, internal help). Lower for free-form queries.

Cost savings. Cache hits cost the embedding call only, often under \$0.0001 each. Cache misses cost the full RAG pipeline. At 30% hit rate, total cost drops by roughly 25%.

13.9 Guardrails and Grounding

RAG reduces but does not eliminate hallucination. Guardrails add layers of defense.

Citation requirement. Have the model cite which retrieved passage supports each claim. Verify the citations match. Reject answers that fail.

NLI verification. Run each claim through a natural language inference model against the retrieved context. Entailment = grounded. Contradiction = block. Neutral = flag for review. DeBERTa-large-mnli is a strong baseline.

Input guardrails. Detect prompt injection in user input and retrieved content. Strip instruction-like patterns. Log suspicious queries.

Output guardrails. PII redaction. Toxicity filters. Content moderation. Structured output schemas to prevent invalid responses.

Defense in depth. No single guardrail catches everything. Layer them. Monitor each layer's trigger rate.

13.10 Evaluating RAG

RAG quality depends on retrieval quality and generation quality. Evaluate both separately and end to end.

Retrieval evaluation. Build a held-out test set of (query, relevant documents) pairs. Measure Recall@k, MRR, NDCG. The most important metric for RAG: if relevant documents are not retrieved, the LLM cannot answer.

Generation evaluation. Faithfulness: is the answer grounded in the retrieved context? Answer relevance: does the answer address the question? Context precision: is the retrieved context relevant? Context recall: does the retrieved context cover what is needed? Ragas is the standard framework that automates these.

End-to-end evaluation. Beyond Ragas metrics, evaluate user satisfaction signals (thumbs, regenerate rate, conversation length). The metrics matter only insofar as they correlate with what users care about.

Robustness evaluation. RGB benchmark (Chen et al., 2023) tests four axes: noise (do irrelevant passages confuse the model?), negative (does the model refuse when no answer is supported?), integration (does the model combine facts across passages?), counterfactual (does the model trust evidence over its prior?). A real RAG should hold up against all four.

Continuous evaluation. Sample 1-5% of production traffic. Score offline. Trend over time. Alert on regression. New deployments must pass these checks before promotion.

13.11 Summary

- RAG combines retrieval with generation, grounding LLM responses in external knowledge. It is the dominant architecture for production LLM applications.
- The ingestion pipeline (parse, chunk, embed, store) builds the knowledge base. The query pipeline (rewrite, embed, retrieve, rerank, generate) uses it.
- Chunking strategy matters: fixed-size, semantic, late, contextual, and hierarchical chunking trade cost for quality.
- Query transformations (HyDE, multi-query, decomposition) handle hard queries.
- The retrieval funnel (sparse + dense + Matryoshka + ColBERT + cross-encoder) optimizes the cost-quality tradeoff at each stage.
- Multi-hop questions need iterative agentic RAG or graph-based approaches (GraphRAG, LightRAG).
- Semantic caching cuts cost; guardrails reduce hallucination; Ragas evaluates end to end.

Further Reading

- Lewis et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks' (2020). the original RAG paper.
- Karpukhin et al., 'Dense Passage Retrieval for Open-Domain Question Answering' (2020). DPR.
- Edge et al., 'From Local to Global: A Graph RAG Approach to Query-Focused Summarization' (2024). Microsoft GraphRAG.
- Gao et al., 'Precise Zero-Shot Dense Retrieval without Relevance Labels' (2022). HyDE.
- Es et al., 'RAGAS: Automated Evaluation of Retrieval Augmented Generation' (2023).

Part VII

Production: LLMOps, Evaluation, Ethics

This part contains chapter(s) 14, 15, 16 covering production: llmops, evaluation, ethics.

Chapter 14

LLMOps and Production System Design

14.1 From Model to System

A working model is not a working product. Bridging that gap is the discipline of LLMOps. It includes architecture (how the components fit together), serving infrastructure (how requests get from users to GPUs and back), monitoring (how you know the system is healthy), evaluation (how you know the system is actually good), and continuous improvement (how the system gets better over time).

Traditional MLOps assumes a model with fixed inputs and outputs that produces clear metrics. LLMOps inherits those assumptions but adds: prompts that change frequently, retrieval components with their own quality, tool calls to external systems, conversational state, alignment-related risks, and an evaluation problem that is fundamentally harder because outputs are open-ended.

This chapter walks the production landscape. It is opinionated about what matters most: latency, cost, observability, evaluation, and safety. The specific tools change quickly; the principles are durable.

14.2 Serving Infrastructure

Modern LLM serving is dominated by a few patterns and tools.

14.2.1 vLLM and Continuous Batching

vLLM (Kwon et al., 2023) introduced PagedAttention, a technique that manages the KV-cache like virtual memory. Combined with continuous batching (dynamically merging requests of different lengths into batches), vLLM achieves 5-24x throughput over naive batching.

The default serving stack for self-hosted LLMs. Compatible with most Hugging Face models. Supports tensor parallelism, quantization (AWQ, GPTQ, FP8), speculative decoding, structured outputs.

Alternatives. Hugging Face TGI (Text Generation Inference). TensorRT-LLM from NVIDIA. SGLang. Ollama for local development. Each has tradeoffs; vLLM is the broadest fit.

14.2.2 Tensor and Pipeline Parallelism

A 70B model does not fit on a single GPU. Tensor parallelism shards each matrix multiplication across devices (split a 8192-wide layer into 8 pieces of 1024 across 8 GPUs). Pipeline parallelism splits the model by layers, with micro-batches flowing through.

Modern frontier serving uses both: tensor parallelism within a node (high inter-GPU bandwidth via NVLink), pipeline parallelism across nodes. Mixed-precision (BF16 or FP8) reduces memory and compute by 2-4x.

Inference parallelism is different from training parallelism. Inference is latency-sensitive; training is throughput-sensitive. Choose configurations accordingly.

14.2.3 Quantization for Serving

Quantizing the served model reduces memory and increases throughput. INT8 quantization typically loses less than 1 point on benchmarks. INT4 (AWQ, GPTQ) loses 1-3 points but offers 4x compression and large speedups. FP8 is the modern compromise: near-fp16 quality with substantial efficiency gains.

Tradeoffs. Quantization adds engineering complexity (calibration, post-training vs quantization-aware training). Quality may drop on long contexts or specific tasks. Test on your eval set, not just published benchmarks.

14.2.4 Speculative Decoding

A small draft model proposes the next k tokens. The main model verifies in a single forward pass. Accepted tokens become output; rejected tokens fall back to the main model.

Speedup. Up to 2-3x for tasks where the draft model's predictions are mostly correct. Code generation and structured outputs benefit most.

Cost. Run two models. The draft model overhead is small but real. Quality is unchanged (verification by the main model ensures the output distribution matches main-model-only).

14.3 Latency and Cost Engineering

User-perceived latency in an LLM application has several components.

Time to first token (TTFT). How long from user submission to the first streamed token. Dominates the user's impression of speed in chat interfaces. Reduced by: shorter prompts (RAG retrieval, prompt caching), smaller models for the first response, speculative decoding.

Inter-token latency. How fast tokens stream after the first. Determined by model size, batching, hardware. Continuous batching helps under load.

Total response latency. TTFT plus total tokens times inter-token latency. Some applications care more about TTFT (chat) and others about total latency (batch jobs).

Cost components. Input tokens: dominant for RAG (long context). Output tokens: dominant for generation-heavy tasks. Tool calls: external service costs plus latency. Inference compute: GPU hours.

Cost optimization patterns. Prompt caching: API providers cache shared prefixes; reuses dramatically reduce cost. Smaller models for simple tasks; route based on query complexity. Aggressive quantization. Reducing retrieval candidates so generation prompts are shorter.

Caching strategy. Exact-match cache for repeated queries. Semantic cache for similar queries. Prompt cache for shared prefixes. Each helps different traffic patterns; combine them.

14.4 Multi-Tenant Serving

Many production LLM applications serve multiple customers with different needs from shared infrastructure. Multi-tenant serving has its own design considerations.

Adapter swapping. Serve a single base model with LoRA adapters per tenant. Load adapters dynamically. Each tenant gets a customized model without provisioning a separate base. Implementations: vLLM with multi-LoRA, S-LoRA.

Routing. Different queries should go to different models. Cheap small model for simple queries. Larger model for complex queries. Reasoning model for math and code. A learned or rule-based router picks. The router must be fast (sub-100ms) and correct often enough to justify the complexity.

Isolation. Tenants should not see each other's data. Per-tenant rate limits prevent one tenant from monopolizing GPUs. Audit logs per tenant for compliance.

Cost allocation. Track tokens, latency, and model usage per tenant. Charge appropriately. Surface usage to customers.

14.5 Observability

You cannot improve what you cannot measure. LLM observability is more complex than traditional service observability because the outputs are open-ended.

Distributed tracing. Each user request is a trace. Spans for retrieval, reranking, prompt construction, LLM generation, post-processing, tool calls. OpenTelemetry is the standard. Hosted platforms: Helicone, LangFuse, Arize, Langsmith. Custom Grafana dashboards work too.

Per-request metrics. Latency (P50, P95, P99). Token counts (input, output). Cost. Tool calls. Cache hits.

Aggregate metrics. Throughput. Error rate. Queue depth. GPU utilization. Cost per active user. These are the metrics a platform team watches.

Quality metrics. User feedback (thumbs, regenerate rate). LLM-as-judge scores on sampled traffic. Refusal rate. Average response length. These drift over time; trend them.

Safety metrics. Jailbreak attempts. PII exposure events. Policy violations. Audit logs.

Alerting. Define SLOs (service level objectives) for latency, error rate, and quality. Alert on regression. Be careful about alert fatigue; the right alerts page on real problems and stay quiet otherwise.

14.6 Tool Use and the Model Context Protocol

Modern LLMs increasingly call external tools: search, code execution, databases, internal APIs. Tool use extends the model's reach but adds engineering complexity.

Tool definition. Each tool has a name, description, input schema, output schema. The LLM reads these and decides which to call. Names and descriptions matter more than people realize; ambiguity here produces wrong tool calls.

The Model Context Protocol (MCP, Anthropic, 2024). A clean abstraction for tools. A FastMCP server exposes tools, resources, and prompts. An agent client connects, negotiates capabilities at session start, and invokes tools via JSON-RPC. Separates tool definition from agent runtime, making tools reusable across agents.

Tool design. Names should be verbs matching user intent. Descriptions should be unambiguous. Argument names are also the prompt; 'date' is bad, 'start_date_iso8601' is good. Errors should be structured (tool_unavailable, invalid_input, rate_limited) so the agent can reason about retry.

Idempotency. Write tools must accept idempotency keys. Retries with the same key are no-ops. Otherwise transient failures become double-charge incidents.

Sandboxing. Code execution tools must run in isolated environments. File system access must be scoped. Network access must be limited. The agent should be allowed to make mistakes without destroying the user's data.

14.7 Agentic Systems

An agent is a program that perceives, plans, acts via tools, and observes results in a loop. Modern AI applications increasingly are agents rather than single LLM calls.

Agent design patterns. Router: classify input, dispatch to a specialized agent or pipeline. Orchestrator-workers: a planner agent delegates sub-tasks to worker agents. Evaluator-optimizer: a generator agent produces, a critic agent evaluates, the generator revises. Autonomy ladder: progressively grant the agent more freedom as it proves reliable.

Single agent vs multi-agent. Single agent with many tools is the default; easier to debug, simpler to deploy. Multi-agent makes sense when responsibilities are genuinely separable and agents need different context windows or capabilities.

State management. Conversational state for multi-turn agents. Persistent memory for long-running agents. Idempotent state transitions so the agent can recover from interruptions.

Evaluation. Agentic systems are harder to evaluate than single LLM calls. Define success criteria for tasks the agent should complete. Track completion rate, intervention rate (human had to step in), and cost per task. End-to-end evaluation on representative tasks is essential.

Failure modes. Hallucinated tools (calling functions that don't exist). Infinite loops (no termination condition). Prompt injection through tool outputs. Mistaken tool choices. Each requires engineering attention.

14.8 MLOps for the LLM Era

Classical MLOps focuses on training, model registries, A/B testing, monitoring. The LLM era extends these.

Prompt versioning. Treat prompts as code. Store in git. Review changes. Tie deployed versions to evaluation results. Roll back on regression.

Evaluation pipelines. Automated evaluation on every prompt change, model version, or RAG corpus update. Pre-merge checks: does this change improve quality? Does it regress anything?

Canary deployments. New prompt, model, or pipeline version goes to a small fraction of traffic. Compare against control. Roll out gradually if metrics look good; roll back if not.

Continuous improvement. Sample production traffic. Identify failure cases. Add them to evaluation set. Iterate on prompts and models. Production data is the most valuable input to improvement.

Cost monitoring. LLM costs scale with usage. Track per-feature, per-tenant, per-user. Alert on spikes. Optimize the dominant cost drivers.

14.9 Building a ChatGPT-Like System

A worked example: how would you build a production chat assistant from scratch?

Frontend. Web app with streaming responses, conversation history, file upload. Mobile apps. APIs for developers.

Backend. Stateless API layer behind a load balancer. Session management with persistent conversation store (Postgres or a NoSQL store). Per-user rate limiting.

Model serving. vLLM behind a load balancer. Continuous batching. KV-cache management. Speculative decoding for latency. Routing for query complexity (small model for simple queries, large for complex).

Models. A frontier base LLM with SFT and RLHF for chat. Optionally specialized variants (code, reasoning, multimodal).

Tool use. Function calling for web search, code execution, image generation, calendar, email. Tool registry. Safe execution sandboxes.

RAG. Optional, for memory and document upload features. Per-user knowledge bases for personalization.

Safety. Input filters (prompt injection, harmful intent), output filters (toxicity, PII), refusal calibration, jailbreak detection. Continuous red teaming.

Operations. Observability (traces, metrics, logs). Feature flags. A/B testing. Incident response.

Compliance: SOC 2, GDPR, regional data residency.

Scale. Tens of millions of users requires global multi-region deployment, specialized hardware, careful capacity planning.

14.10 Summary

- Production LLM systems are more than models: they are serving infrastructure, prompts, retrievers, tools, guardrails, and monitoring.
- vLLM with continuous batching is the modern self-hosted serving default. Tensor and pipeline parallelism handle large models. Quantization (INT8, AWQ, FP8) reduces cost.
- Latency components: time to first token, inter-token latency, total response time. Cost components: input tokens, output tokens, tool calls, inference compute.
- Multi-tenant serving uses adapter swapping (LoRA per tenant) and routing to serve diverse needs from shared infrastructure.
- Observability requires distributed tracing, per-request metrics, quality monitoring, and safety tracking.
- MCP standardizes tool exposure. Agents extend LLMs with tools, planning, and persistent state.
- MLOps for the LLM era includes prompt versioning, automated evaluation pipelines, canary deployments, and continuous improvement from production data.

Further Reading

- Kwon et al., 'Efficient Memory Management for Large Language Model Serving with PagedAttention' (2023). vLLM.
- Sheng et al., 'S-LoRA: Serving Thousands of Concurrent LoRA Adapters' (2023).
- Leviathan, Kalman, Matias, 'Fast Inference from Transformers via Speculative Decoding' (2022).
- Anthropic, 'Building Effective Agents' engineering blog (2024).

Chapter 15

Evaluation Methodologies

15.1 Why Evaluation Is the Hard Part

It is easy to build something that looks impressive. It is hard to know whether what you built is actually good. The gap between 'works on a demo' and 'works in production' is mostly evaluation. Teams that invest in evaluation ship reliably. Teams that don't ship surprises.

LLM evaluation is harder than classical ML evaluation. Classical ML has clear right answers (accuracy, F1, RMSE). LLM outputs are open-ended; there is no single correct response to 'summarize this document'. Reference-based metrics (BLEU, ROUGE) capture only surface overlap. Human evaluation is the gold standard but expensive. LLM-as-judge is cheap and scalable but has well-documented biases.

This chapter walks the evaluation toolkit: classical metrics, modern LLM-specific approaches, RAG-specific evaluation, methodology for building evaluation suites that actually inform decisions.

15.2 Classification Metrics

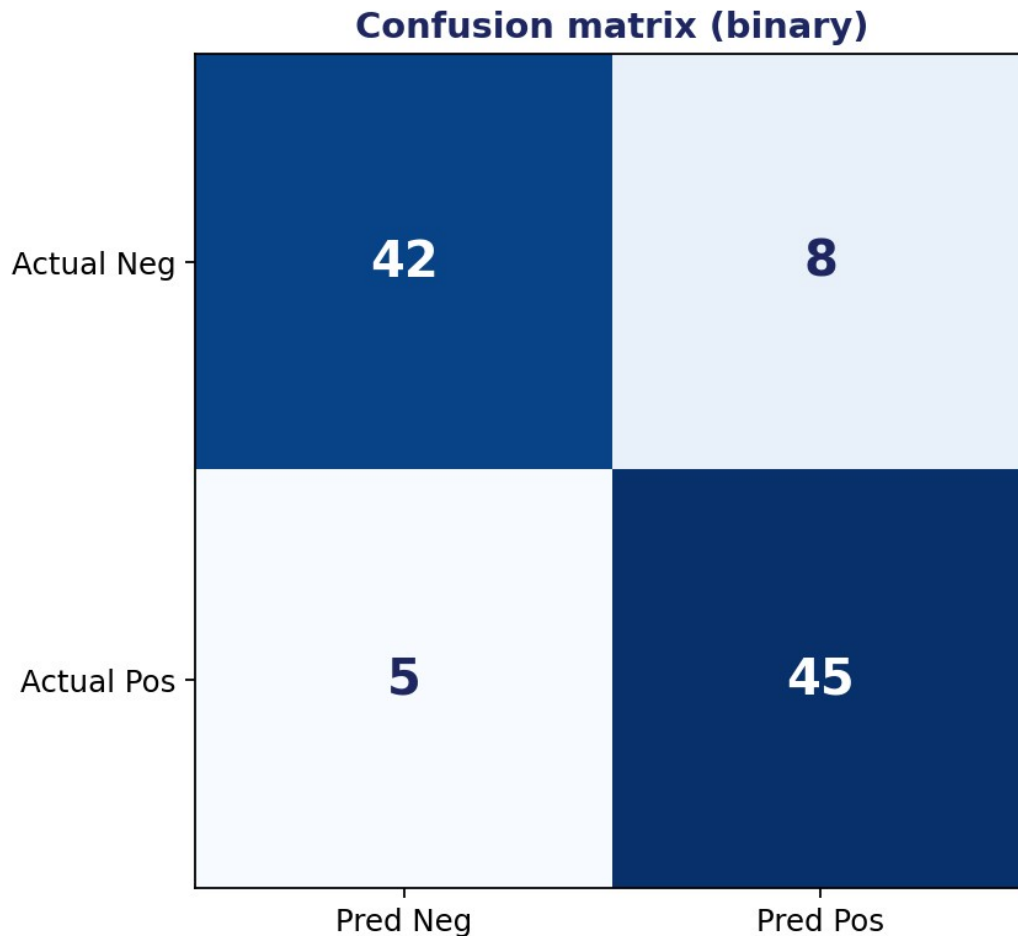


Figure 15.1: Confusion matrix for binary classification.

For classification tasks, evaluation is well-understood.

Accuracy. Fraction of predictions that are correct. Simple but misleading on imbalanced datasets. A 99% accurate model on a 99%-negative dataset may be predicting 'negative' always.

Precision. True positives divided by predicted positives. Of the items the model said were positive, how many were? Important when false positives are costly (spam filter blocking real email).

Recall. True positives divided by actual positives. Of the items that are positive, how many did the model catch? Important when false negatives are costly (cancer screening).

F1 score. Harmonic mean of precision and recall. Balanced single metric. F-beta generalizes: F2 weights recall more, F0.5 weights precision more.

ROC curve and AUC. Plot true positive rate against false positive rate at all thresholds. Area under the curve measures ranking quality across all thresholds. Use when you have a probability output and care about ranking, not a single threshold.

Precision-Recall curve. For imbalanced datasets, the PR curve and its area are more informative than ROC. A model can have high ROC-AUC but poor precision at any useful threshold.

15.3 Generation Metrics

For text generation, reference-based metrics measure overlap with one or more reference outputs.

BLEU (Bilingual Evaluation Understudy). N-gram overlap, with a brevity penalty for short outputs. Designed for machine translation. Strengths: simple, fast, well-understood. Weaknesses: rewards surface overlap; misses paraphrase; correlates only weakly with human judgment on creative tasks.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation). N-gram overlap, recall-oriented. ROUGE-1 (unigrams), ROUGE-2 (bigrams), ROUGE-L (longest common subsequence). The summarization standard.

METEOR. Aligns words by exact match, stem, synonym, and paraphrase. Higher human-judgment correlation than BLEU. Slower; language-specific resources required.

chrF. Character-level F-score. Better than BLEU for morphologically rich languages.

BERTScore. Use a pretrained encoder to embed candidate and reference. Compute token-level cosine similarity, then average. Captures semantic similarity that surface metrics miss. Higher correlation with human judgment than BLEU/ROUGE.

COMET, BLEURT. Learned metrics trained on human judgment data. State of the art for translation evaluation. Slow but accurate.

When to use reference-based metrics. Translation, summarization with known reference outputs. Quick iteration during development. Use as one of several signals, not the only one.

15.4 Perplexity

Perplexity = $\exp(\text{per-token cross-entropy})$ on a held-out corpus. Lower is better. The fundamental intrinsic evaluation for language models.

What perplexity measures. How well the model predicts the next token. A perplexity of 10 means the model is, on average, as uncertain as if choosing uniformly from 10 options at each step.

What perplexity misses. Factuality (a fluent lie has low perplexity). Reasoning (predicting the next token of a chain-of-thought is different from being able to reason). Instruction following. Calibration. Safety. All the things that matter for actual LLM quality.

When to use. Sanity check during pretraining. Compare models trained with the same tokenizer on the same evaluation data. Detect regression in language modeling capability after fine-tuning.

When not to rely on it. Comparing differently-tokenized models (a smaller vocab artificially increases perplexity). Judging downstream quality below a threshold (a model with PPL=5 and PPL=4 are both fluent; the difference does not predict chat quality).

15.5 LLM Benchmarks

A standardized way to compare LLMs. Many benchmarks; pick those that match your use case.

MMLU (Massive Multitask Language Understanding). 57 subjects, multiple-choice. Tests broad knowledge. Most LLM papers report MMLU. Strong models hit 80-90% accuracy in 2025.

HellaSwag. Commonsense reasoning via sentence completion. Tests whether the model picks the natural continuation.

GSM8K. Grade school math word problems. Tests arithmetic reasoning. Strong models hit 90%+ with chain-of-thought.

MATH. Competition mathematics. Much harder than GSM8K. The proving ground for reasoning models (o1, R1).

HumanEval, MBPP. Code generation. Pass@k: fraction of problems solved within k samples. Standard for code LLMs.

BIG-Bench, BBH (BIG-Bench Hard). Diverse hard tasks. Cover capabilities MMLU misses.

TruthfulQA. Tests for known false beliefs (urban legends, misconceptions). Distinguishes models that parrot common falsehoods from those that know the truth.

MT-Bench. Multi-turn dialogue evaluated by GPT-4. Closer to actual chat use cases than knowledge benchmarks.

Chatbot Arena. Crowd-sourced pairwise comparison. The most ecologically valid benchmark for chat quality.

Domain-specific. MedQA (medical), FinanceBench (financial), LegalBench (legal). Use these when relevant to your use case; general benchmarks may not predict domain performance.

Caveats. Benchmarks get gamed. Top labs train on the test set inadvertently (data contamination). Look at multiple benchmarks and recent results rather than trusting any single number.

15.6 LLM-as-a-Judge

Have a separate strong LLM (typically GPT-4 or Claude) score outputs against a rubric or pairwise compare candidates. Cheap and scalable; replaces (or supplements) human evaluation.

Setup. Define a rubric: helpfulness, factuality, relevance, format compliance, etc. Prompt the judge with the question, the candidate answer(s), and the rubric. Get scores or pairwise verdicts. Aggregate across many examples.

Biases to control. Position bias: when two candidates are presented, the judge often prefers the first. Randomize positions; or always score both orderings and average. Verbosity bias: longer responses are

often preferred even when shorter ones are better. Normalize by length or instruct the judge to ignore length. Self-preference bias: a judge tends to prefer outputs from its own model family. Use a different judge model than the model under evaluation.

Reliability. LLM-as-judge correlates well with human judgment on simple rubrics. Correlation drops on subjective tasks (creativity, humor) and specialized domains (medical, legal). Sanity-check with human evaluation on a sample.

Cost. A judge call per evaluation. Cheaper than human evaluation, more expensive than automated metrics. Use on samples (1% of production traffic, 100 examples in a CI pipeline) rather than every example.

15.7 RAG Evaluation: Ragas

RAG-specific metrics from the Ragas framework (Es et al., 2023). Four core metrics, each addressing a different failure mode.

Faithfulness. Are all claims in the answer supported by the retrieved context? Catches hallucination. Decompose the answer into claims; for each, check whether the context supports it (using an LLM judge or NLI).

Answer relevance. Does the answer actually address the question? Catches off-topic responses. Generate candidate questions from the answer; compare them to the original question.

Context precision. Are the retrieved passages relevant? Catches retrieval that returns junk. For each retrieved passage, score relevance to the question.

Context recall. Does the retrieved context cover what is needed to answer? Catches retrieval that misses important information. Compare the retrieved context against a ground-truth answer.

All four together. Faithfulness + answer relevance measure generation quality. Context precision + context recall measure retrieval quality. The four-axis picture is more informative than any single metric.

Limitations. The metrics are themselves computed by LLMs. Same biases apply. Use as automated screening; sample human review for high-stakes decisions.

15.8 Hallucination Evaluation

Hallucination has its own evaluation toolkit beyond Ragas.

TruthfulQA. Tests known false beliefs. Distinguishes models that parrot falsehoods from those that know better.

FactScore. Decompose generated text into atomic facts. Verify each fact against a knowledge source. Report the fraction supported. Strong correlation with human judgment.

FEVER. Benchmark for claim verification. Three-way classification: supported, refuted, not enough info. Adapted for LLM output verification.

QAGS. Generate questions from the candidate text; answer them using the source. Mismatch indicates hallucination.

SelfCheckGPT. Sample multiple responses; compare for consistency. Inconsistent answers signal hallucination. No external knowledge base required.

Citation accuracy. For RAG with citations, verify that cited sources actually contain the claim. Common metric in production systems.

Confidence calibration. The model's predicted probability or verbalized confidence should correlate with correctness. Calibrate via temperature scaling or training-time techniques.

15.9 Bias and Safety Evaluation

Models trained on web text reflect web biases. Production systems must evaluate for harm.

BBQ (Bias Benchmark for QA). Probes stereotypes across nine demographic categories. Measures whether models reinforce stereotypes when context is ambiguous.

StereoSet, CrowS-Pairs. Measure stereotype bias in language models via contrasting sentences.

ToxiGen. Tests model responses to potentially harmful inputs.

AdvBench. Jailbreak prompts. Measures refusal rate on adversarial inputs.

WMDP (Weapons of Mass Destruction Proxy). Tests dangerous capability (chem/bio/cyber) knowledge that frontier models should not provide.

HarmBench. Comprehensive safety evaluation across harm categories.

Process. Run safety evaluation before deployment. Track over time. Add discovered failures to evaluation sets. Red team continuously; new attack patterns emerge regularly.

15.10 Building an Evaluation Suite

Public benchmarks are useful for comparing models. They are insufficient for evaluating your specific application. Build a custom evaluation suite.

Step 1: Define what 'good' means. What user outcomes does the system produce? What failure modes matter most? Concrete examples beat abstract rubrics.

Step 2: Curate test cases. Sample real user queries (or write representative ones). Cover: happy path, edge cases, adversarial cases, out-of-distribution cases. Aim for 100-1000 cases; quality over quantity.

Step 3: Establish ground truth. For each case, write the ideal response, rubric scores, or pass/fail criteria. Domain experts where relevant. Document assumptions and disagreements.

Step 4: Build automated evaluation. Run the system; score against ground truth using appropriate metrics (exact match, F1, LLM-as-judge, Ragas). Aggregate. Report.

Step 5: Continuous expansion. Every production failure becomes a new test case. Every new feature gets test cases before launch. The evaluation suite grows with the system.

Step 6: Run on every change. Prompt changes, model updates, RAG corpus changes, infrastructure changes all run through the evaluation. CI/CD for evaluation.

Pitfalls. Evaluation drift: ground truth ages out. Periodic refresh. Cherry-picking: report aggregate metrics with confidence intervals, not hand-picked examples. Overfitting to evaluation: hold out a small set you only run quarterly.

15.11 A/B Testing in Production

Offline evaluation is necessary but insufficient. Real users behave differently than test sets. A/B testing in production catches what offline evaluation misses.

Setup. Hypothesis: state precisely what you expect ('Model B reduces support resolution time by 5%'). Pre-register. Sample size: power analysis to determine N. Stable user assignment so the same user sees the same arm.

Metrics. Primary: the hypothesis metric. Secondary: leading indicators (satisfaction, retention). Guardrail: safety violations, latency, cost.

Duration. Long enough to capture weekly cycles. Typically 2-4 weeks.

Analysis. Standard statistical tests. Bonferroni or similar correction for multiple metrics. Sanity-check distribution match between arms.

Decision. Pre-defined launch criteria. If treatment wins primary by X% with $p < 0.05$ and no guardrail violations, launch.

Caveats. Novelty effects: users may prefer new features simply because they are new. Run long enough for the novelty to wear off. Interference: if treatment and control share resources (model serving capacity), results can be confounded.

15.12 Summary

- LLM evaluation is the hard part of building LLM systems. Teams that invest in evaluation ship reliably.
- Classical metrics (accuracy, F1, ROC-AUC) work for classification. BLEU and ROUGE work for translation and summarization. Perplexity is an intrinsic language modeling metric.

- LLM-as-a-judge enables scalable evaluation of open-ended outputs. Control for position, verbosity, and self-preference biases.
- Public benchmarks (MMLU, HumanEval, MT-Bench) compare models but don't predict performance on your application.
- Ragas evaluates RAG along four axes: faithfulness, answer relevance, context precision, context recall.
- Hallucination evaluation (TruthfulQA, FactScore, SelfCheckGPT) and safety evaluation (BBQ, ToxiGen, AdvBench) catch failure modes specific to LLMs.
- Build a custom evaluation suite. Grow it with discovered failures. Run it on every change. A/B test in production for the final word.

Further Reading

- Zheng et al., 'Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena' (2023).
- Es et al., 'RAGAS: Automated Evaluation of Retrieval Augmented Generation' (2023).
- Liang et al., 'HELM: Holistic Evaluation of Language Models' (2023).
- Chen et al., 'Benchmarking Large Language Models in Retrieval-Augmented Generation' (2023). RGB benchmark.

Chapter 16

Ethics, Safety, and the Path Ahead

16.1 The Responsibility of AI Engineers

Every technology has tradeoffs. AI has unusually large ones. The same system that helps a doctor diagnose a rare disease can spread medical misinformation at scale. The same model that summarizes meetings can generate convincing deepfakes. The same image generator that powers a creative tool can fabricate non-consensual content.

The engineers who build these systems carry the responsibility for thinking about how they will be used and misused. This is not a philosophical aside; it is part of the engineering. A system that scales harm is not a successful system, even if its accuracy is high.

This chapter walks through the major categories of ethical concern and the practical patterns that mitigate them. We do not pretend to completeness; the questions are genuinely hard and the answers evolve. But the categories and patterns here are the table stakes for professional AI engineering.

16.2 Truthfulness and Hallucination

Generative models produce confident-sounding text regardless of whether they are right. Users often cannot tell the difference. Hallucinated medical advice, legal interpretation, financial recommendation, or historical fact has real consequences.

Mitigation patterns. Ground responses in retrieved evidence (RAG). Require citations. Verify citations. Train models to refuse when uncertain rather than guess. Surface confidence to users. Disclose AI involvement so users calibrate appropriately.

Domain-specific care. Medical, legal, financial, and scientific applications need higher standards. Expert review of model outputs may be required. Disclaimers help but do not absolve. A model that consistently produces wrong medical advice is harmful even if it says 'consult a doctor' at the end.

Detection. Sample production traffic. Verify against trusted sources. Trend hallucination rates. Alert on drift.

16.3 Bias and Fairness

Models trained on web text reflect the biases of the web: underrepresentation of minority perspectives, stereotypes baked into common phrasings, dominant cultural assumptions about who does what. These biases propagate to model outputs and to systems that depend on them.

Examples. Hiring systems that rate male candidates higher because the training data reflects historical hiring patterns. Recommendation systems that reinforce filter bubbles. Image generators that depict doctors as men and nurses as women when no gender was specified in the prompt. Translation systems that switch genders when no information is available.

Detection. Demographic parity in outcomes. Counterfactual evaluation (change a demographic attribute in the input; does the output change?). Stereotype benchmarks (BBQ, StereoSet, CrowS-Pairs). Disparate impact analysis.

Mitigation. Curate training data for representation. Apply fairness constraints during training (adversarial debiasing, regularization). Audit deployed systems regularly. Diverse evaluation teams. Counterfactual augmentation to balance representation.

Difficult tradeoffs. Fairness criteria sometimes conflict (demographic parity vs equal opportunity vs calibration). The right choice is context-dependent and may require domain expertise and stakeholder involvement.

16.4 Privacy

AI systems handle large amounts of data, much of it personal. Privacy concerns span training data, inference inputs, and the data the model may inadvertently expose.

Training data privacy. Models can memorize and reproduce training data. Documented cases: GPT-2 reproducing copyrighted text, GitHub Copilot emitting verbatim code with original author comments. Mitigations: deduplicate aggressively, use differential privacy when sensitive data is in training, audit for memorization with attack-based probes.

Inference-time privacy. User inputs may contain PII. Storing and logging them creates exposure. Minimize retention. Encrypt at rest and in transit. PII detection and redaction in logs. Per-user access control.

Regulatory compliance. GDPR (Europe), CCPA (California), HIPAA (US healthcare), GLBA (US financial). Different regions, different rules. Build for the strictest applicable jurisdiction; document data flows and consents.

Model output privacy. Models can leak training data through targeted prompts. Membership inference attacks reveal whether a specific record was in training. For sensitive applications, evaluate against these attack vectors.

16.5 Copyright and Data Provenance

The data used to train modern foundation models is the subject of active litigation. Models trained on copyrighted text and images may reproduce training content; the legal status of training on copyrighted data is still being decided in many jurisdictions.

Provenance tracking. Document data sources. Maintain records of licenses. Avoid training on data with restrictive licenses unless you have explicit permission.

Filter for copyrighted reproduction. Detect when outputs reproduce training data verbatim. Refuse to generate such outputs. Provide attribution where possible.

Opt-out mechanisms. Some data owners want their content excluded from training. Respect robots.txt-style opt-outs (e.g., OpenAI's GPTBot, Anthropic's ClaudeBot, ai.txt proposals).

Compensation models. Pay-for-training arrangements are emerging. Adobe Firefly trained only on licensed Adobe Stock content. Getty's Generative AI tool similarly. The economics of compensating data contributors are still being worked out.

16.6 Misuse: Deepfakes, Disinformation, Fraud

Generative AI lowers the cost of producing convincing fake content. The consequences range from harassment to fraud to election interference.

Deepfakes. Synthetic images, audio, and video that depict real people saying or doing things they did not. Targets: politicians, celebrities, private individuals (often as a form of harassment). Mitigation: watermarking and detection tools, platform policies, legal frameworks. None of these are complete; the arms race continues.

Disinformation. AI-generated text scales the production of false narratives. Distinguishing AI-generated text from human-written text is increasingly hard. Mitigations: watermarking generated content, provenance tracking (C2PA), media literacy education, platform interventions.

Fraud. Voice cloning for impersonation scams. Text generation for spear-phishing. Image generation for identity document fraud. Each has specific countermeasures (multi-factor authentication, transaction limits, anomaly detection) but the threat surface is widening.

What developers can do. Build detection tools and watermarking into products. Refuse to help with clearly malicious requests. Support responsible disclosure of vulnerabilities. Engage with policy discussions.

16.7 Concentration of Power and Capability

Frontier AI is expensive. Training a frontier model costs hundreds of millions of dollars. Only a few companies can afford this. The benefits and risks of frontier AI are concentrated in those companies.

Implications. Decisions about model behavior, safety, access, and alignment are made by a small number of organizations. Misalignment between their incentives and broader public interest is structural.

Open-weights mitigation. Open releases of frontier-comparable models (LLaMA, Mistral, Qwen) democratize access. They also create new risks: removing safety training, fine-tuning for malicious purposes, easier proliferation of dangerous capabilities.

Governance approaches. Voluntary commitments (frontier model labs agreeing to red-team and disclose). Government oversight (EU AI Act, executive orders in the US). International coordination (still nascent). Industry self-regulation (Partnership on AI, Frontier Model Forum).

The right approach is contested. Engineers should engage thoughtfully with these questions rather than treating them as someone else's problem.

16.8 Environmental Impact

Training and serving large AI models consumes substantial energy. A frontier training run can use 10+ gigawatt-hours; serving consumes more in aggregate, summed across all queries.

Carbon footprint. Depends on data center power sources. Cloud providers differ; some buy renewable energy at scale, others use grid power with high emissions.

Efficiency improvements. Mixed precision (FP8, BF16) reduces compute and memory. Quantization (INT8, INT4) reduces inference cost. Sparse architectures (MoE) reduce active compute per token. Smaller well-tuned models often suffice; default to the smallest model that meets quality.

Reporting. Some labs publish energy and carbon estimates for training. More transparency would help. Engineers can request and weigh this information in vendor decisions.

Demand-side considerations. Just because we can use AI for a task does not mean we should. For many tasks, simpler statistical methods or no automation at all is more appropriate. Right-size the technology.

16.9 Safety: Alignment and Catastrophic Risk

Alignment research focuses on making AI systems behave the way we want, even as they become more capable. The motivating concern: a sufficiently capable system pursuing the wrong objective could cause harm at scale.

Near-term alignment. The systems we deploy today have measurable failure modes: hallucination, jailbreaks, reward hacking, sycophancy. These are the alignment problems engineers face daily. RLHF, DPO, constitutional AI, and the broader alignment pipeline address them.

Long-term alignment. As capabilities grow, new alignment problems emerge: deceptive alignment (the model behaves well during training but pursues different goals at deployment), mesa-optimization (the model develops internal optimizers with different objectives), distribution shift in high-stakes settings. These are active research areas.

Catastrophic risk. A vocal subset of AI researchers believes advanced AI could pose existential risks. Others view these concerns as overblown. The debate is unsettled. Reasonable engineers can disagree

about probability and timeline; the relevant question is what precautions are worth taking given uncertainty.

Practical precautions. Frontier model labs run safety evaluations, red-team for dangerous capabilities, slow-roll deployments to detect issues. International coordination on biosecurity, cyber capabilities, autonomous weapons is increasingly active. Engineers can support these efforts and bring safety concerns into their own work.

16.10 Governance and Documentation

Documentation enables accountability. Three artifacts that have become standard.

Model cards (Mitchell et al., 2019). Document a model: intended use, limitations, evaluation metrics across demographic groups, training data sources, known biases. Published with model releases.

Datasheets for datasets (Gebru et al., 2018). Document a dataset: how it was collected, who is represented, what consents apply, known limitations. Published with dataset releases.

System cards. Document a deployed system: components, model versions, guardrails, evaluation results, monitoring procedures. Audit-friendly.

Internal governance. Ethics review boards, especially for sensitive applications. Pre-launch review of new capabilities. Post-incident review processes. Continuous engagement with affected communities.

External standards. ISO/IEC 42001 (AI management systems), NIST AI Risk Management Framework, EU AI Act compliance, soc 2 type II. Each imposes structure on AI governance; choose based on industry and jurisdiction.

16.11 The Path Ahead

AI capabilities continue to expand. Predicting specifics is hard; broad directions are clearer.

Multimodal becomes default. Text-only models will become a special case. Frontier models in 2026 and beyond will handle text, images, audio, video, code, and structured data natively in both input and output.

Agents become standard. Systems that plan, call tools, and act over extended horizons will move from research to production. Better tool-use, longer effective horizons, more reliable multi-step reasoning.

Cost continues to fall. Inference cost per quality point drops year over year. Open-weights models close on closed-source frontier. On-device models become capable enough for many tasks.

Reasoning becomes ubiquitous. Inference-time compute scaling (chain-of-thought, RL on verifiable rewards) makes deep reasoning available across model sizes. The expensive end-of-2024 reasoning models become the commodity 2027 base.

Personalization at scale. Memory across sessions, learned user preferences, per-user fine-tunes via PEFT. The line between general-purpose assistant and personal assistant blurs.

Engineering matures. LLMOps tooling consolidates. Evaluation becomes more standardized. Production patterns become more codified.

Open questions. How much of human work shifts to AI augmentation versus automation? How are economic gains distributed? How does the role of human judgment change in regulated domains? How do we govern systems that exceed individual human capability? These questions will be answered in the decade ahead, partly by what engineers build and how they build it.

16.12 Closing Thoughts

This book has covered a lot of ground: from the mathematical foundations of neural networks through transformers, multimodal models, prompt engineering, fine-tuning, RAG, vector databases, production infrastructure, evaluation, and ethics. The field is moving fast. Specific architectures, model names, and tools will change. The principles will outlast them.

Gradient descent and the chain rule will still describe how models learn. The attention mechanism will still appear in any sequence-modeling architecture worth deploying. The bias-variance tradeoff will still shape every model selection decision. The cost-quality-latency triangle will still govern production tradeoffs.

Beyond the principles, the most important habit is the one this book has tried to model throughout: combine theoretical understanding with production engineering. Theoretical understanding without engineering produces papers nobody uses. Engineering without theoretical understanding produces systems that fail in surprising ways. The engineers who do the most useful work hold both at once.

The opportunity in front of us is large. AI systems are becoming genuinely useful across more and more domains. The engineering challenges are real but tractable. The ethical challenges are real and we are collectively figuring them out. The work matters. Build well.

Further Reading

- Bender et al., 'On the Dangers of Stochastic Parrots' (2021).
- Bommasani et al., 'On the Opportunities and Risks of Foundation Models' (2021).
- Mitchell et al., 'Model Cards for Model Reporting' (2019).
- Gebru et al., 'Datasheets for Datasets' (2018).
- Hendrycks and Mazeika, 'X-Risk Analysis for AI Research' (2022).
- Anthropic, OpenAI, Google DeepMind safety publications. track current frontier safety research.

Appendix A

Mathematical Notation

Symbols and conventions used throughout the book.

$\mathbb{R}^n$ n -dimensional real-valued vector space

$\mathbb{R}^{m \times n}$ m -by- n matrix of real values

$x \in \mathbb{R}^n$ x is a vector with n components

x_i i -th component of vector x

$\|x\|$ norm of vector x (Euclidean unless otherwise specified)

A^T transpose of matrix A

A^{-1} inverse of matrix A

$\nabla f(x)$ gradient of f at x

$\partial f / \partial x$ partial derivative of f with respect to x

$E[X]$ expected value of random variable X

$P(X = x)$ probability mass function (discrete)

$p(x)$ probability density function (continuous)

$\sigma(x)$ sigmoid function: $1 / (1 + \exp(-x))$

$\text{softmax}(z)$ $\exp(z)$ normalized to sum to 1

$D_{\text{KL}}(P \parallel Q)$ Kullback-Leibler divergence from P to Q

Q, K, V query, key, value matrices in attention

d_k, d_{model} key dimension, model dimension

θ model parameters (generic)

η learning rate

β (alignment) KL coefficient in PPO; temperature in DPO

Appendix B

Glossary

ANN: Artificial Neural Network or Approximate Nearest Neighbor (context-dependent)

Attention: Mechanism weighting inputs by relevance to a query

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2018)

BPE: Byte Pair Encoding, a subword tokenization algorithm

Backpropagation: Algorithm for computing gradients in neural networks via the chain rule

CLIP: Contrastive Language-Image Pretraining (Radford et al., 2021)

Chinchilla scaling: Compute-optimal training: ~20 tokens per parameter

CoT: Chain-of-Thought prompting / reasoning

Cross-entropy: Standard classification loss; $-\log P(\text{true class})$

DPO: Direct Preference Optimization (Rafailov et al., 2023)

Embedding: Dense vector representation of a discrete object

FAISS: Facebook AI Similarity Search library

Fine-tuning: Continuing training of a pretrained model on task-specific data

GAN: Generative Adversarial Network

GPT: Generative Pretrained Transformer

GRPO: Group Relative Policy Optimization (Shao et al., 2024)

HNSW: Hierarchical Navigable Small World graph for ANN search

IVF: Inverted File index for ANN search

KL divergence: Kullback-Leibler divergence: $D_{\text{KL}}(P || Q)$

LLM: Large Language Model

LoRA: Low-Rank Adaptation; a parameter-efficient fine-tuning method

MCP: Model Context Protocol; standard for tool exposure to LLMs

MoE: Mixture of Experts

Multi-Head Attention: Parallel attention computations with different projections

PEFT: Parameter-Efficient Fine-Tuning

PPO: Proximal Policy Optimization (Schulman et al., 2017)

Perplexity: $\exp(\text{per-token cross-entropy})$; a language model evaluation metric

Product Quantization (PQ): Vector compression: split into subvectors, quantize each

RAG: Retrieval-Augmented Generation

RLHF: Reinforcement Learning from Human Feedback

RLVR: Reinforcement Learning with Verifiable Rewards

Ragas: RAG evaluation framework with faithfulness/relevance metrics

Reranking: Re-scoring retrieval candidates with a stronger model

ResNet: Residual Network: deep network with skip connections (He et al., 2015)

RoPE: Rotary Position Embeddings

SFT: Supervised Fine-Tuning

Softmax: $\exp(z)$ normalized to sum to 1; produces probability distribution

Sparse retrieval: Lexical retrieval (BM25, SPLADE)

Tokenization: Splitting text into discrete units (tokens) for model input

Transformer: Attention-based neural architecture (Vaswani et al., 2017)

ViT: Vision Transformer

Word2Vec: Static word embedding method (Mikolov et al., 2013)

qLoRA: Quantized LoRA: 4-bit base model with 16-bit adapters

vLLM: High-throughput LLM serving framework with PagedAttention

Appendix C

Bibliography

Selected references cited throughout the book, in alphabetical order.

- Anthropic, 'Building Effective Agents' engineering blog (2024).
- Bai et al., 'Constitutional AI: Harmlessness from AI Feedback' (2022).
- Bender et al., 'On the Dangers of Stochastic Parrots' (2021).
- Bishop, Pattern Recognition and Machine Learning (2006).
- Brown et al., 'Language Models are Few-Shot Learners' (2020). GPT-3.
- Chen et al., 'Benchmarking Large Language Models in Retrieval-Augmented Generation' (2023). RGB.
- Christiano et al., 'Deep Reinforcement Learning from Human Preferences' (2017).
- DeepSeek-AI, 'DeepSeek-R1' (2025).
- Dettmers et al., 'QLoRA: Efficient Finetuning of Quantized LLMs' (2023).
- Devlin et al., 'BERT: Pre-training of Deep Bidirectional Transformers' (2018).
- Dosovitskiy et al., 'An Image is Worth 16x16 Words' (2020). ViT.
- Edge et al., 'From Local to Global: A Graph RAG Approach' (2024). Microsoft GraphRAG.
- Es et al., 'RAGAS: Automated Evaluation of Retrieval Augmented Generation' (2023).
- Gao et al., 'Precise Zero-Shot Dense Retrieval without Relevance Labels' (2022). HyDE.
- Gebru et al., 'Datasheets for Datasets' (2018).
- Goodfellow, Bengio, Courville, Deep Learning (2016).
- He et al., 'Deep Residual Learning for Image Recognition' (2015). ResNet.
- Ho et al., 'Denoising Diffusion Probabilistic Models' (2020).
- Hoffmann et al., 'Training Compute-Optimal Large Language Models' (2022). Chinchilla.
- Hu et al., 'LoRA: Low-Rank Adaptation of Large Language Models' (2021).
- Jurafsky and Martin, Speech and Language Processing (3rd ed. draft).
- Khattab and Zaharia, 'ColBERT' (2020).
- Khattab et al., 'DSPy: Compiling Declarative LM Calls' (2023).
- Kingma and Welling, 'Auto-Encoding Variational Bayes' (2013). VAE.
- Krizhevsky, Sutskever, Hinton, 'ImageNet Classification with Deep CNNs' (2012). AlexNet.
- Kwon et al., 'PagedAttention for LLM Serving' (2023). vLLM.
- Lewis et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP' (2020).
- Li et al., 'BLIP-2' (2023).
- Liu et al., 'LLaVA: Visual Instruction Tuning' (2023).
- MacKay, Information Theory, Inference, and Learning Algorithms (2003).
- Malkov and Yashunin, 'Efficient ANN Search via Hierarchical Navigable Small World' (2016). HNSW.
- Mikolov et al., 'Distributed Representations of Words and Phrases' (2013). Word2Vec.
- Mitchell et al., 'Model Cards for Model Reporting' (2019).
- Murphy, Probabilistic Machine Learning (2022, 2023).

- Ouyang et al., 'Training Language Models to Follow Instructions' (2022). InstructGPT.
- Pennington, Socher, Manning, 'GloVe' (2014).
- Radford et al., 'Learning Transferable Visual Models from Natural Language' (2021). CLIP.
- Rafailov et al., 'Direct Preference Optimization' (2023).
- Raffel et al., 'Exploring the Limits of Transfer Learning with T5' (2020).
- Reimers and Gurevych, 'Sentence-BERT' (2019).
- Rombach et al., 'High-Resolution Image Synthesis with Latent Diffusion' (2022). Stable Diffusion.
- Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4th ed., 2020).
- Schulman et al., 'Proximal Policy Optimization Algorithms' (2017).
- Shao et al., 'DeepSeekMath' (2024). GRPO.
- Su et al., 'RoFormer: Enhanced Transformer with Rotary Position Embedding' (2021).
- Touvron et al., 'LLaMA: Open and Efficient Foundation Language Models' (2023).
- Vaswani et al., 'Attention Is All You Need' (2017).
- Wei et al., 'Chain-of-Thought Prompting Elicits Reasoning' (2022).
- Zheng et al., 'Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena' (2023).

About the Author

Pavan R

Pavan R is the lead instructor at Proxiant Academy, where he designs and delivers the full curriculum across AI fundamentals, agents, fine-tuning, and retrieval-augmented generation. He works at the intersection of applied research and production engineering, with a focus on making frontier AI techniques practical for working teams. His work spans enterprise RAG systems, multi-agent architectures, alignment engineering, and the operational practices that turn impressive demos into reliable products.

Colophon

Artificial Intelligence, Machine Learning, and Generative AI

First Edition, 2026

Set in Calibri 11pt body, Calibri 28pt chapter titles.

Diagrams produced with matplotlib. Layout assembled with python-docx.

PDF generated via LibreOffice headless conversion.

Published by Proxiant Academy. Reference identifier: PA-AIMLGEN-2026-01.

© 2026 Proxiant Academy. All rights reserved.